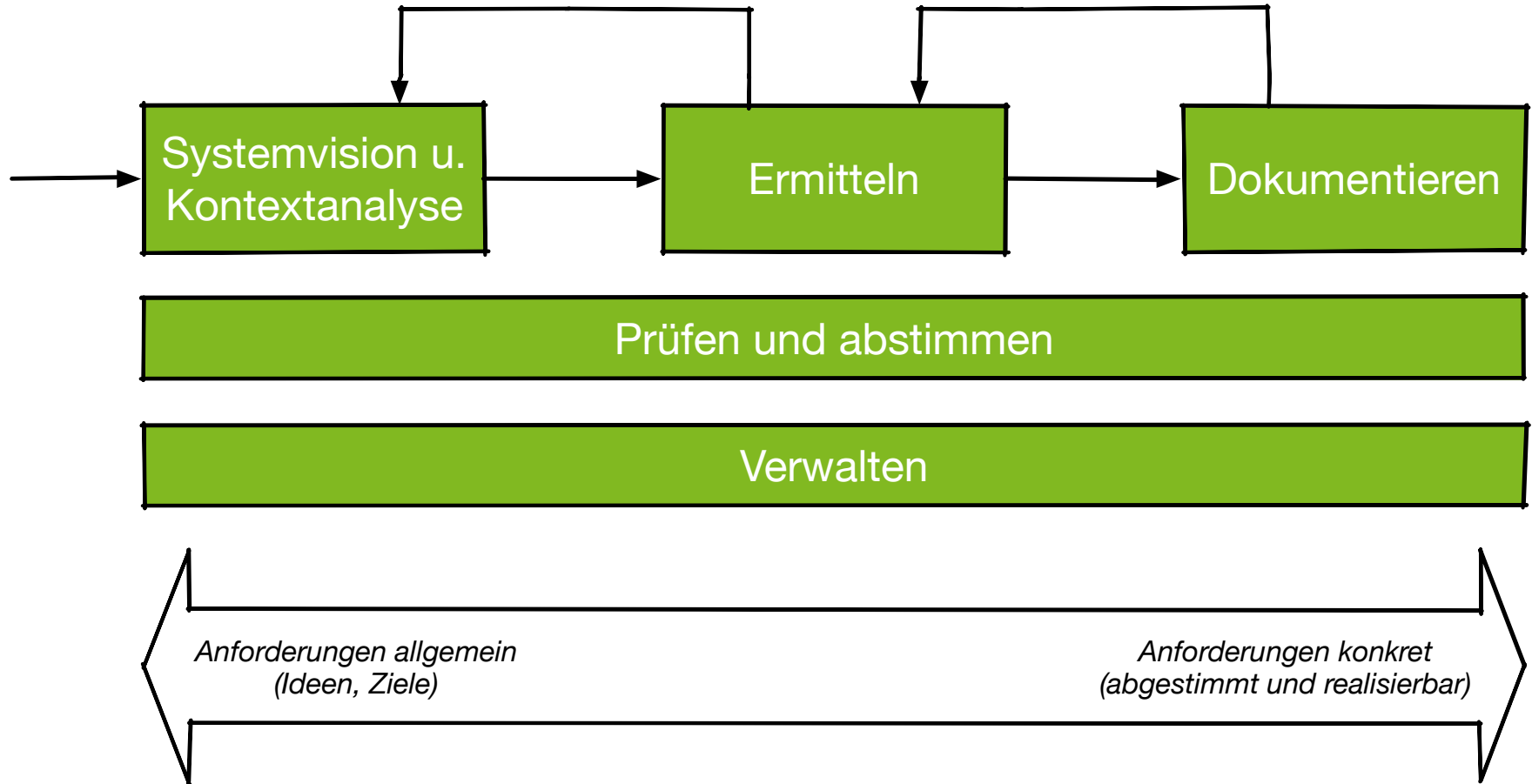


Requirements Engineering

Inhalte bezogen auf die
Prüfung zum IREB® CPRE FL

Aktivitäten des Requirements Engineering



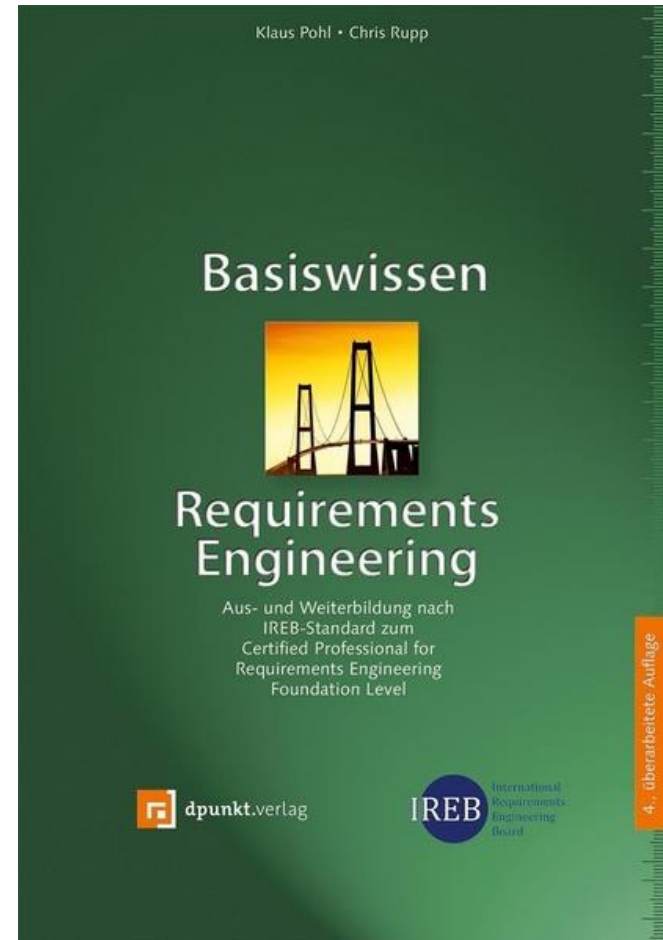
Pohl, K., & Rupp, C. (2011). *Basiswissen Requirements Engineering*. dPunkt Verlag.

Agenda

1. Einleitung und Grundlagen
2. System und Systemkontext abgrenzen
3. Anforderungen ermitteln
4. Anforderungen dokumentieren
5. Anforderungen natürlichsprachig dokumentieren
6. Anforderungen modellbasiert dokumentieren
7. Anforderungen prüfen und abstimmen
8. Anforderungen verwalten
9. Werkzeugunterstützung

Wenn nicht anders angegeben, dann entstammen die Inhalte dem Werk

Pohl, K., & Rupp, C. (2015). *Basiswissen Requirements Engineering: Aus- und Weiterbildung nach IREB-Standard zum Certified Professional for Requirements Engineering Foundation Level* (4 ed.). dpunkt.verlag.



Agenda

1. **Einleitung und Grundlagen**
2. System und Systemkontext abgrenzen
3. Anforderungen ermitteln
4. Anforderungen dokumentieren
5. Anforderungen natürlichsprachig dokumentieren
6. Anforderungen modellbasiert dokumentieren
7. Anforderungen prüfen und abstimmen
8. Anforderungen verwalten
9. Werkzeugunterstützung

1. Einleitung und Grundlagen

Einführung

- Unter Requirements Engineering versteht man das Erheben, Dokumentieren, Prüfen und Verwalten von Anforderungen im Rahmen der Entwicklung eines Systems.
- Fehlerhaftes oder nicht vorhandenes Requirements Engineering führt unter anderem zu Problemen bei der Entwicklung:
 - Fehlende Anforderungen müssen nachträglich implementiert werden.
 - Der Entwickler interpretiert die Anforderung falsch und programmiert das Verkehrte.
 - Der Entwickler muss (mehrmals) Rücksprache halten, was zu einem zeitlichen Mehraufwand führt.
- Ein Fehler, der erst in der Entwicklungsphase korrigiert wird, ist um ein Vielfaches teurer, als ein Fehler, der bereits während des Requirements Engineering behoben wurde.

1. Einleitung und Grundlagen

Einführung

- Symptome für mangelhaftes Requirements Engineering:
 - Anforderungen fehlen
 - Anforderungen sind unvollständig
 - Anforderungen sind mehrdeutig formuliert
- Gründe für mangelhaftes Requirements Engineering:
 - Dinge werden als selbstverständlich vorausgesetzt
 - Unterschiedliche Wissensstände haben unterschiedliche Begriffsverständnisse zur Folge
 - Ergebnisse müssen kurzfristig erzielt werden

1. Einleitung und Grundlagen

Definition: Anforderung

Eine Anforderung ist:

- (1) Eine Bedingung oder Fähigkeit, die von einem Benutzer (Person oder System) zur Lösung eines Problems oder zur Erreichung eines Ziels benötigt wird.
- (2) Eine Bedingung oder Fähigkeit, die ein System oder Teilsystem erfüllen oder besitzen muss, um einen Vertrag, eine Norm, eine Spezifikation oder andere, formell vorgegebene Dokumente zu erfüllen.
- (3) Eine dokumentierte Repräsentation einer Bedingung oder Eigenschaft gemäß (1) oder (2).

1. Einleitung und Grundlagen

Beispiele für Anforderungen

- Der Kampfpanzer muss in der Lage sein Daten vom UAV zu empfangen.
- Das System muss dem Nutzer die Möglichkeit bieten, Artikel in den Warenkorb zu legen.
- Das System muss barrierefrei nutzbar sein.
- Das Sturmgewehr muss bei Schnellfeuer auch nach 150 Schuss Ziele in einer Entfernung über 300 bis 400 Meter ohne Abweichung treffen
- Das System muss den rechtlichen Bestimmungen zum Datenschutz nach EU-DSGVO genügen.
- Die Entwicklung des Systems darf maximal 200 PT in Anspruch nehmen.

1. Einleitung und Grundlagen

Definition: Stakeholder

Ein Stakeholder eines Systems ist eine Person oder Organisation, die (direkt oder indirekt) Einfluss auf die Anforderungen des betrachtenden Systems hat.

1. Einleitung und Grundlagen

Definition: Requirements Engineering

Das Requirements Engineering ist ein systematischer und disziplinierter Ansatz zur Spezifikation und zum Management von Anforderungen mit den folgenden Zielen:

- (1) Die relevanten Anforderungen zu kennen, Konsens unter den Stakeholdern über die Anforderungen herzustellen, die Anforderungen konform zu vorgegebenen Standards zu dokumentieren und die Anforderungen systematisch zu managen.
- (2) Die Wünsche und Bedürfnisse der Stakeholder zu verstehen, zu dokumentieren sowie die Anforderungen zu spezifizieren und zu managen, um das Risiko zu minimieren, dass das System nicht den Wünschen und Bedürfnissen der Stakeholder entspricht.

1. Einleitung und Grundlagen

Haupttätigkeiten des Requirements Engineering

Zu den Haupttätigkeiten des Requirements Engineering zählen:

1. Anforderungen ermitteln
2. Anforderungen dokumentieren
3. Anforderungen prüfen
4. Anforderungen verwalten

Je nach Vorgehensmodell ist das Requirements Engineering eine abgeschlossene Phase (z.B. im Wasserfallmodell oder im V-Modell) oder ein das komplette Entwicklungsprojekt begleitender Prozess (z.B. im eXtreme Programming).

1. Einleitung und Grundlagen

Kommunikation ist das A und O

Gutes Requirements Engineering funktioniert nicht ohne gute Kommunikation.
Voraussetzungen für eine gute Kommunikation sind:

1. Ein gleicher Wissensstand (bspw. erzeugt durch einen Glossar)
2. Die richtige Wahl des Kommunikationsmediums
3. Die vollständige Wissensweitergabe
4. Das richtige Vorwissen voraussetzen

1. Einleitung und Grundlagen

Fähigkeiten eines Requirements Engineers

Der Requirements Engineer nimmt eine zentrale Rolle im Projekt ein. Er sollte für die Wahrnehmung seiner Aufgaben über folgende sieben Fähigkeiten verfügen:

1. Analytisches Denken
2. Empathie
3. Kommunikationsfähigkeit
4. Konfliktlösungsfähigkeit
5. Moderationsfähigkeit
6. Selbstbewusstsein
7. Überzeugungsfähigkeit

1. Einleitung und Grundlagen

Arten von Anforderungen

Es existieren drei verschiedene Arten von Anforderungen:

1. Funktionale Anforderungen
2. Qualitätsanforderungen
3. Randbedingungen

1. Einleitung und Grundlagen

Definition: Funktionale Anforderung

Eine funktionale Anforderung ist eine Anforderung bezüglich des Ergebnisses eines Verhaltens, das von einer Funktion des Systems bereitgestellt werden soll.

Beispiele:

- Das System muss dem Redakteur die Möglichkeit bieten, Kleidungsrubriken anlegen zu können.
- Wenn ein Nutzer einen Artikel anklickt, muss er zur Detailseite dieses Artikels weitergeleitet werden.
- Wenn ein Nutzer einen Artikel in den Warenkorb legt, soll er dazu passende Artikel als Empfehlung angezeigt bekommen.

1. Einleitung und Grundlagen

Definition: Qualitätsanforderung

Eine Qualitätsanforderung ist eine Anforderung, die sich auf ein Qualitätsmerkmal bezieht, das nicht durch funktionale Anforderungen abgedeckt wird.

Beispiele:

- Das Content-Management-System muss dem Redakteur die Möglichkeit bieten, Inhalte ohne technische Kenntnisse anlegen und pflegen zu können.
- Eine Detailseite muss innerhalb von 2 Sekunden geladen sein.
- Der Online-Shop muss auf sämtlichen gängigen Endgeräten benutzbar sein.

1. Einleitung und Grundlagen

Definition: Randbedingung

Eine Randbedingung ist eine Anforderung, die den Lösungsraum jenseits dessen einschränkt, was notwendig ist, um die funktionalen Anforderungen und Qualitätsanforderungen zu erfüllen.

Beispiele:

- Die Entwicklung des Systems darf maximal 200 Personentage in Anspruch nehmen.
- Die Entwicklung des Systems muss bis zum xx.xx.xxxx abgeschlossen sein.
- Gesetzliche Bestimmungen (bspw. Telemediengesetz)

1. Einleitung und Grundlagen

Vertiefung: Qualitätsanforderungen

- Qualitätsanforderungen werden in der Praxis oftmals nur unzureichend dokumentiert.
- Qualitätsanforderungen sollten stets getrennt von funktionalen Anforderungen dokumentiert werden.
- Mögliche Kategorien für Qualitätsanforderungen sind:
 - Angemessenheit, Sicherheit, Genauigkeit der Berechnung, Interoperabilität
 - Zuverlässigkeit
 - Benutzbarkeit
 - Effizienz
 - Änderbarkeit
 - Übertragbarkeit

1. Einleitung und Grundlagen

Beispiele für Anforderungen

FA Der Kampfpanzer muss in der Lage sein Daten vom UAV zu empfangen.

FA Das System muss dem Nutzer die Möglichkeit bieten, Artikel in den Warenkorb zu legen.

QA Das System muss barrierefrei nutzbar sein.

QA Das Sturmgewehr muss bei Schnellfeuer auch nach 150 Schuss Ziele in einer Entfernung über 300 bis 400 Meter ohne Abweichung treffen

RB Das System muss den rechtlichen Bestimmungen zum Datenschutz nach EU-DSGVO genügen.

RB Die Entwicklung des Systems darf maximal 200 PT in Anspruch nehmen.

1. Einleitung und Grundlagen

Zusammenfassung

- Unter Requirements Engineering versteht man das Erheben, Dokumentieren und Verwalten von Anforderungen im Rahmen der Entwicklung eines Systems.
- Fehlerhaftes oder nicht vorhandenes Requirements Engineering führt unter anderem zu Problemen bei der Entwicklung.
- Gutes Requirements Engineering funktioniert nur mit guter Kommunikation.
- Es gibt unterschiedliche Anforderungen (Funktionale Anforderungen, Qualitätsanforderungen, Randbedingungen).
- Der Requirements Engineer muss über verschiedene Fähigkeiten verfügen (bspw. analytisches Denken und Konfliktlösungsfähigkeit).

Agenda

1. Einleitung und Grundlagen
2. **System und Systemkontext abgrenzen**
3. Anforderungen ermitteln
4. Anforderungen dokumentieren
5. Anforderungen natürlichsprachig dokumentieren
6. Anforderungen modellbasiert dokumentieren
7. Anforderungen prüfen und abstimmen
8. Anforderungen verwalten
9. Werkzeugunterstützung

2. System und Systemkontext abgrenzen

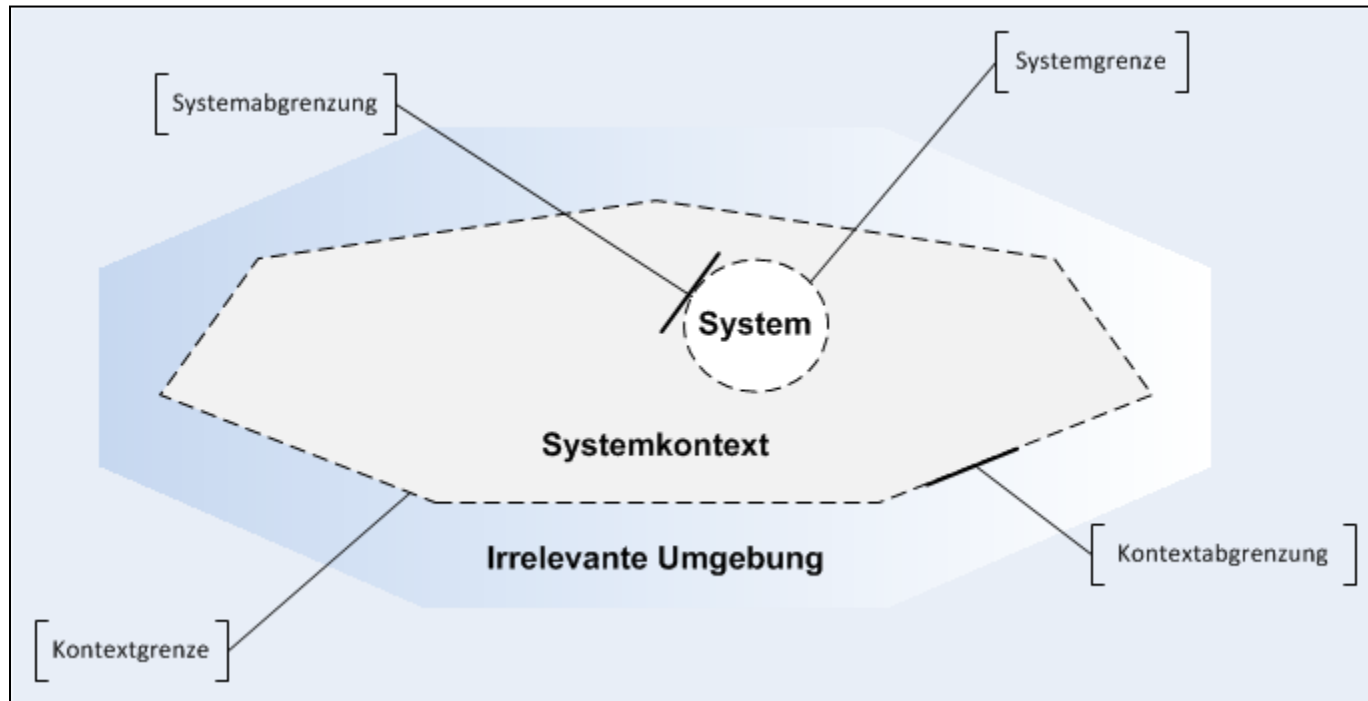
Systemkontext

- Systeme stehen in der Regel nicht für sich alleine, sondern interagieren mit anderen Systemen. Zur Ermittlung der Anforderungen muss daher abgegrenzt werden, was zum System und seiner Umgebung (Systemkontext) gehört und was nicht.
- Wichtig ist, die Abgrenzung in der Soll-Perspektive vorzunehmen (und nicht in der Ist-Perspektive).
- Folgende Aspekte können sich im Systemkontext befinden: Personen, Systeme im Betrieb, Prozesse, Ereignisse und Dokumente.
- Erste Anforderungen ergeben sich durch diese Aspekte und müssen immer im jeweiligen Kontext definiert werden.

2. System und Systemkontext abgrenzen

Definition: Systemkontext

Der Systemkontext ist der Teil der Umgebung eines Systems, der für die Definition und das Verständnis der Anforderungen des betrachteten Systems relevant ist.



2. System und Systemkontext abgrenzen

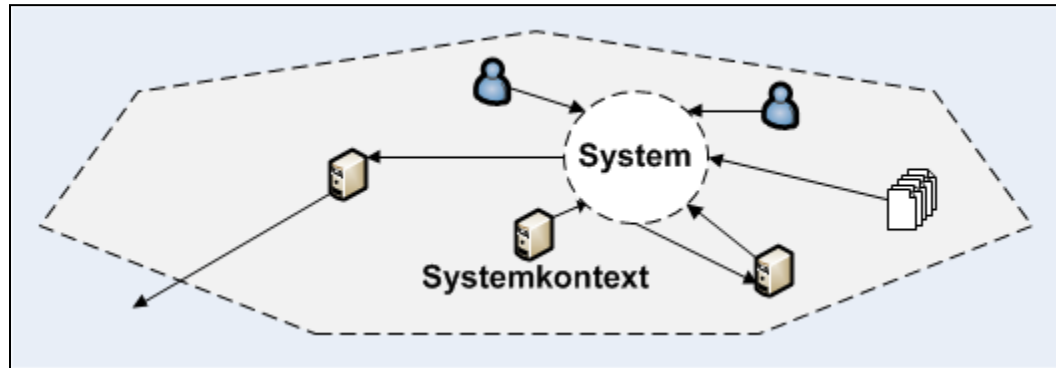
System- und Kontextgrenzen bestimmen

- Die Definition des Systemkontexts sollte vom Requirements Engineering durchgeführt werden.
- Im Rahmen der Systemabgrenzung wird die Grenze festgelegt, die aufzeigt, welche Aspekte zum System gehören und welche Aspekte zum Systemkontext.
- Im Rahmen der Kontextabgrenzung wird bestimmt, welche Aspekte (also bspw. welche anderen Systeme oder welche Prozesse) eine Beziehung zum geplanten System haben.

2. System und Systemkontext abgrenzen

Definition: Systemgrenze festlegen

Die Systemgrenze separiert das geplante System von seiner Umgebung. Sie grenzt den im Rahmen des Entwicklungsprozesses gestaltbaren und veränderbaren Teil der Realität von Aspekten in der Umgebung ab, die durch den Entwicklungsprozess nicht verändert werden.



2. System und Systemkontext abgrenzen

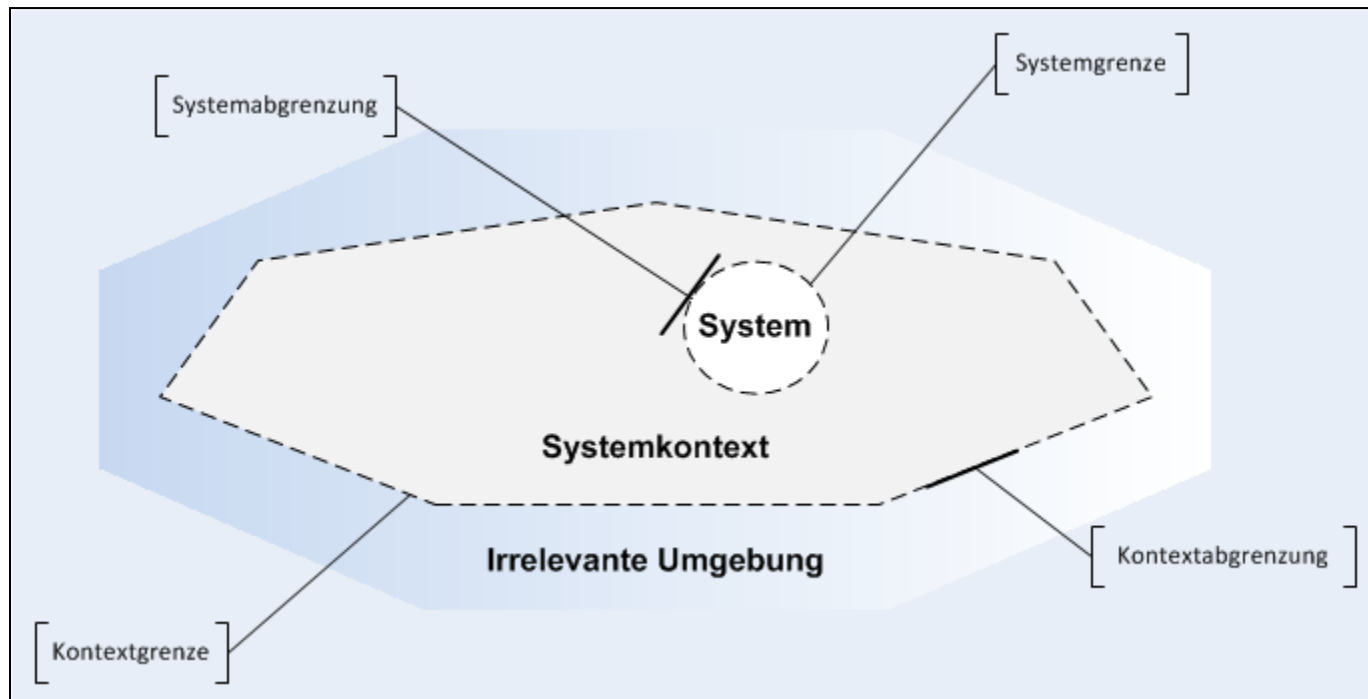
Systemgrenze festlegen

- Ausgangspunkt der Abgrenzung sind Schnittstellen. Diese lassen sich in Quellen (Input) und Senken (Output) unterteilen.
- Mögliche Quellen und Senken eines Systems sind:
 - Stakeholder
 - Bestehende Systeme
- Teilweise lässt sich die exakte Systemgrenze erst im Nachhinein festlegen. Bis dahin existiert eine Grauzone, die berücksichtigt werden muss.

2. System und Systemkontext abgrenzen

Definition: Kontextgrenze festlegen

Die Kontextgrenze separiert den relevanten Teil der Umgebung eines geplanten Systems vom irrelevanten Teil, d.h. dem Teil der Umgebung, der keinen Einfluss auf das geplante System und damit auch keinen Einfluss auf die Anforderungen dieses Systems hat.



2. System und Systemkontext abgrenzen

Kontextgrenze festlegen

- Auch die Kontextgrenze lässt sich unter Umständen nicht von Beginn an klar festlegen und muss nach und nach konkretisiert werden.
- Da die Kontextgrenze den Systemkontext von der gesamten irrelevanten Realität abgrenzt, ist eine vollständige und präzise Kontextabgrenzung für einzelne Systeme praktisch nicht möglich.
- Die Grauzone der Kontextabgrenzung umfasst somit identifizierte Aspekte der Umgebung, für die unklar ist, ob sie eine Beziehung zum System haben oder nicht.

2. System und Systemkontext abgrenzen

Systemkontext dokumentieren

- Zur Dokumentation des Systemkontexts eignen sich insbesondere Use-Case- oder Datenflussdiagramme.
- Gegebenenfalls macht es Sinn, mittels unterschiedlicher Dokumentationsformen einen möglichst vollständiges Bild zu generieren.

2. System und Systemkontext abgrenzen

Zusammenfassung

- Der Systemkontext ist der Teil der Realität, der Einfluss auf das zu entwickelnde System und damit auf die Anforderungen hat.
- Folgende Aspekte können sich im Systemkontext befinden: Personen, Systeme im Betrieb, Prozesse, Ereignisse und Dokumente.
- Wichtig ist, die Abgrenzung in der Soll-Perspektive vorzunehmen (und nicht in der Ist-Perspektive).
- Im Rahmen der Systemabgrenzung wird die Grenze festgelegt, die aufzeigt, welche Aspekte zum System gehören und welche Aspekte zum Systemkontext.
- Im Rahmen der Kontextabgrenzung wird bestimmt, welche Aspekte (also bspw. welche anderen Systeme oder welche Prozesse) eine Beziehung zum geplanten System haben.

Agenda

1. Einleitung und Grundlagen
2. System und Systemkontext abgrenzen
3. **Anforderungen ermitteln**
4. Anforderungen dokumentieren
5. Anforderungen natürlichsprachig dokumentieren
6. Anforderungen modellbasiert dokumentieren
7. Anforderungen prüfen und abstimmen
8. Anforderungen verwalten
9. Werkzeugunterstützung

3. Anforderungen ermitteln

Anforderungsquellen

- Stakeholder
- Dokumente
- Systeme in Betrieb

3. Anforderungen ermitteln

Stakeholder

- Die Identifikation der richtigen Stakeholder ist ein wesentlicher Erfolgsfaktor.
- Werden Stakeholder (und somit auch ihre Anforderungen) nicht berücksichtigt, wird dies zwangsläufig zu zahlreichen Änderungsanträgen im Livebetrieb führen.
- Die Suche nach relevanten Stakeholdern sollte systematisch durchgeführt werden. Ausgangspunkt können initiale Stakeholder sein.
- Unternehmensspezifische, aber projektunabhängige Checklisten könnten einen standardisierten Weg zur Stakeholderidentifikation sein.

3. Anforderungen ermitteln

Stakeholder

- Für die Verwaltung der Stakeholder eignet sich eine Tabelle mit u.a. folgenden Daten:
 - Name
 - Funktion (Rolle)
 - Kontaktdaten
 - Zeitliche und räumliche Verfügbarkeit
 - Relevanz des Stakeholders
 - Wissensgebiet
 - Eigene Ziele und Interessen in Bezug auf das Projekt

3. Anforderungen ermitteln

Stakeholder

- Nicht alle Stakeholder sind intrinsisch motiviert. Um alle Stakeholder „mit ins Boot zu holen“, sollte ein kontinuierlicher Informationsfluss gewährleistet werden; regelmäßige Statusbesprechungen helfen, aus Projektbetroffenen Projektbeteiligte zu machen.
- Stakeholder, die nicht zureichend integriert werden, könnten sich dem Projekt kritisch entgegenstellen
- Die Zusammenarbeit kann ggf. mündlich oder schriftlich fixiert werden:
 - Aufgaben
 - Verantwortungsbereiche
 - Weisungsbefugnisse
 - Individuelle Ziele
 - Kommunikationswege
 - Feedback-Schleifen

3. Anforderungen ermitteln

Stakeholder

- Aus der Stakeholdervereinbarung resultieren unterschiedliche Pflichten.
- Der Requirements Engineer
 - Spricht die Sprache der Stakeholder
 - Arbeitet sich gründlich in das jeweilige Fachgebiet ein
 - Erstellt ein Anforderungsdokument
 - Kann die Arbeitsergebnisse verständlich machen
 - Pflegt einen respektvollen Umgang mit den Stakeholdern
 - Präsentiert seine Ideen und Alternativen und deren Realisierung
 - Sorgt dafür, dass das spezifizierte System den funktionalen und qualitativen Ansprüchen der Stakeholder gerecht wird

3. Anforderungen ermitteln

Stakeholder

- Der Stakeholder
 - Führt den Requirements Engineer in das Fachgebiet ein
 - Versorgt den Requirements Engineer mit Anforderungen
 - Formuliert die Anforderungen zielgerecht und gewissenhaft
 - Trifft Entscheidungen zeitgerecht
 - Respektiert die Einschätzung der Kosten und Machbarkeit des Requirements Engineer
 - Priorisiert die Anforderungen
 - Überprüft die dokumentierten Anforderungen des Requirements Engineer
 - Kommuniziert unverzüglich Änderungen in den Anforderungen
 - Befolgt den vorgegebenen Änderungsprozess
 - Respektiert das vorgegebene Requirements Engineering

3. Anforderungen ermitteln

Anforderungskategorisierung

- Für die Anforderungsermittlung ist das Wissen, welche Bedeutung die Anforderungen für die Zufriedenheit der Stakeholder haben, hilfreich.
- Die Zufriedenheit kann nach Kano wie folgt klassifiziert werden:
 - Basisfaktoren (als selbstverständlich vorausgesetzte Merkmale)
 - Leistungsfaktoren (explizit geforderte Merkmale)
 - Begeisterungsfaktoren (Merkmale, die der Stakeholder nicht kennt und erst während der Benutzung als angenehme und nützliche Überraschung kennenlernt)
- Im Laufe der Zeit werden aus Begeisterungsfaktoren Leistungsfaktoren und hieraus wiederum Basisfaktoren.

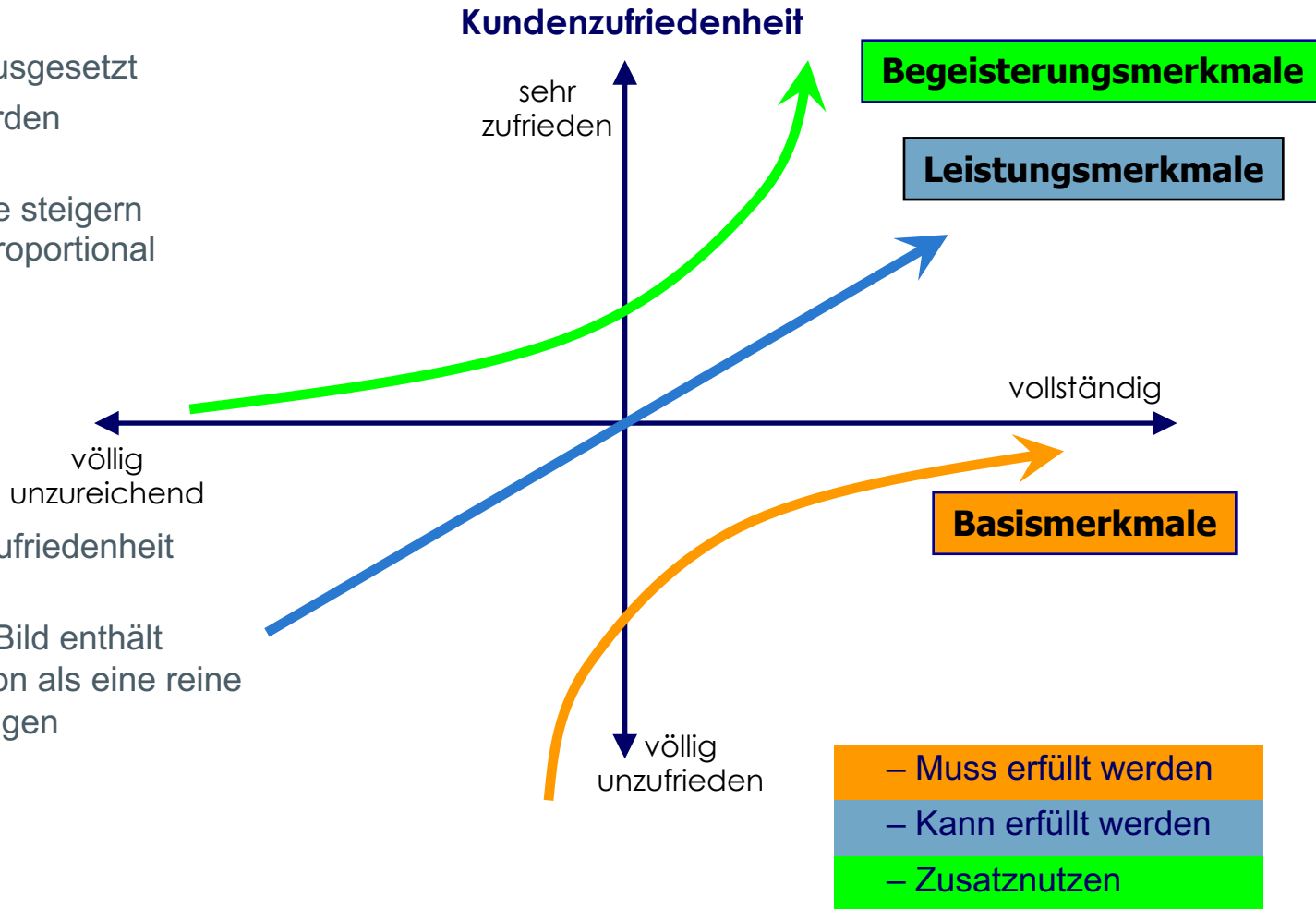
3. Anforderungen ermitteln

Das Kano-Modell

- Basismerkmale werden selbstverständlich vorausgesetzt
- Leistungsmerkmale werden explizit gefordert
- Begeisterungsmerkmale steigern die Zufriedenheit überproportional

Kriterienabdeckung (Erfüllungsgrad)

- Das Gegenteil von Unzufriedenheit ist nicht Zufriedenheit
- Das zweidimensionale Bild enthält deutlich mehr Information als eine reine Priorität von Anforderungen



(c) Beneken & Hammerschall.

Online-Unterlagen aus Hammerschall, U., & Beneken, G. (2013). Software Requirements (1 ed.). Pearson Studium.

3. Anforderungen ermitteln

Anforderungskategorisierung

- Basisfaktoren müssen in jedem Fall erfüllt werden, um die Stakeholder zufrieden zu stellen.
- Leistungsfaktoren sollten erfüllt werden; fehlen einzelne Leistungsfaktoren wird das Produkt zwar aller Voraussicht nach dennoch vom Stakeholder akzeptiert, aber mit jedem fehlenden Leistungsfaktor sinkt die Zufriedenheit.
- Begeisterungsfaktoren steigern die Zufriedenheit um ein Vielfaches.

3. Anforderungen ermitteln

Ermittlungstechniken

- Mittels der Ermittlungstechniken können die bewussten, unbewussten und unterbewussten Anforderungen der Stakeholder ermittelt werden.
- Es existiert keine allgemeingültige Ermittlungstechnik; vielmehr müssen unterschiedliche Faktoren berücksichtigt werden:
 - Die Unterscheidung nach bewussten, unbewussten und unterbewussten Anforderungen, die ermittelt werden sollen
 - Die Termin- und Budgetvorgaben sowie die Verfügbarkeit relevanter Stakeholder
 - Die Erfahrung des Requirements Engineer mit der entsprechenden Ermittlungstechnik
 - Angestrebter Detaillierungsgrad
 - Die Chancen und Risiken des Projekts

3. Anforderungen ermitteln

Ermittlungstechniken

- Mögliche Risikofaktoren:
 - Menschliche Einflüsse
 - Organisatorische Einflüsse
 - Fachliche inhaltliche Einflüsse
- Letztlich empfiehlt es sich, die Techniken situationsbedingt zu kombinieren.
- Ermittlungstechniken lassen sich folgendermaßen gruppieren:
 - Befragungstechniken
 - Kreativitätstechniken
 - Dokumentenzentrierte Techniken
 - Beobachtungstechniken
 - Unterstützende Techniken

3. Anforderungen ermitteln

Befragungstechniken

- Mittels der Befragungstechniken direkt vom Stakeholder eine möglichst genaue und unverfälschte Aussage über seine Anforderungen an das System zu erhalten.
- Befragungen können in zwei Varianten erfolgen:
 - Interview
 - Vorteil: Der Interviewer kann direkt mit dem Stakeholder interagieren.
 - Nachteil: Hoher Zeitaufwand
 - Fragebogen
 - Vorteil: Viele Stakeholder können zeitgleich befragt werden.
 - Nachteil: Keine Interaktion möglich

3. Anforderungen ermitteln

Kreativitätstechniken

- Kreativitätstechniken eignen sich vorwiegend für die Entwicklung innovativer Anforderungen wie bspw. Begeisterungsfaktoren.
- Es existieren vier verschiedene Varianten:
 - Brainstorming
 - Vorgehen: In einer Gruppe von 5 bis 10 Personen werden Vorschläge oder Ideen gesammelt, die ohne Bewertung oder Kommentierung notiert werden. Aus diesen Vorschlägen werden wiederum neue Ideen generiert usw.
 - Vorteil: In kurzer Zeit können viele Ideen entwickelt werden.
 - Nachteil: Ggf. schwierige Umsetzung bei ungünstigen Gruppendynamiken

3. Anforderungen ermitteln

Kreativitätstechniken

- Fortsetzung Varianten Kreativitätstechniken:
 - Brainstorming paradox
 - Vorgehen: Analog zum Brainstorming, aber mit der Vorgabe, Ziele zu sammeln, die NICHT erreicht werden sollen. Anschließend werden Maßnahmen entwickelt, wie diese „Nicht-Ziele“ verhindert werden können.
 - Vorteil: Den Teilnehmern wird bewusst, welche Aktivitäten zu negativen Resultaten führen können.
 - Nachteil: Ggf. schwierige Umsetzung bei ungünstigen Gruppendynamiken

3. Anforderungen ermitteln

Kreativitätstechniken

- Fortsetzung Varianten Kreativitätstechniken:
 - Perspektivenwechsel
 - Vorgehen: Es werden verschiedene Sichtweisen definiert (bspw. „Sechs-Hut-Denken“). Jede Sichtweise wird von jedem Teilnehmer einmal eingenommen. Aus dieser Sichtweise heraus muß dann diskutiert werden.
 - Vorteil: Gut geeignet, wenn Stakeholder eine eingeeengte Sichtweise besitzen.
 - Nachteil: Es wird keine tiefe Detaillierungsebene erreicht (oder nur mit viel Aufwand).

3. Anforderungen ermitteln

Kreativitätstechniken

- Fortsetzung Varianten Kreativitätstechniken:
 - Analogietechnik
 - Vorgehen: Bei der Analogietechnik werden ähnliche Strukturen (bspw. in der Natur) gesucht.
 - Vorteil: Eröffnet neue Perspektiven
 - Nachteil: Es wird keine tiefe Detaillierungsebene erreicht (oder nur mit viel Aufwand).

3. Anforderungen ermitteln

Dokumentenzentrierte Techniken

- Dokumentenzentrierte Techniken verwenden Lösungen und Erfahrungen bestehender Systeme wieder.
- Es empfiehlt sich, diese Techniken mit anderen Ermittlungstechniken zu kombinieren, um die Gültigkeit der ermittelten Anforderungen zu bestimmen und um neue Anforderungen an das zu entwickelnde System herauszufinden.
- Folgende drei dokumentenzentrierte Techniken existieren:
 - Systemarchäologie
 - Perspektivenbasiertes Lesen
 - Wiederverwendung

3. Anforderungen ermitteln

Dokumentenzentrierte Techniken

- Charakterisierung der einzelnen Techniken:
 - Systemarchäologie
 - Vorgehen: Mit der Systemarchäologie werden Informationen bezüglich eines neuen Systems aus der Dokumentation eines Systems oder des Systems selbst identifiziert.
 - Vorteil: Das Verfahren führt zu vielen sehr detaillierten Anforderungen und ist die einzige Ermittlungstechnik, die eine vollständige Umsetzung von Alt- in Neusysteme gewährleistet.
 - Nachteil: Das Verfahren ist sehr zeitaufwendig.

3. Anforderungen ermitteln

Dokumentenzentrierte Techniken

- Fortsetzung Charakterisierung der einzelnen Techniken:
 - Perspektivenbasiertes Lesen
 - Vorgehen: Diese Technik wird genutzt, um ein Dokument aus einer bestimmten Perspektive zu lesen, beispielsweise aus der Perspektive der Entwicklung oder der Qualitätssicherung.
 - Vorteil: Es wird eine fokussierte Analyse ermöglicht.
 - Nachteil: Das Verfahren ist sehr zeitaufwendig.

3. Anforderungen ermitteln

Dokumentenzentrierte Techniken

- Fortsetzung Charakterisierung der einzelnen Techniken:
 - Wiederverwendung
 - Vorgehen: Immer wiederkehrende Anforderungen werden in einer Datenbank gespeichert.
 - Vorteil: Durch die Wiederverwendung können Kosten gespart werden.
 - Nachteil: Keiner

3. Anforderungen ermitteln

Beobachtungstechniken

- Wenn Stakeholder nicht die Zeit besitzen, ihr Wissen rein kommunikativ weiterzugeben, können sich Beobachtungstechniken eignen.
- Der Requirements Engineer beobachtet die Stakeholder bei ihrer Arbeit und dokumentiert die einzelnen Arbeitsschritte.
- Zudem können direkt potentielle Fehler und Risiken identifiziert werden.
- Es können gezielte Anforderungen dokumentiert werden.
- Es existieren folgende zwei Varianten:
 - Feldbeobachtung
 - Apprenticing

3. Anforderungen ermitteln

Beobachtungstechniken

- Charakterisierung der einzelnen Techniken
 - Feldbeobachtung
 - Vorgehen: Der Requirements Engineer ist vor Ort und beobachtet und dokumentiert die unmittelbar stattfindenden Tätigkeiten.
 - Vorteil: Diese Technik eignet sich gut bei sprachlich schwer vermittelbaren Abläufen.
 - Nachteil: Diese Technik eignet sich nur bei wirklich beobachtbaren Abläufen.
 - Apprenticing
 - Vorgehen: Der Requirements Engineer erlernt die Tätigkeit des Stakeholders und führt sie selbst aus.
 - Vorteil: Der Requirements Engineer kann Anforderungen erfahren, die für den Stakeholder selbstverständlich geworden sind. Zudem wird das Machtverhältnis zwischen Requirements Engineer und Stakeholder umgedreht.
 - Nachteil: Das Verfahren ist sehr zeitaufwendig.

3. Anforderungen ermitteln

Unterstützende Techniken

- Es existieren zahlreiche unterstützende Techniken:
 - Mindmapping: Anlegen einer grafischen Darstellung mit Verfeinerungsbeziehungen und Abhängigkeiten zwischen Begriffen.
 - Workshops: Zusammentreffen von mehreren Beteiligten zur gemeinsamen Diskussion.
 - CRC-Karten: Bei der CRC-Karten-Technik (Class Responsibility Collaboration) werden Kontextaspekte genommen und auf einer Karteikarte deren jeweilige Eigenschaften und Beziehungen notiert. Hieraus können dann Anforderungen ermittelt werden.
 - Audio- und Videoaufzeichnung: Hierdurch können Abläufe oder Gespräche dokumentiert und dem Requirements Engineer zeit- und ortsunabhängig zur Verfügung gestellt werden.
 - Use-Case-Modellierung: Mit den Use Cases wird die Außensicht des zu erstellenden Systems dokumentiert.
 - Prototypen: Prototypen werden zum Erproben bereits vorhandener Anforderungen verwendet.

- Ähnlich zu Eisbergen, existiert zu jeder Information ein überdimensional großer Bereich an Zusatzinformationen „unter der Wasseroberfläche“:
 - Selbstverständliches
 - Vergessenes
 - Verdrängtes
 - ...
- Diese Informationen gilt es bei der Ermittlung zu heben (wie einen Schatz).



3. Anforderungen ermitteln

Zusammenfassung

- Es existieren drei Anforderungsquellen: Stakeholder, Dokumente und Systeme in Betrieb
- Anforderungen können folgendermaßen kategorisiert werden: Basisfaktoren, Leistungsfaktoren und Begeisterungsfaktoren
- Es existieren folgende Ermittlungstechniken:
 - Befragungstechniken
 - Kreativitätstechniken
 - Dokumentenzentrierte Techniken
 - Beobachtungstechniken
 - Unterstützende Techniken

Agenda

1. Einleitung und Grundlagen
2. System und Systemkontext abgrenzen
3. Anforderungen ermitteln
4. **Anforderungen dokumentieren**
5. Anforderungen natürlichsprachig dokumentieren
6. Anforderungen modellbasiert dokumentieren
7. Anforderungen prüfen und abstimmen
8. Anforderungen verwalten
9. Werkzeugunterstützung

4. Anforderungen dokumentieren

Definition: Anforderungsdokument / Anforderungsspezifikation

- Anforderungsspezifikation (Pohl, Rupp, 2011)
 - Eine systematisch dargestellte Sammlung von Anforderungen (typischerweise für ein System oder eine Komponente), die vorgegebenen Kriterien genügt.
- Lastenheft (DIN 69901 – 5):
 - Vom Auftraggeber festgelegte Gesamtheit der Forderungen an die Lieferung und Leistungen eines Auftragnehmers innerhalb eines Auftrags.
- Pflichtenheft (DIN 69901 – 5):
 - Das vom Auftragnehmer erarbeitete Realisierungsvorhaben aufgrund der Umsetzung des vom Auftraggeber vorgegebenen Lastenhefts

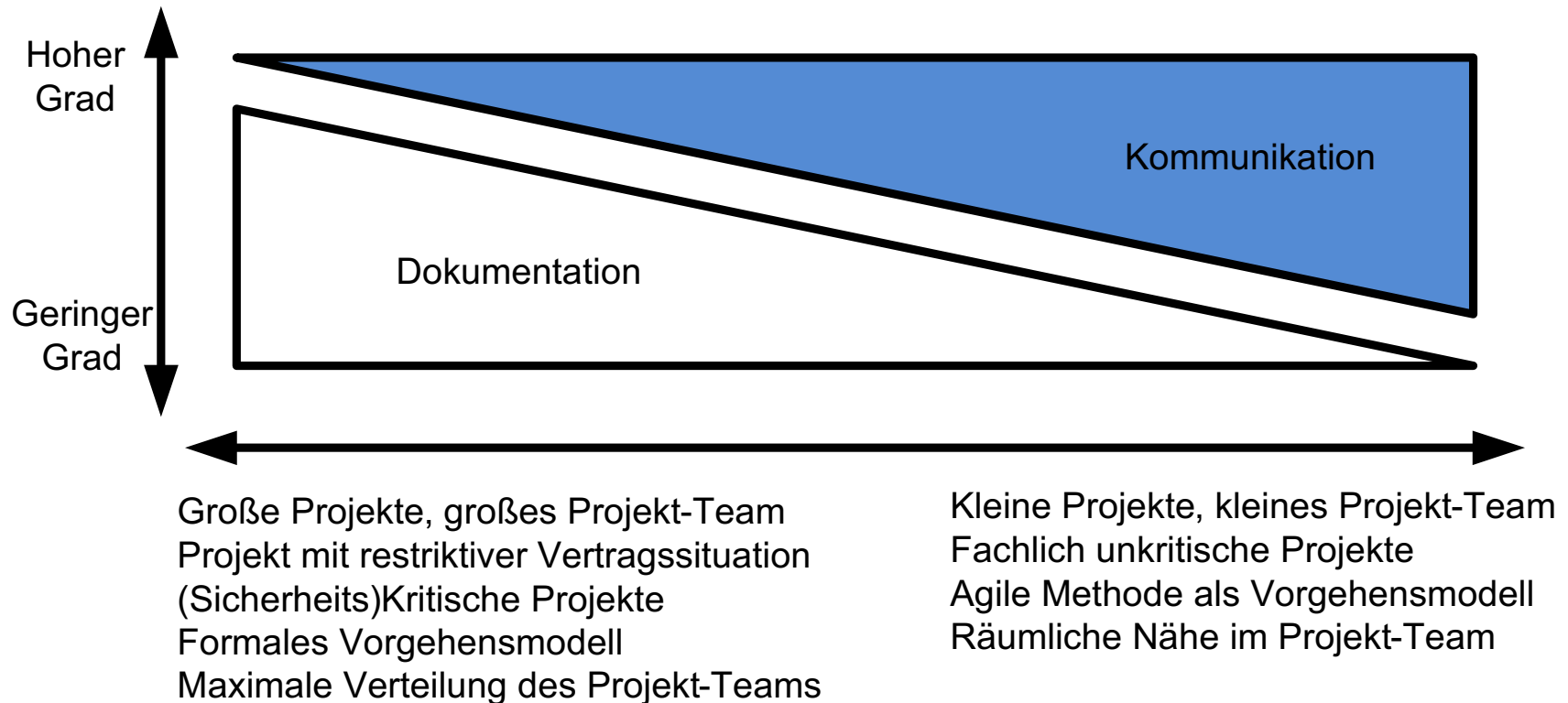
4. Anforderungen dokumentieren

Dokumentgestaltung

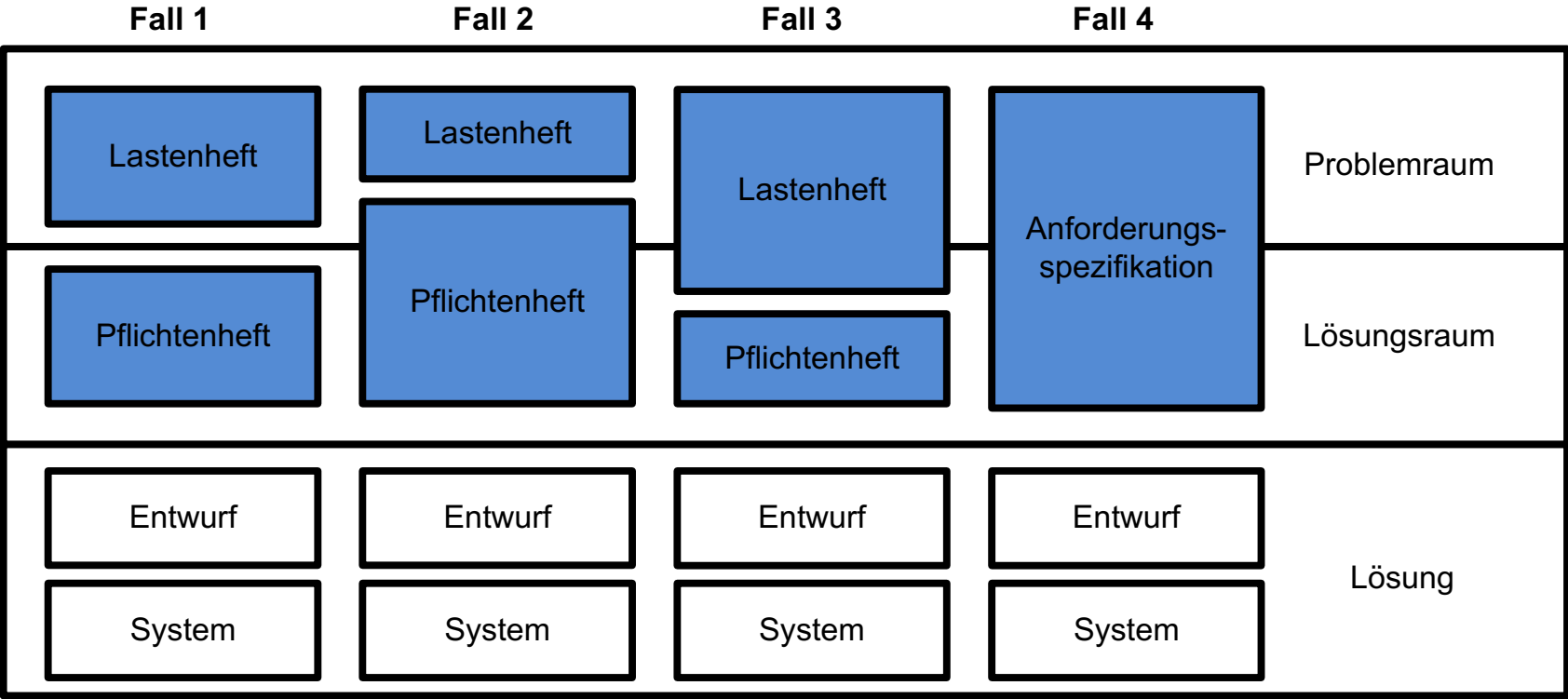
- Prinzipiell sind angefangen von natürlichsprachiger Beschreibung in Prosaform über strukturierten natürlichsprachigen Text bis hin zu formalen Techniken (z.B. Modellierungen) alle Dokumentationstechniken zur Dokumentation von Anforderungen einsetzbar.
- Die wesentlichen Gründe für die Dokumentation von Anforderungen sind:
 - Anforderungen sind die Basis für die Systementwicklung
 - Anforderungen sind rechtlich relevant
 - Anforderungsdokumente sind komplex
 - Anforderungen sollten allen Beteiligten zugänglich sein

Dokumentation vs. Kommunikation

- Wie viel Dokumentation ist notwendig?
- Wie viel Kommunikation ist möglich?



Abgrenzung zw. Lastenheft u. Pflichtenheft



(c) Beneken & Hammerschall.
Online-Unterlagen aus Hammerschall, U., & Beneken, G. (2013). Software Requirements (1 ed.). Pearson Studium.

Abgrenzung zw. Lastenheft u. Pflichtenheft

- Fall 1: LH bezieht sich auf den Problemraum, PH bezieht sich auf den Lösungsraum.
- Fall 2: Der Auftraggeber verfügt nicht über genug Know-how, um eine ausreichende Anforderungsanalyse vorzunehmen.
- Fall 3: der Auftraggeber möchte Teile der Lösung explizit vorgeben.
- Fall 4: Auftraggeber und Auftragnehmer arbeiten gemeinsam an einem Anforderungsdokument.

(c) Beneken & Hammerschall.

Online-Unterlagen aus Hammerschall, U., & Beneken, G. (2013). Software Requirements (1 ed.). Pearson Studium.

4. Anforderungen dokumentieren

Arten der Dokumentation

- Es gibt drei verschiedene Perspektiven:
 - Strukturperspektive
 - Funktionsperspektive
 - Verhaltensperspektive
- Möglichkeiten der Ausgestaltung:
 - Natürliche Sprache: Universell einsetzbar und für jeden verständlich, allerdings Gefahr der Mehrdeutigkeit.
 - Konzeptuelle Modelle: Nicht universell einsetzbar, Leser muss die Notation beherrschen. Dafür eindeutiger durch höheren Grad der Formalisierung.
 - Mischformen

4. Anforderungen dokumentieren

Dokumentenstrukturen

- Durch die Verwendung von Standardgliederungen wird eine Grobstruktur für das Dokument vorgegeben.
- Die Verwendung von Standardgliederungen hat folgende Vorteile:
 - Standardgliederungen erleichtern die Einarbeitung neuer Mitarbeiter.
 - Standardgliederungen ermöglichen eine schnellere Erfassung ausgewählter Inhalte.
 - Standardgliederungen ermöglichen selektives Lesen / Überprüfen von Anforderungsdokumenten.
 - Standardgliederungen ermöglichen eine automatische Überprüfung von Anforderungsdokumenten (z.B. auf Vollständigkeit).
 - Standardgliederungen ermöglichen eine einfache Wiederverwendung von Inhalten in Anforderungsdokumenten.

4. Anforderungen dokumentieren

Dokumentenstrukturen

- Die drei verbreitetsten Varianten von Standardgliederungen sind:
 - Der Rational Unified Process (RUP)
 - Der Auftraggeber erstellt ein Business Model (Geschäftsregeln, Geschäftsanwendungsfälle, Geschäftsziele). Dieses dient als Basis für die Anforderungen an das System
 - Der Auftragnehmer nutzt die Strukturen der Software Requirements Specification (SRS), um die Gesamtheit der Anforderungen zu dokumentieren.
 - Der IEEE-Standard 830-1998
 - Enthält eine Gliederung, die speziell für die Dokumentation von Softwareanforderungen entwickelt worden ist.
 - V-Modell
 - Lastenheft („was“ und „wofür“) und Pflichtenheft („wie“).

4. Anforderungen dokumentieren

Dokumentenstrukturen

- Gliederungsvorschlag:
 - Einleitung
 - Zweck, Systemumfang, Stakeholder, Definitionen, Akronyme und Abkürzungen, Referenzen, Übersicht
 - Allgemeine Übersicht
 - Systemumfeld, Architekturbeschreibungen, Systemfunktionalität, Nutzer und Zielgruppen, Randbedingungen, Annahmen
 - Anforderungen
 - funktionale Anforderungen, Qualitätsanforderungen
 - Anhang
 - Index

4. Anforderungen dokumentieren

Verwendung von Anforderungsdokumenten

- Anforderungsdokumente können folgenden Zwecken dienen:
 - Planung
 - Architekturentwurf
 - Implementierung
 - Test
 - Änderungsmanagement
 - Systemnutzung und Systemwartung
 - Vertragsmanagement

4. Anforderungen dokumentieren

Qualitätskriterien für Anforderungsdokumente

- Um eine geeignete Basis für die nachgelagerten Entwicklungsschritte zu bilden, muss das Anforderungsdokument folgenden Qualitätskriterien genügen:
 - Eindeutigkeit und Konsistenz
 - Qualität der Einzelanforderungen ist Voraussetzung
 - Klare Struktur
 - Ermöglicht selektives Lesen
 - Modifizierbarkeit und Erweiterbarkeit
 - Inhalt und Struktur sollten die Änderbarkeit unterstützen
 - Vollständigkeit
 - Es müssen alle Anforderungen vorhanden sein und diese müssen vollständig dokumentiert sein.
 - Belege, Verweise und Quellenangaben sind formale Notwendigkeiten
 - Verfolgbarkeit
 - Beziehung zu anderen Entwicklungsdokumenten

4. Anforderungen dokumentieren

Qualitätskriterien für Anforderungen

- Folgende Qualitätskriterien sollten (u.a. gemäß IEEE-Standard 830-1998) für Anforderungen gelten:
 - Abgestimmt
 - Bewertet
 - Eindeutig
 - Gültig und aktuell
 - Korrekt
 - Konsistent
 - Prüfbar
 - Realisierbar
 - Verfolgbar
 - Vollständig
 - Verständlich

4. Anforderungen dokumentieren

Qualitätskriterien für Anforderungen

- Folgende Grundprinzipien sollten für Anforderungen gelten:
 - Kurze Sätze und kurze Absätze
 - Zusammengehörende Absätze sollten maximal sieben Sätze umfassen.
 - Nur eine Anforderung pro Satz formulieren
 - Anforderungen werden im Aktiv formuliert und enthalten nur ein Prozesswort.

4. Anforderungen dokumentieren

Glossar

- Eine häufige Ursache von Konflikten liegt im unterschiedlichen Begriffsverständnis der beteiligten Personen.
- Ein Glossar wirkt diesen Konflikten entgegen. Es enthält:
 - Kontextspezifische Fachbegriffe
 - Abkürzungen und Akronyme
 - Alltägliche Begriffe, die im gegebenen Kontext eine spezifische Bedeutung haben
 - Synonyme: Verschiedene Begriffe mit der gleichen Bedeutung
 - Homonyme: Begriffe mit verschiedenen Bedeutungen
- Unter Umständen macht es Sinn, ein projektübergreifendes Glossar zu erstellen und dieses dann jeweils projektspezifisch anzupassen.

4. Anforderungen dokumentieren

Glossar

- Regeln für den Umgang mit einem Glossar:
 - Das Glossar muss zentral verwaltet werden
 - Verantwortlichkeit schaffen
 - Das Glossar muss projektbegleitend gepflegt werden
 - Das Glossar muss allgemein zugänglich sein
 - Das Glossar muss verbindlich verwendet werden
 - Die Herkunft der Begriffe sollte im Glossar enthalten sein
 - Das Glossar sollte mit den Stakeholdern abgestimmt sein
 - Die Einträge des Glossars müssen eine einheitliche Struktur aufweisen

4. Anforderungen dokumentieren

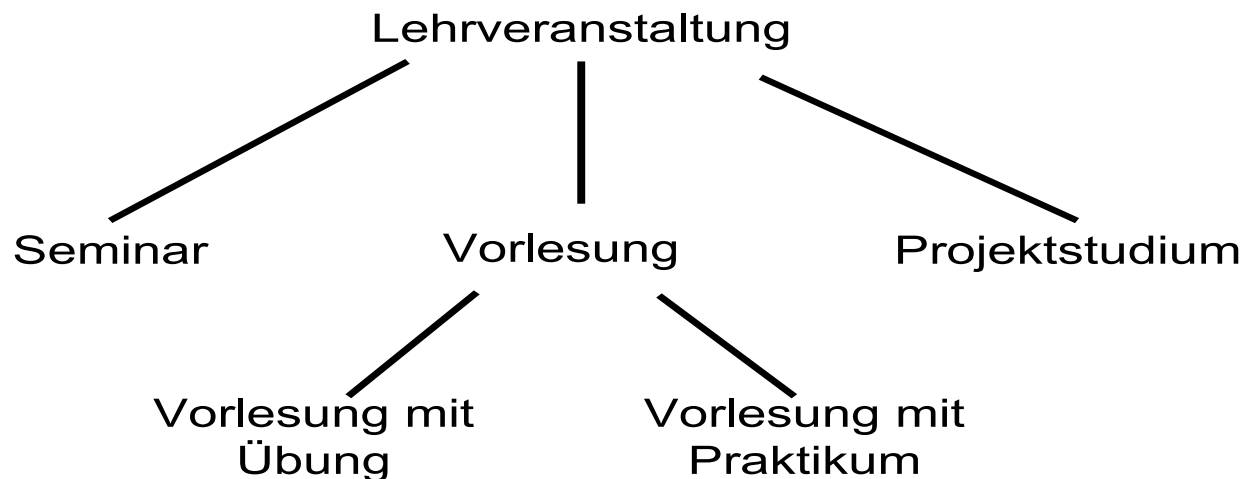
Glossar – Beispiel:

Begriff	Evaluationsbogen
Synonyme	Feedback-Bogen
Beschreibung	Ein Evaluationsbogen ist eine Sammlung von Fragen zur Bewertung einzelner Lehrveranstaltungen. Er kann Online oder in Papierform vorliegen. Im Evaluierungsprozess bezeichnet der Begriff die hochschulweite Vorlage, die fakultätsweite Vorlage, den konkreten Bogen für eine Lehrveranstaltung und die einzelne Kopie für den Studierenden zum Ausfüllen.
Abgrenzung	Der Begriff ist abzugrenzen vom Prozess der Evaluation

4. Anforderungen dokumentieren

Taxonomie

- Taxonomien definieren Klassifikationsschemata für Begriffe.
- Sie erlauben insbesondere hierarchische Zusammenhänge zwischen Begriffen darzustellen.
- Beispiel einer Taxonomie für Lehrveranstaltungen



4. Anforderungen dokumentieren

Zusammenfassung

- Dokumentgestaltung
 - Strukturperspektive, Funktionsperspektive, Verhaltensperspektive
 - Natürliche Sprache, konzeptuelle Modelle, Mischformen
- Dokumentenstrukturen
 - Standardisierte Strukturen (RUP, IEEE-Standard 830-1998, V-Modell)
 - Gliederungsempfehlung
- Verwendung von Anforderungsdokumenten
- Qualitätskriterien für Anforderungsdokumente und Anforderungen
- Glossar

Agenda

1. Einleitung und Grundlagen
2. System und Systemkontext abgrenzen
3. Anforderungen ermitteln
4. Anforderungen dokumentieren
5. **Anforderungen natürlichsprachig dokumentieren**
6. Anforderungen modellbasiert dokumentieren
7. Anforderungen prüfen und abstimmen
8. Anforderungen verwalten
9. Werkzeugunterstützung

Sprachliche Effekte

- Die natürliche Sprache eignet sich zur Dokumentation von Anforderungen, da sie von jedem Stakeholder gelesen und verstanden werden kann.
- Die natürliche Sprache birgt jedoch die Gefahr der Mehrdeutigkeit.
- Des weiteren können Transformationseffekte auftreten.
- Folgende Transformationseffekten müssen beachtet werden:
 - Nominalisierung
 - Substantive ohne Bezugsindex
 - Universalquantoren
 - Unvollständig spezifizierte Bedingungen
 - Unvollständig spezifizierte Prozesswörter

Nominalisierung

- Durch die Nominalisierung wird ein Prozess zu einem Ereignis gemacht. Sämtliche für die Beschreibung des Prozesses wichtigen Informationen gehen dabei verloren.
- Beispiel:
 - Anforderung: „Bei der Benutzerregistrierung sollte eine Bonitätsprüfung erfolgen.“
 - Potentielle Fragen: An welcher Stelle der Benutzerregistrierung? Durch wen erfolgt die Bonitätsprüfung?
- Prinzipiell spricht nichts gegen die Verwendung von Nominalisierungen, aber es darf kein Spielraum für Interpretationen des Prozesses entstehen. Zudem muss auch der Verlauf des Prozesses mit allen potenziell auftretenden Ausnahmen, Eingabe- und Ausgabeparametern klar sein.

5. Anforderungen natürlichsprachig dokumentieren

Substantive ohne Bezugsindex

- Substantive ohne Bezugsindex geben einen großen Interpretationsspielraum.
- Beispiel:
 - Anforderung: „Die Daten sollen dem Benutzer auf dem Terminal angezeigt werden.“
 - Potentielle Fragen: Welche Daten genau? Welchem Benutzer genau? Auf welchem Terminal genau?
 - Verbesserung: „Das System soll dem registrierten Benutzer seine Rechnungsdaten auf dem Terminal, an dem er angemeldet ist, anzeigen.“

Universalquantoren

- Universalquantoren sind Angaben über Häufigkeiten. Bei der Verwendung besteht die Gefahr, dass das spezifizierte Verhalten nicht für alle so bezeichneten Objekte zutrifft.
- Beispiel:
 - Typische Universalquantoren: „nie“, „immer“, „kein“, „jeder“, „alle“, „irgendeiner“ und „nichts“
 - Anforderung: „Das System soll bei jedem Seitenaufruf alle Datensätze abgleichen.“
 - Potentielle Fragen: Wirklich bei jedem Seitenaufruf? Wirklich alle Datensätze?
 - Verbesserung: „Das System soll beim ersten Seitenaufruf die ersten hundert Datensätze abgleichen.“

5. Anforderungen natürlichsprachig dokumentieren

Unvollständig spezifizierte Bedingungen

- Anforderungen, die Bedingungen enthalten, geben das Verhalten bei Eintritt der Bedingung an, müssen aber auch beschreiben, was passieren soll, wenn die Bedingung nicht eintritt.
- Beispiel:
 - Anforderung: „Das Restaurantsystem soll einem registrierten Gast bei einem Alter von über 17 Jahren alle im Lokal angebotenen Getränke anzeigen.“
 - Verbesserung: „Das Restaurantsystem soll einem registrierten Gast:
 - bei einem Alter von unter 16 Jahren ausschließlich alkoholfreie Getränke anzeigen;
 - bei einem Alter von 16-17 Jahren alkoholfreie und alkoholische Getränke ohne Branntwein anzeigen;
 - bei einem Alter von über 17 Jahren alle im Lokal angebotenen Getränke anzeigen.

5. Anforderungen natürlichsprachig dokumentieren

Unvollständig spezifizierte Prozesswörter

- Manche Prozesswörter (Verben) erfordern mehr als ein Substantiv, um vollständig spezifiziert zu sein. Das Verb „übertragen“ benötigt bspw. in der Regel die Ergänzungen, was von wo wohin übertragen wird.
- Beispiel:
 - Anforderung: „Zur Anmeldung des Benutzers werden die Login-Daten eingegeben.“
 - Verbesserung: „Das System soll dem Benutzer ermöglichen, seinen Benutzernamen und sein Passwort (über die Tastatur) am Rechner einzugeben.“
- Anforderungen sollten immer im Aktiv formuliert werden, nicht im Passiv.

5. Anforderungen natürlichsprachig dokumentieren

Satzschablonen

- Ein einfach erlernbarer und leicht einzusetzender Ansatz zur Reduzierung sprachlicher Effekte bereits während der Formulierung von Anforderungen ist die Satzschablone.
- Der Einsatz der Satzschablone unterstützt den Autor einer Anforderung, syntaktische Eindeutigkeit der Anforderung zu erreichen und Anforderungen hoher Qualität in einem optimalen Zeit- und Kostenrahmen zu verfassen.
- Der Einsatz einer Satzschablone empfiehlt sich insbesondere in Verbindung mit einem Glossar.
- Die Satzschablone sollte eingesetzt werden, wenn sich in den Projekten die Bereitschaft der Mitarbeiter herauskristallisiert, einer stark normierten Vorgehensweise zu folgen.

5. Anforderungen natürlichsprachig dokumentieren

Definition: Satzschablone

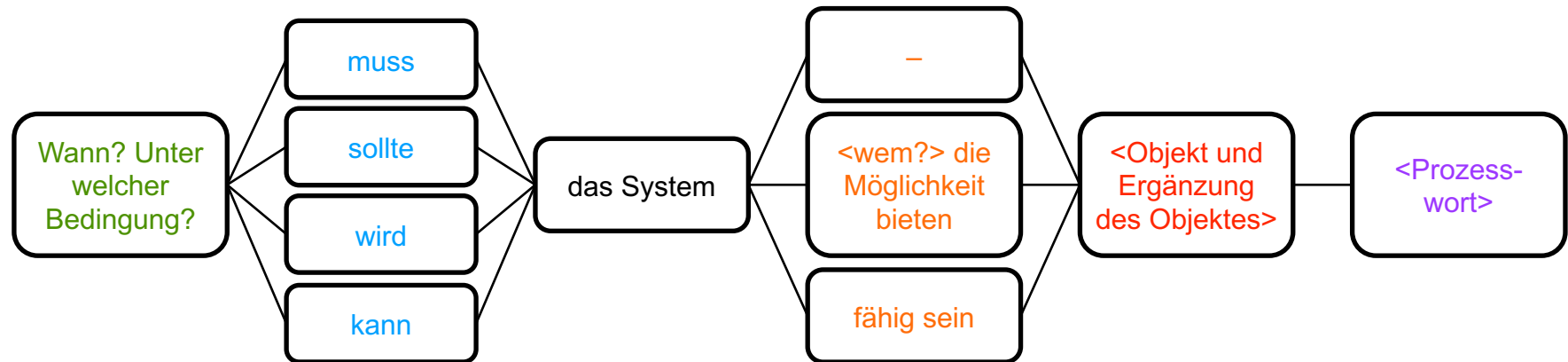
Eine Satzschablone (Requirements Template) ist ein Bauplan für die syntaktische Struktur einer einzelnen Anforderung.

5. Anforderungen natürlichsprachig dokumentieren

Satzschablonen

- Die Formulierung einer Anforderung mittels Satzschablone erfolgt i.d.R. in fünf Schritten.
- Beispiel: Anforderung an ein System zum Ticketverkauf in einem Kino

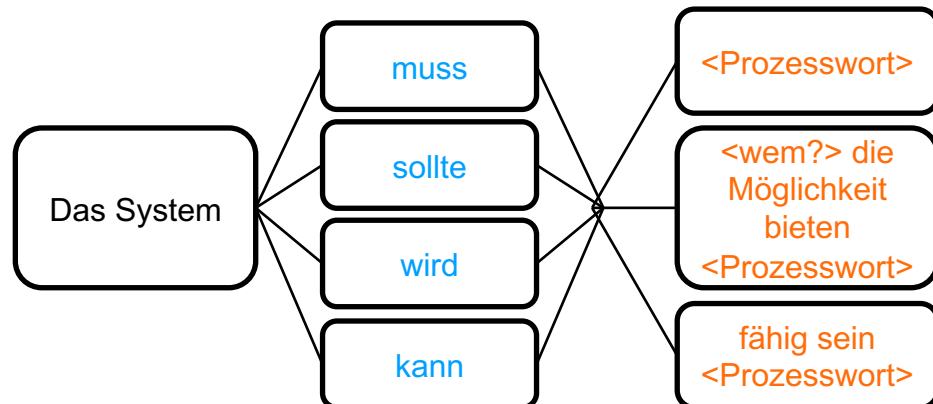
Bei Verfügbarkeit der gewünschten Kategorie und Preisklasse, muss das System dem Ticketverkäufer die Möglichkeit bieten, einem Kunden das entsprechende Ticket zu verkaufen.



5. Anforderungen natürlichsprachig dokumentieren

Satzschablonen

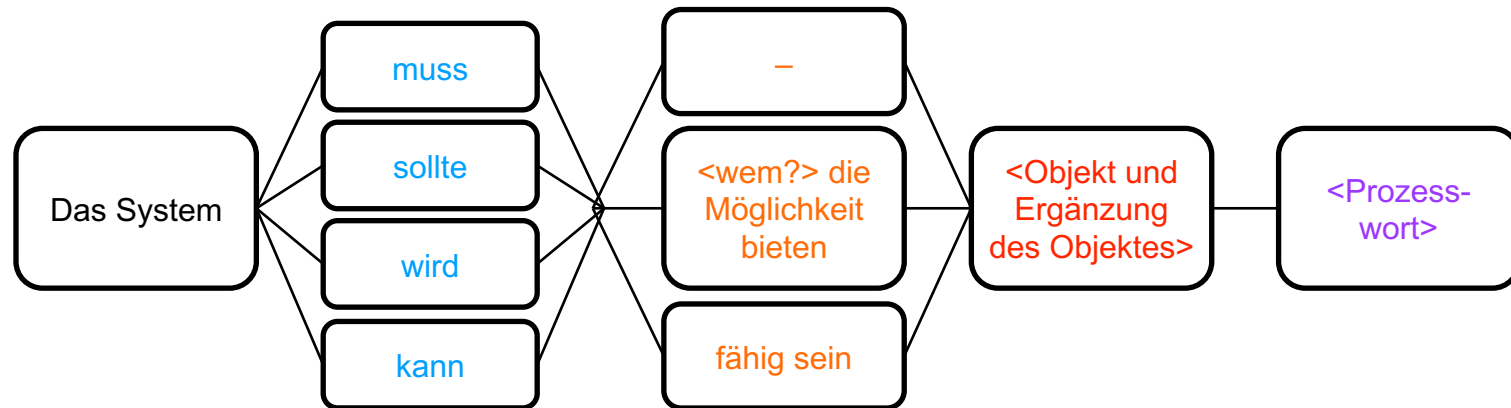
- Schritt 1: Festlegen der rechtlichen Verbindlichkeit
 - Unterscheidung zwischen rechtlich bindenden („muss“), dringend empfohlenen („sollte“) und zukünftigen („wird“) Anforderung.
- Schritt 2: Der Kern der Anforderung
 - Im Mittelpunkt jeder Anforderung steht die geforderte Funktionalität (z.B. drucken, speichern, übertragen, berechnen), auch Prozess genannt.
- Schritt 3: Charakterisieren der Aktivität des Systems
 - Es existieren drei Arten:
 - Selbständige Systemaktivität
 - Benutzerinteraktion
 - Schnittstellenanforderung



5. Anforderungen natürlichsprachig dokumentieren

Satzschablonen

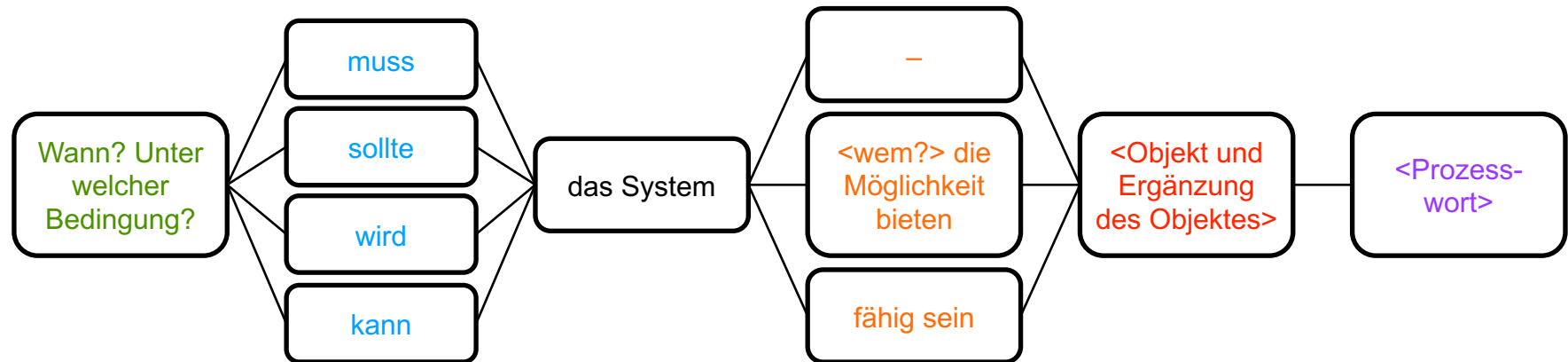
- Schritt 4: Objekte einfügen
 - Manche Prozesswörter benötigen ein oder mehrere Objekte (z.B. „<was>“ oder „<wo>“).



5. Anforderungen natürlichsprachig dokumentieren

Satzschablonen

- Schritt 5: Formulierung von logischen und zeitlichen Bestimmungen
 - Für Anforderungen ist es typisch, dass die geforderte Funktionalität nicht fortwährend, sondern nur unter bestimmten zeitlichen („sobald“) oder logischen („falls“) Bedingungen ausgeführt oder zur Verfügung gestellt wird.
 - Die vollständige Satzschablone:

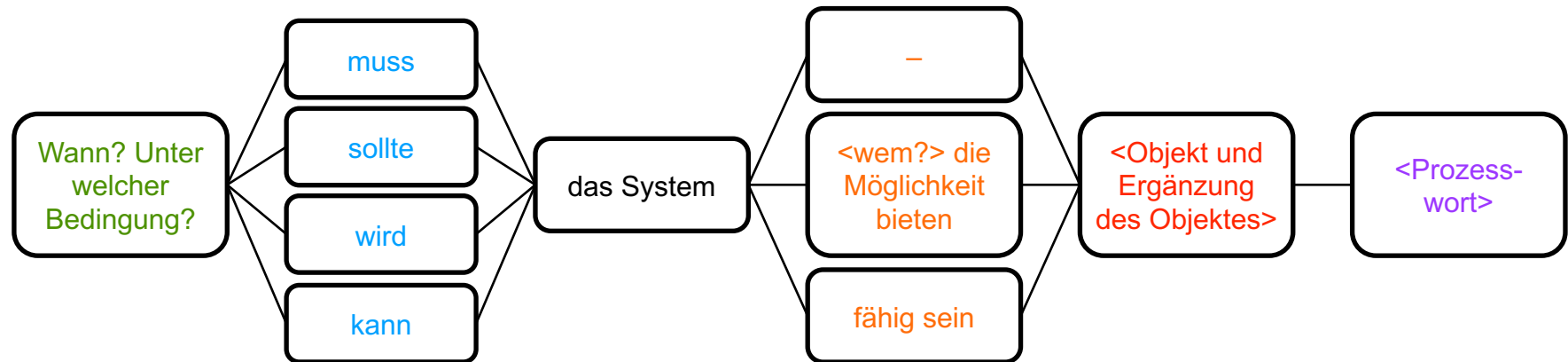


5. Anforderungen natürlichsprachig dokumentieren

Satzschablonen – Beispiel 2:

- Anforderung an ein System zur Steuerung der Jalousien in einem Gebäude:

Bei einer Windstärke von über 60km/h muss das System fähig sein, die Jalousien im gesamten Gebäude automatisch zu öffnen und die Steuerung für manuelle Betätigung zu sperren.



5. Anforderungen natürlichsprachig dokumentieren

Zusammenfassung

- Sprachliche Effekte
 - Nominalisierung
 - Substantive ohne Bezugsindex
 - Universalquantoren
 - Unvollständig spezifizierte Bedingungen
 - Unvollständig spezifizierte Prozesswörter

- Satzschablonen

Agenda

1. Einleitung und Grundlagen
2. System und Systemkontext abgrenzen
3. Anforderungen ermitteln
4. Anforderungen dokumentieren
5. Anforderungen natürlichsprachig dokumentieren
6. **Anforderungen modellbasiert dokumentieren**
7. Anforderungen prüfen und abstimmen
8. Anforderungen verwalten
9. Werkzeugunterstützung

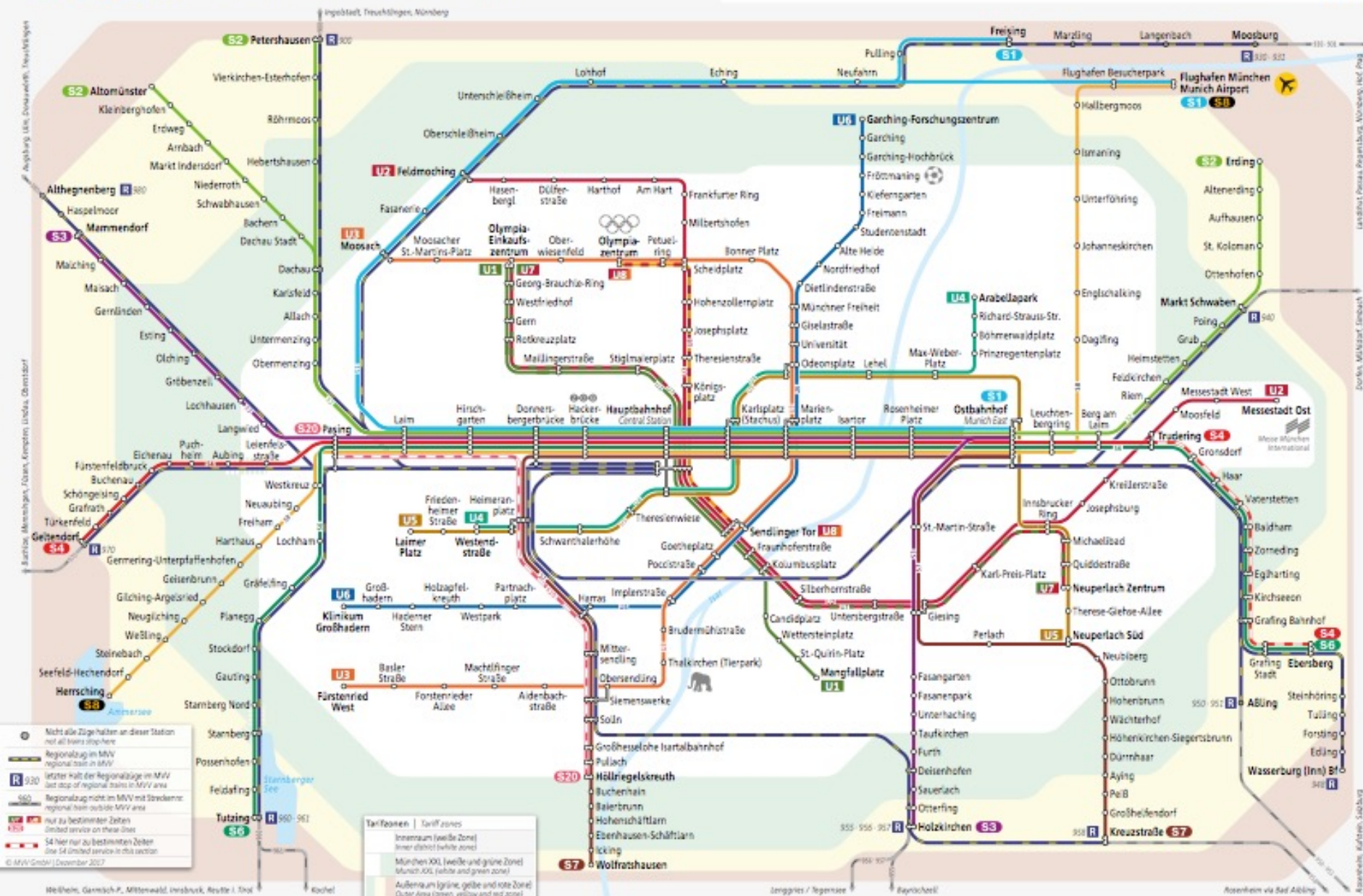
Definition: Modell

Ein Modell ist ein abstraktes Abbild einer existierenden oder einer noch zu schaffenden Realität.

Modelle besitzen drei wesentliche Eigenschaften (nach Stachowiak, 1973):

- Das **Abbildungsmerkmal**: «Modelle sind stets Modelle von etwas, nämlich Abbildungen, Repräsentationen natürlicher oder künstlicher Originale, die selbst wieder Modelle sein können»
- Das **Verkürzungsmerkmal**: «Modelle erfassen im allgemeinen nicht alle Attribute des durch sie repräsentierten Originals, sondern nur solche, die den jeweiligen Modellerschaffern und/ oder Modellbenutzern relevant scheinen»
- Das **pragmatische Merkmal**: Ein Modell hat den Zweck, unter best. Bedingungen und bezüglich best. Fragestellungen das Original zu ersetzen.





- Zur Konstruktion konzeptueller Modelle werden **konzeptuelle Modellierungssprachen** verwendet. Eine konzeptuelle Modellierungssprache wird durch ihre Syntax und ihre Semantik definiert.
- Konzeptuelle Modellierungssprachen können hinsichtlich ihres Formalisierungsgrades in informale, semiformale und formale Modellierungssprachen eingeteilt werden.
 - Bei formalen Modellierungssprachen (dann häufig auch Spezifikationssprachen genannt) ist die Semantik präzise definiert.
 - Bei informalen Modellierungssprachen ist die Semantik in der Sprachdefinition nur umgangssprachlich festgelegt.

Anforderungsmodelle

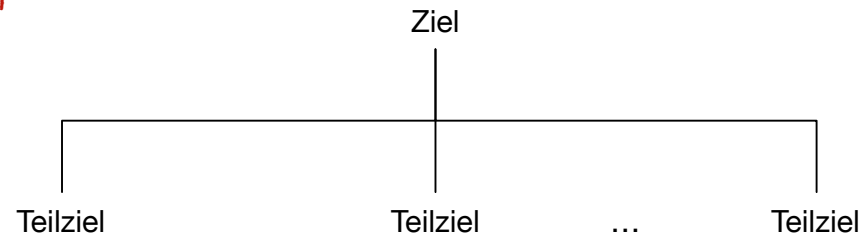
- Konzeptuelle Modelle, die Anforderungen eines Systems dokumentieren, werden als Anforderungsmodelle bezeichnet.
- Als De-facto-Standard für Anforderungsmodelle hat sich die Unified Modeling Language (UML) durchgesetzt.
- Anforderungsmodelle dokumentieren die Anforderungen des betrachtenden Systems, indem sie spezifische Aspekte des zugrunde liegenden Problems modellieren (im Gegensatz hierzu dokumentieren Modelle in der Systementwicklung Lösungsaspekte).
- Anforderungsmodelle weisen folgende Vorteile aus:
 - Sie sorgen für eine bessere Verständlichkeit
 - Sie unterstützen unterschiedliche Perspektiven
 - Sie ermöglichen eine zweckabhängige Abstraktion

Zielmodelle

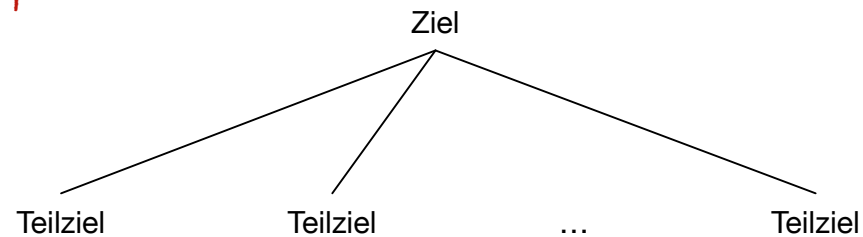
- Ziele beschreiben Intentionen von Stakeholdern.
- Ziele eignen sich besonders dazu, die Vision des Systems zu verfeinern (**Zieldekomposition**).
- Neben der natürlichsprachigen Beschreibung von Zielen können auch Zielmodelle verwendet werden.
- Ein Beispiel für ein Zielmodell sind **Und-Oder-Bäume**.

Zielmodelle / Und-Oder Baum

UND - Dekomposition

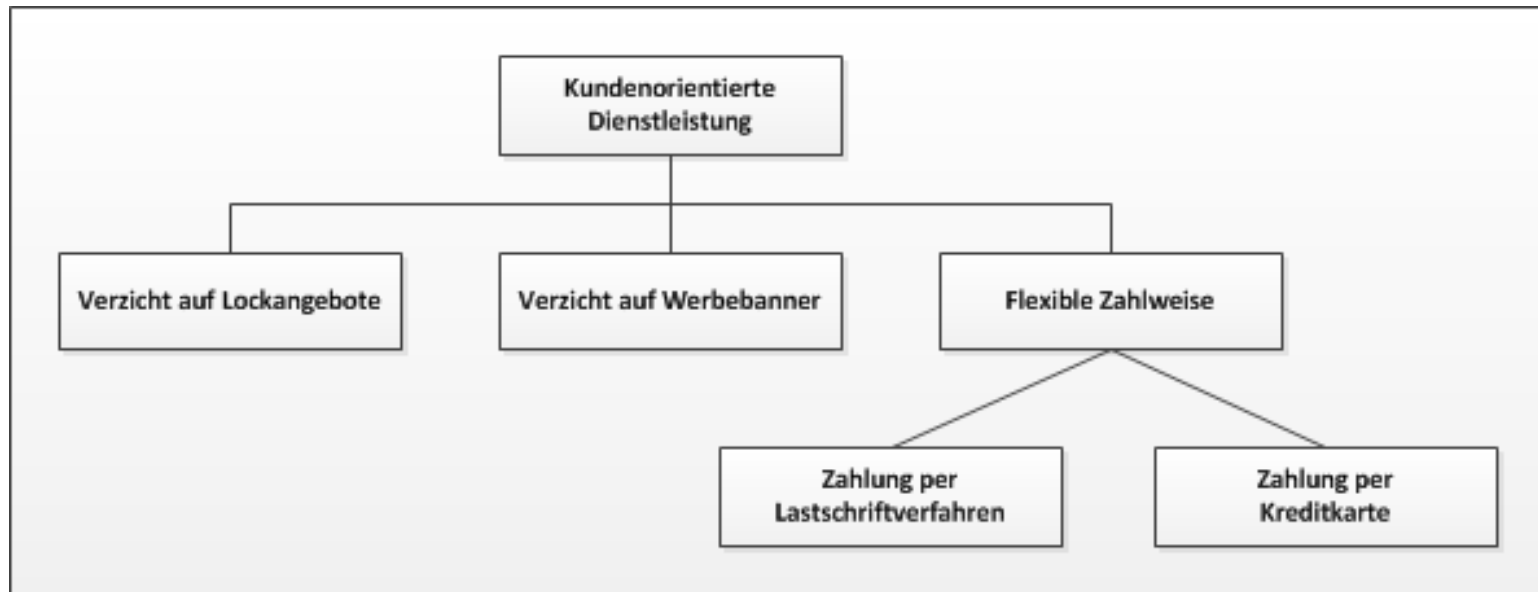


ODER - Dekomposition



Zielmodelle

- Beispiel für einen Und-Oder-Baum:



Anwendungsfälle (Use Cases)

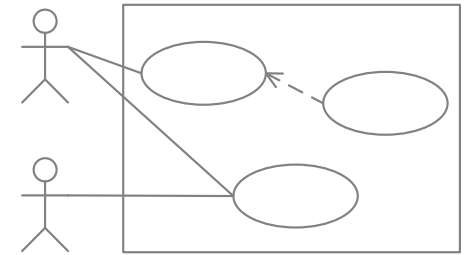
- Use Cases ermöglichen die Dokumentation und Analyse der Funktionalität eines geplanten oder existierenden Systems.
- Use Cases bestehen aus zwei Bestandteilen:
 - Use-Case-Diagramme
 - Use-Case-Spezifikationen

Anwendungsfälle (Use Cases)

- Use-Case-Diagramme dienen dazu aus einer Nutzungssicht die Funktionalität des betrachteten System, deren Beziehungen untereinander sowie die Beziehungen des Systems zu dessen Umgebung überblicksartig zu dokumentieren.
- Mögliche Komponenten eines Use-Case-Diagramms (hier: UML-Anwendungsfalldiagramm):
 - Use Cases
 - Akteure (Personen und Systeme)
 - Systemgrenzen
 - Extend-Beziehungen
 - Include-Beziehungen
 - Kommunikation zwischen Akteur und Use Case

Anwendungsfälle (Use-Cases)

Ein Anwendungsfall ist die Spezifikation einer Reihe von Aktionen, die von einem System ausgeführt werden und zu einem beobachtbaren Ergebnis führen, das in der Regel für einen oder mehrere Akteure oder andere Interessengruppen des Systems von Wert ist.



Die Beschreibung von Use Cases besteht aus zwei Bestandteilen:

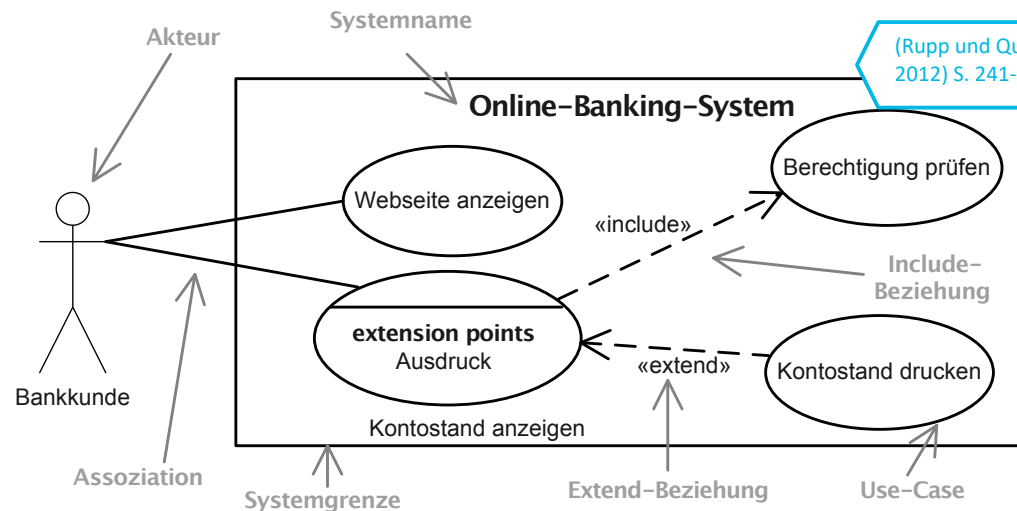
- Use-Case-Diagramme
- Use-Case-Spezifikationen

Ein Use-Case-Diagramm und seine Bestandteile

Use-Case-Diagramme dienen dazu aus einer Nutzungssicht die Funktionalität des betrachteten Systems, deren Beziehungen untereinander sowie die Beziehungen des Systems zu dessen Umgebung überblicksartig zu dokumentieren.

Ein Use-Case-Diagramm enthält die grafische Darstellung ...

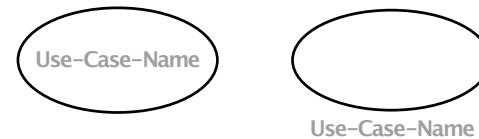
- des Systems,
- der Use-Cases,
- der Akteure außerhalb des Systems,
- der Beziehungen zwischen Akteur und Use-Case, oder Use-Cases untereinander sowie
- Generalisierungen von Akteuren.



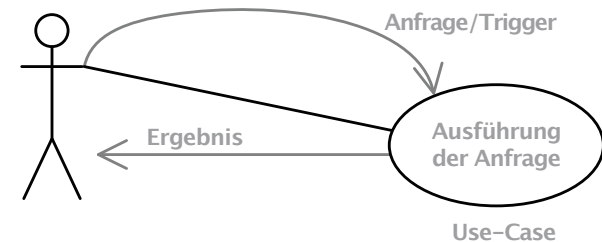
Ein Use-Case-Diagramm und seine Bestandteile



- In aller Regel wird ein Use-Case durch eine Ellipse dargestellt.
- Der Name des Use-Cases wird dabei inner- oder unterhalb der Ellipse notiert.

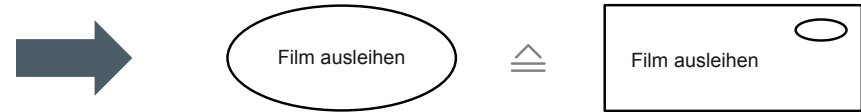


- Ein UseCase wird stets von einem Akteur ausgelöst bzw. instanziiert (Trigger)
- und führt zu einem fachlichen Ergebnis (Datei auf dem Medium abgelegt).
- Die Bezeichnung des UseCase spiegelt die Sicht des Akteurs wider (z. B. „Film ausleihen“)
- Ein UseCase darf auch gleichzeitig mehrfach instanziiert werden (gleichzeitiges Abspeichern von mehreren Dateien). D. h., auch wenn Sie in einem Diagramm nur fünf UseCases sehen, können Hunderte von realen Abläufen gleichzeitig durch das System abgewickelt werden.

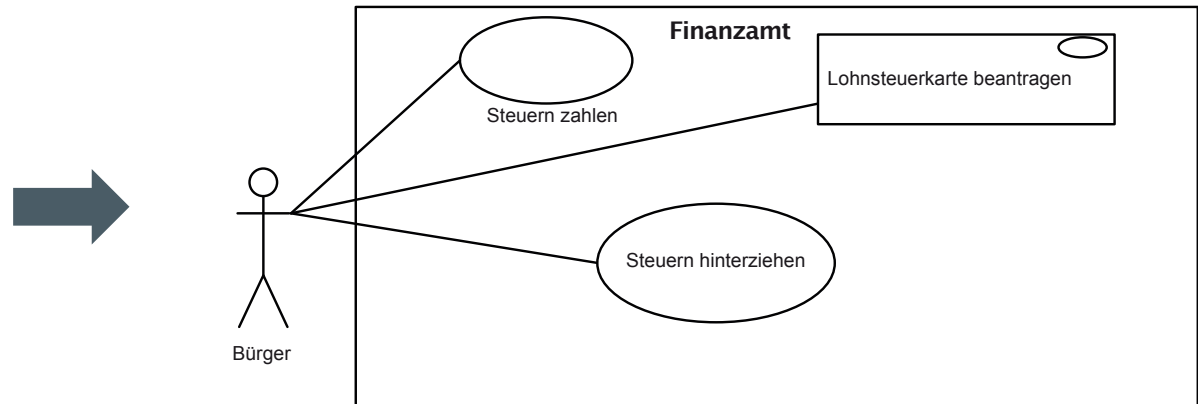


Ein Use-Case-Diagramm und seine Bestandteile

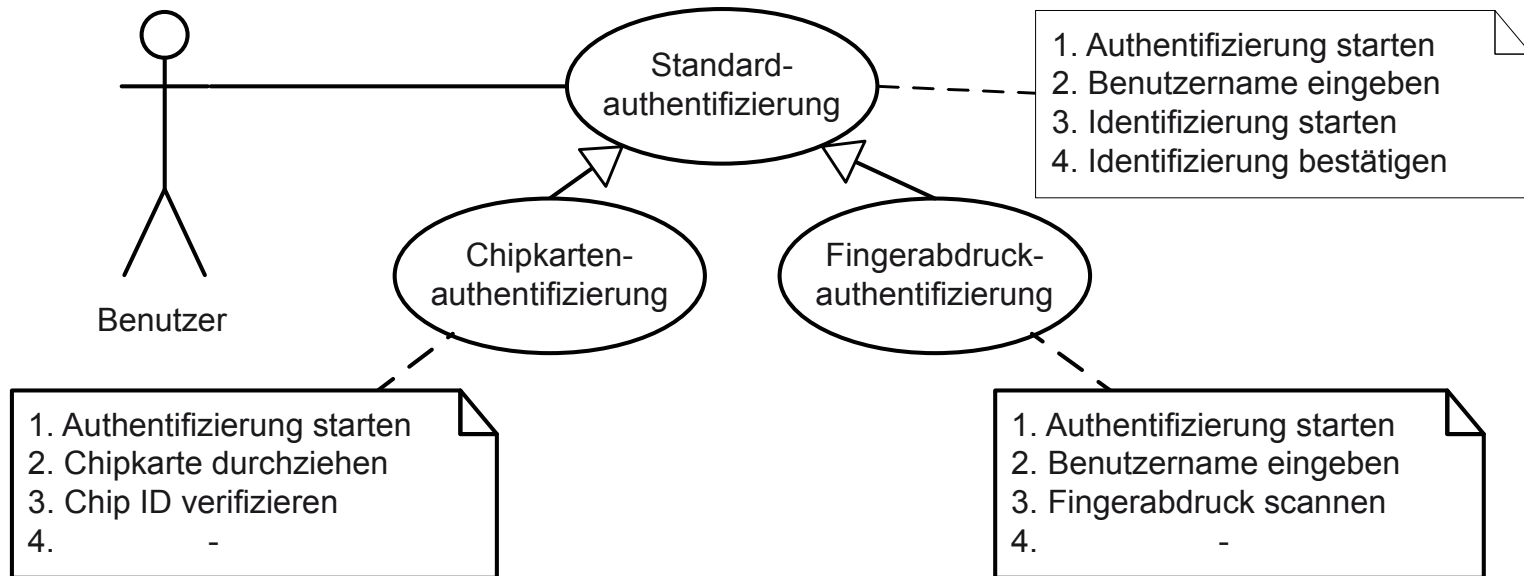
- Da ein Use-Case im Metamodellsinne einen verhaltensspezifischen *Classifier* darstellt, dürfen Sie statt der üblichen Ellipsennotation auch die von Klassen bekannte Rechtecksnotation verwenden



- Das rechts stehende Diagramm zeigt die verschiedenen Notationsmöglichkeiten von Use-Cases



Ein Use-Case-Diagramm und seine Bestandteile

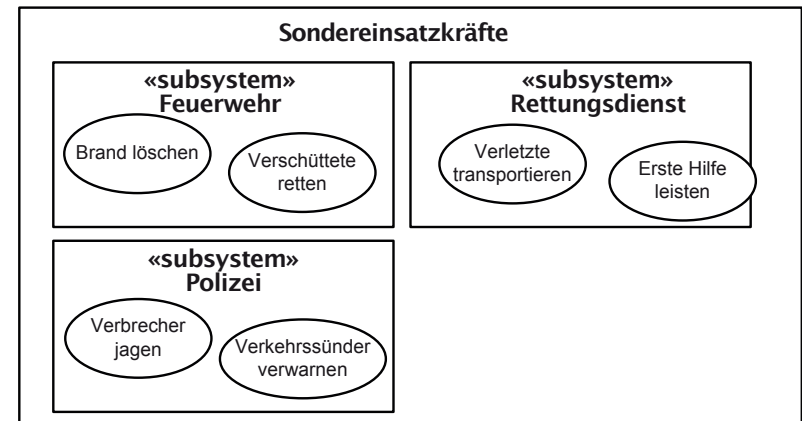
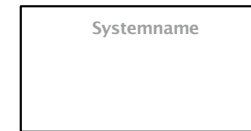


Eine Generalisierungsbeziehung zwischen Use-Cases. Die Kommentare verdeutlichen die Einzelschritte.

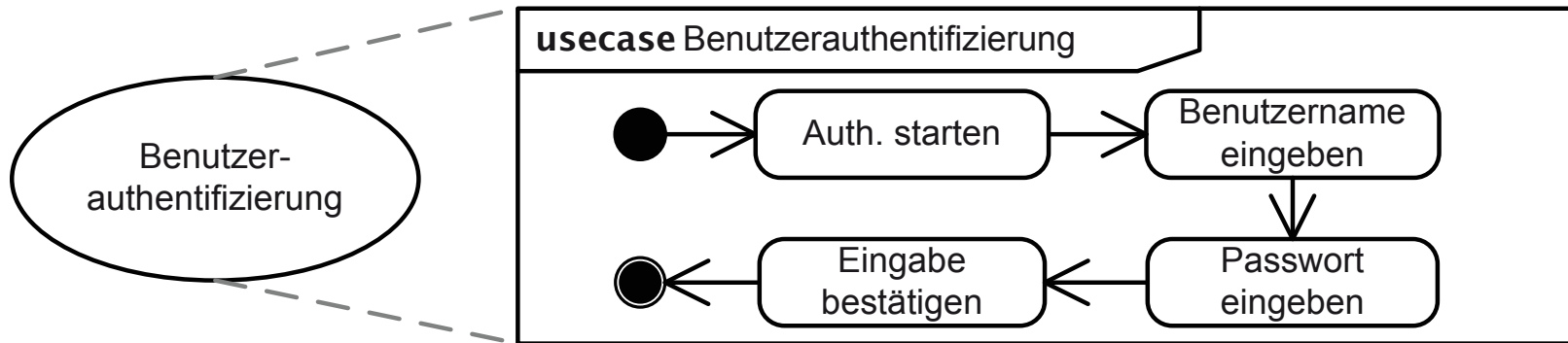
Ein Use-Case-Diagramm und seine Bestandteile

- Das System wird als rechteckiger Kasten abgebildet, wobei die Kanten des Systems die Systemgrenzen darstellen.
- Der Name des Systems wird innerhalb des Kastens angegeben.

- Das Diagramm rechts zeigt exemplarisch eine Abgrenzung verschiedener Teilbereiche (System und Subsysteme, die die Use-Cases realisieren) und ordnet beispielsweise Use-Cases wie *Brand löschen* und *Verschüttete retten* dem Subsystem Feuerwehr zu.
- Es ist nicht zwingend notwendig, das System zu modellieren. Ein Use-Cas-Diagramm ist auch ohne Angabe der Einheit, die den Use-Case realisiert, vollständig (gibt die Möglichkeit, sich zunächst auf die Verhaltensdefinition zu beschränken und erst in einem späteren Schritt dieses Verhalten entsprechenden Einheiten zuzuweisen)



Ein Use-Case-Diagramm und seine Bestandteile



Detaillierte Verhaltensbeschreibung eines Use-Case

Ein Use-Case-Diagramm und seine Bestandteile

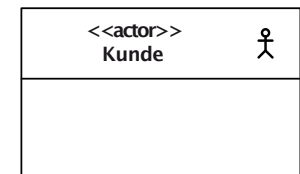
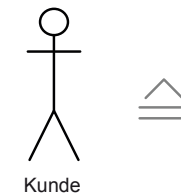
- Die gebräuchlichste Notation für einen Akteur ist das Strichmännchen, mit dem Namen des Akteurs oberhalb oder unterhalb.
- Allerdings erlaubt Ihnen die UML, den Namen auch rechts oder links vom Strichmännchen zu notieren.
- Vorgeschrieben ist nur, dass der Name in der Umgebung des Strichmännchens stehen muss.



Name des Akteurs

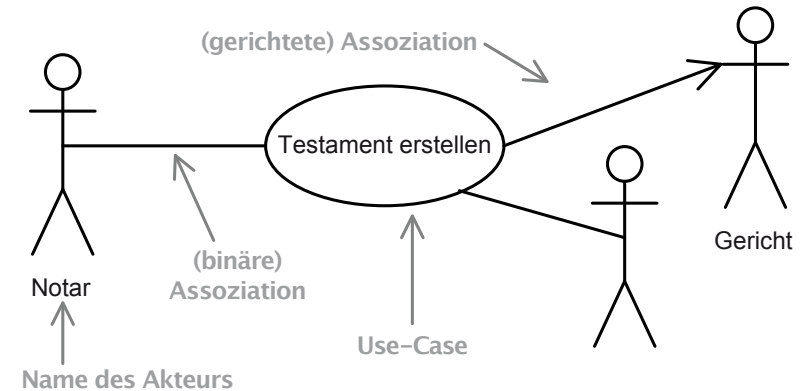


Rechtecksnotation

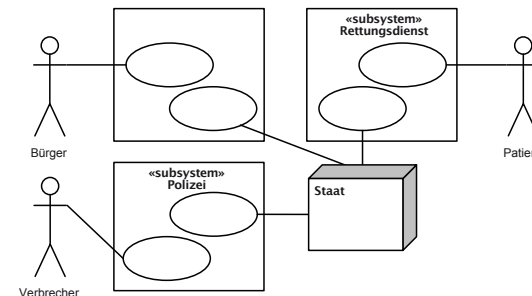


Ein Use-Case-Diagramm und seine Bestandteile

- In einem Use-Case-Diagramm interagiert ein Akteur mit einem Use-Case, indem er dessen Ausführung anstößt oder an der Ausführung aktiv oder passiv beteiligt ist.
- Zwischen dem Use-Case und dem Akteur findet ein wechselseitiger Austausch von Signalen und Daten statt.
- Das „Drehbuch“ dafür liefert die Verhaltensdefinition des internen Use-Case-Ablaufs.

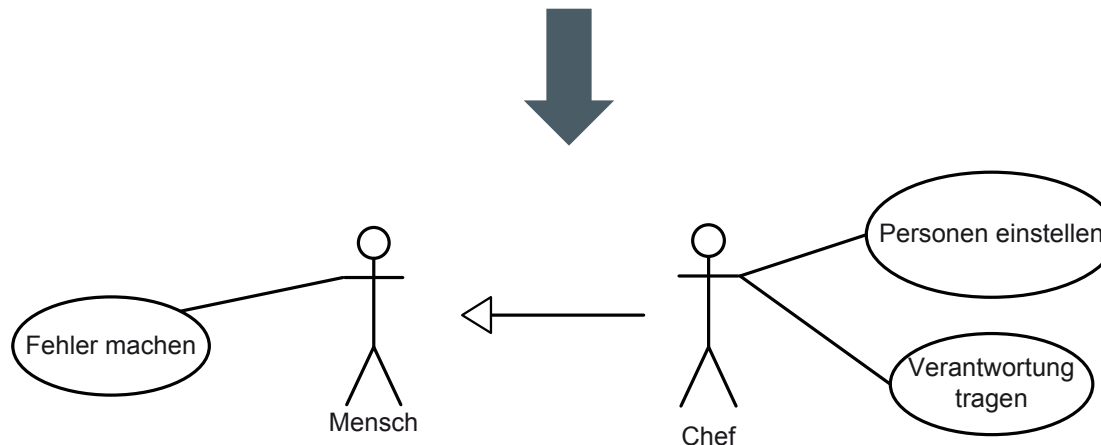


- Wenn Sie das System modellieren, müssen Sie die Akteure immer außerhalb der Systemgrenzen (Rahmen) anbringen.



Ein Use-Case-Diagramm und seine Bestandteile

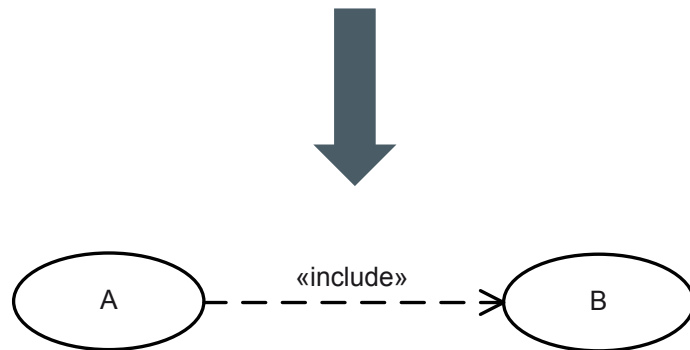
- Eine *Classifier* -Eigenschaft, die Akteure besitzen, ist die Generalisierungs- und Spezialisierungsmöglichkeit
- Hierbei wird die Kommunikationsbeziehung zwischen Akteur und Use-Case weitervererbt: Der spezialisierte Akteur ist an den gleichen Use-Case-Abläufen beteiligt wie der vererbende Akteur.



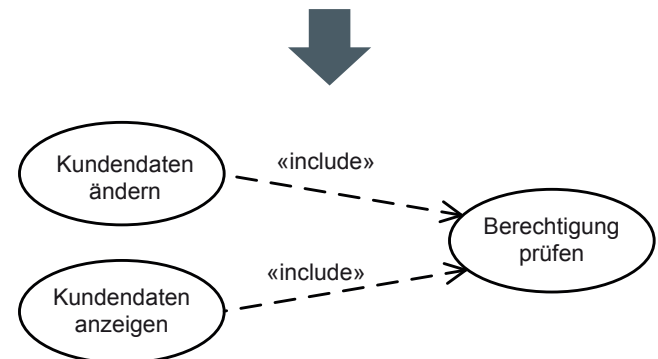
Ein Use-Case-Diagramm und seine Bestandteile

Die «include»-Beziehung

Eine **Include-Beziehung** definiert, dass ein Anwendungsfall das in einem anderen Anwendungsfall definierte Verhalten enthält.

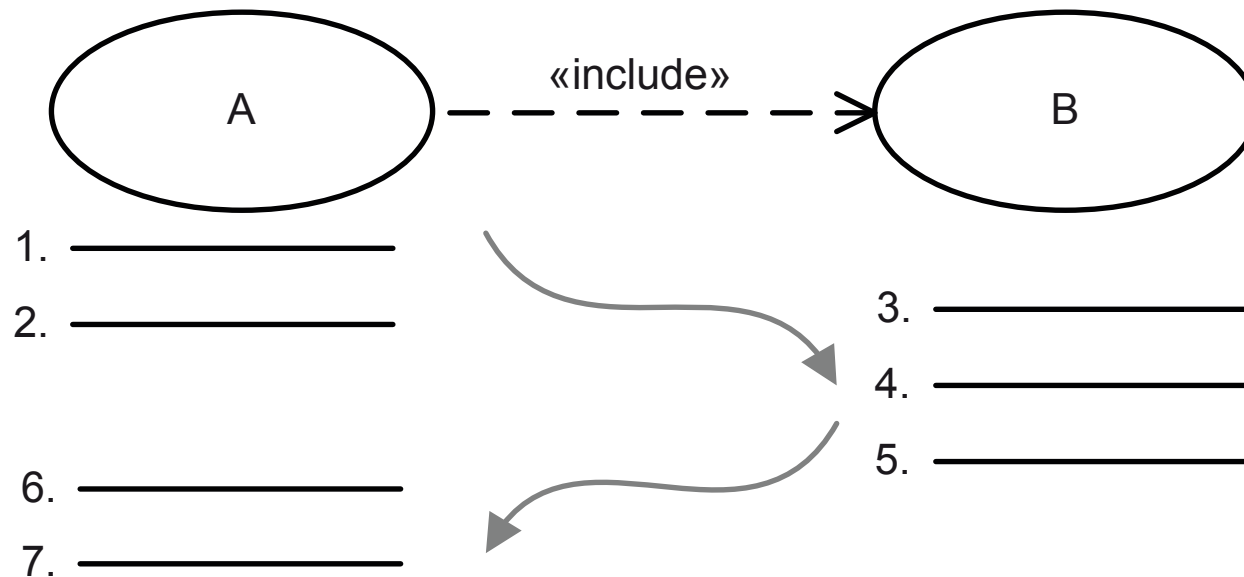


Zentral beschriebenes Verhalten
inkludieren



Ein Use-Case-Diagramm und seine Bestandteile

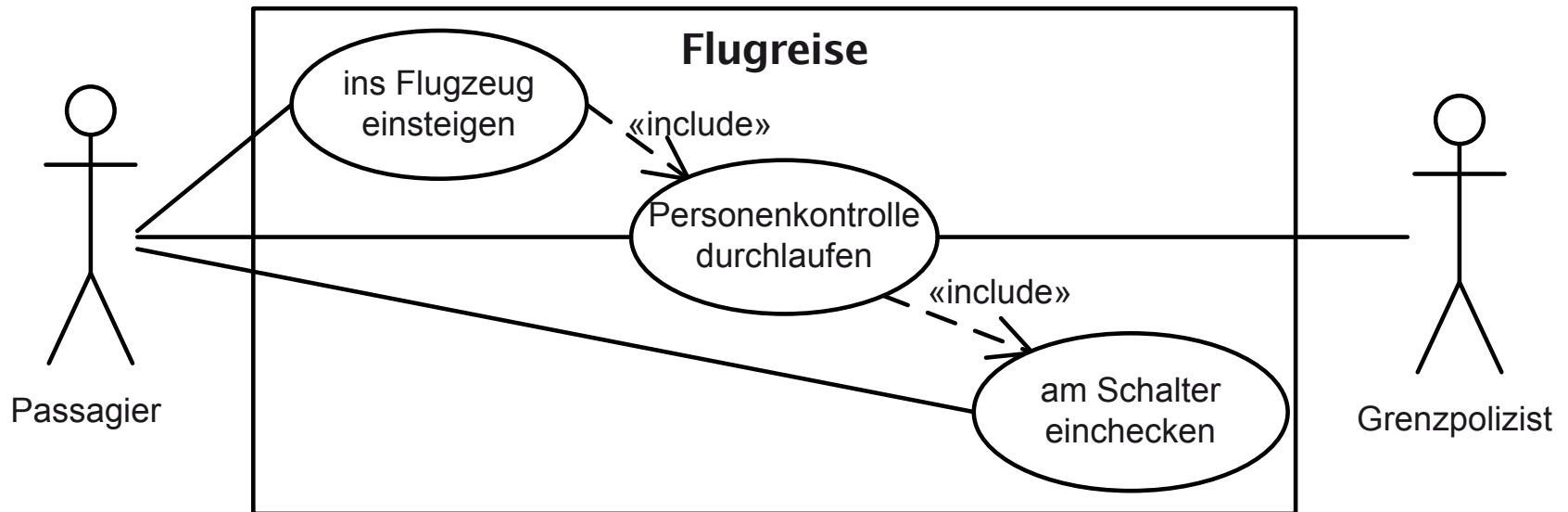
Die «include»-Beziehung



Der Ablauf einer «include»-Beziehung

Ein Use-Case-Diagramm und seine Bestandteile

Die «include»-Beziehung

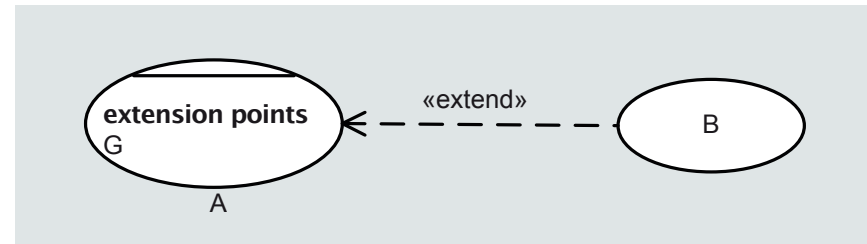


«include»-Beziehungen in der Anwendung

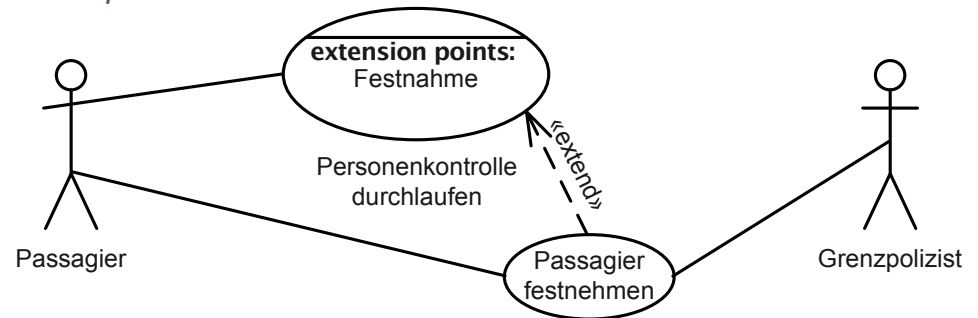
Ein Use-Case-Diagramm und seine Bestandteile

Die «extend»-Beziehung

- **Extend:** Eine Beziehung von einem erweiternden Anwendungsfall zu einem erweiterten Anwendungsfall, die angibt, wie und wann das im erweiternden Anwendungsfall definierte Verhalten in das im erweiterten Anwendungsfall definierte Verhalten eingefügt werden kann.
- **Extension Point:** Ein Erweiterungspunkt identifiziert einen Punkt im Verhalten eines Anwendungsfalls, an dem dieses Verhalten durch das Verhalten eines anderen (erweiternden) Anwendungsfalls erweitert werden kann, wie durch eine extend-Beziehung angegeben.



Beispiel:

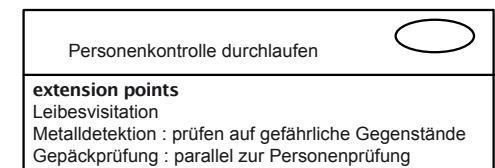
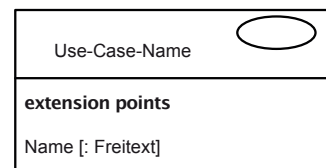
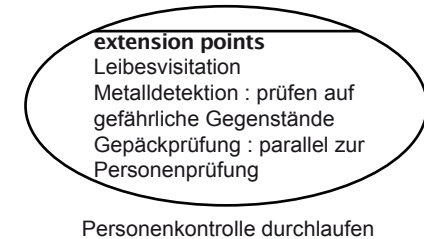
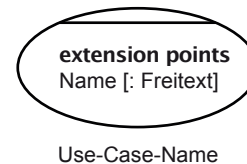


Ein Use-Case-Diagramm und seine Bestandteile

Die «extend»-Beziehung

- Die Erweiterungspunkte werden in einem mit **extension points** bezeichneten Abschnitt innerhalb der Use-Case-Ellipse dargestellt und müssen benannt sein.
- Die Bezeichnung des Use-Cases verschiebt sich unter die Ellipse.
- Sie dürfen optional an den Namen des Erweiterungspunktes einen beliebigen Text anhängen. Dieser Text ist durch einen Doppelpunkt vom Namen des Erweiterungspunktes abzutrennen.
- Sie haben dadurch die Möglichkeit, den Erweiterungspunkt präziser zu beschreiben oder die Auftritsstelle im Use-Case-Ablauf anzugeben.

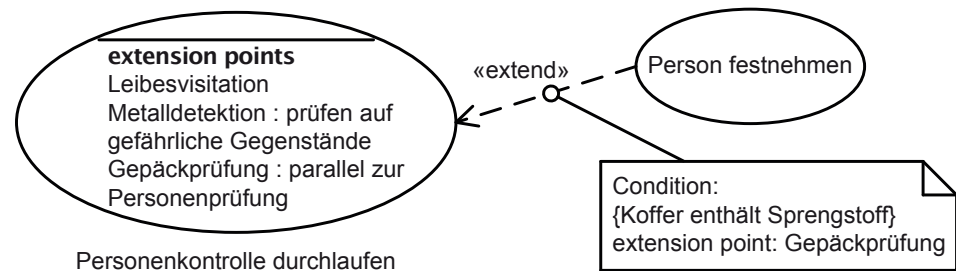
- Erweiterungspunkte eines UseCases in der Rechtecksnotation



Ein Use-Case-Diagramm und seine Bestandteile

Die «extend»-Beziehung

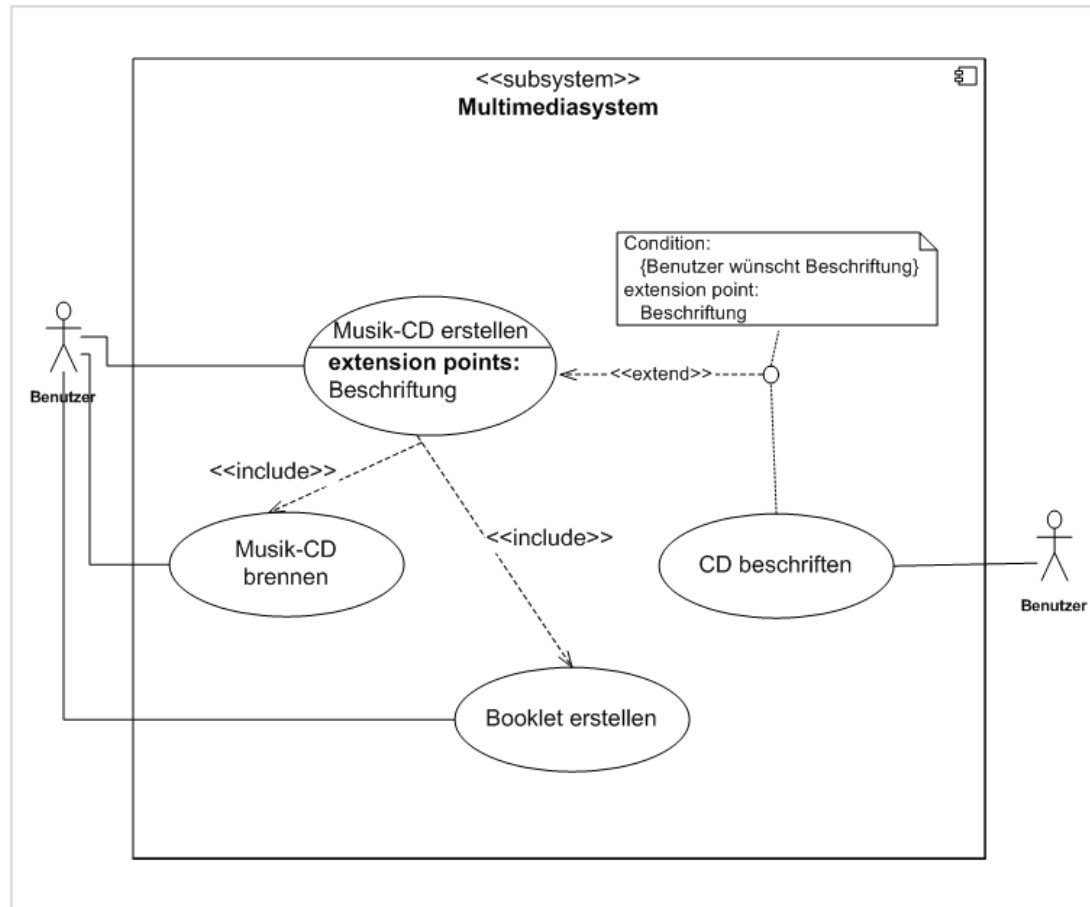
- Neben dem Erweiterungspunkt können Sie zudem eine Bedingung für die Erweiterung angeben.
- Die Bedingung wird bei Erreichen des Erweiterungspunktes geprüft.
- Ist die Bedingung wahr, wird der Ablauf erweitert, das heißt, der entsprechende referenzierte Use-Case durchlaufen.
- Ist die Bedingung nicht erfüllt, läuft der Use-Case-Ablauf „normal“ weiter.
- Die entsprechenden Bedingungen und der zugehörige Erweiterungspunkt werden als Notizzettel (**condition**) an die «extend»-Beziehung notiert.



Die Bedingungen eines
Erweiterungspunkts als Notiz

Ein Use-Case-Diagramm und seine Bestandteile

Beispielmodell



Anwendungsfälle (Use Cases)

- Um weiterführende Informationen abzubilden, empfiehlt sich der Einsatz von Use-Case-Spezifikationen in Form einer Schablone.

Nr.	Abschnitt	Inhalt / Erläuterung
01	Bezeichner	Eindeutiger Bezeichner des Use Case
02	Name	Eindeutiger Name für den Use Case
03	Autoren	Namen der Autoren, die an dieser Use-Case-Beschreibung mitgearbeitet haben
04	Priorität	Wichtigkeit des Use Case gemäß der verwendeten Priorisierungstechnik
05	Kritikalität	Kritikalität des Use Case, z.B. hinsichtlich des Schadensausmaßes bei Fehlverhalten des Use Cases
06	Quelle	Bezeichnung der Quelle ([Stakeholder Dokument System]), von der der Use Case stammt
07	Verantwortlicher	Der für diesen Use Case verantwortliche Stakeholder
08	Beschreibung	Komprimierte Beschreibung des Use Case
09	Auslösendes Ereignis	Angaben des Ereignisses, das den Use Case auslöst

Anwendungsfälle (Use Cases)

- Um weiterführende Informationen abzubilden, empfiehlt sich der Einsatz von Use-Case-Spezifikationen in Form einer Schablone.

Nr.	Abschnitt	Inhalt / Erläuterung
10	Akteure	Auflistung der Akteure, die mit dem Use Case in Beziehung stehen
11	Vorbedingung	Eine Liste notwendiger Voraussetzungen, die erfüllt sein müssen, bevor die Ausführung des Use Case beginnen kann
12	Nachbedingung	Eine Liste von Zuständen, in denen sich das System unmittelbar nach der Ausführung des Hauptszenarios befindet
13	Ergebnis	Beschreibung der Ausgaben, die während der Ausführung des Use Case erzeugt werden
14	Hauptszenario	Beschreibung des Hauptszenarios eines Use Case
15	Alternativszenarien	Beschreibungen von Alternativszenarien des Use Case oder lediglich Angabe der auslösenden Ereignisse
16	Ausnahmeszenarien	Beschreibung von Ausnahmeszenarien des Use Case oder lediglich Angabe der auszulösenden Ereignisse
17	Qualitäten	Querbezüge zu Qualitätsanforderungen

Drei Perspektiven auf Anforderungen

- In der modellbasierten Dokumentation von Anforderungen werden typischerweise die folgenden drei Perspektiven unterschieden:
 - Strukturperspektive
 - Funktionsperspektive
 - Verhaltensperspektive
- Die in den Modellen der verschiedenen Perspektiven abgebildeten Aspekte finden sich zum Teil auch in anderen Perspektiven wieder, so dass die drei Perspektiven nicht disjunkt sind.

Strukturperspektive

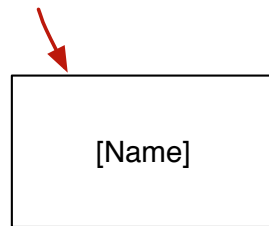
- In der Strukturperspektive werden bspw. die Struktur von Ein- und Ausgabedaten sowie die statisch-strukturellen Aspekte von Nutzungs- und Abhängigkeitsbeziehungen des Systems im Systemkontext dokumentiert.
- Mögliche Modellierungssprachen sind:
 - Entity-Relationship-Diagramme nach Chen (ER-Diagramme)
 - UML-Klassendiagramme

Strukturperspektive

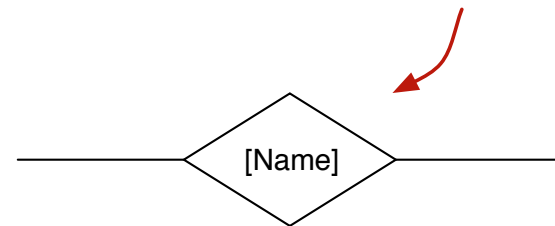
- Das ER-Diagramm besitzt folgende Komponenten:

- Entitätstyp
- Beziehungstyp
- Attribut
- Kardinalitäten

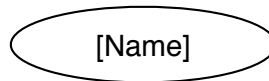
Entitätstyp



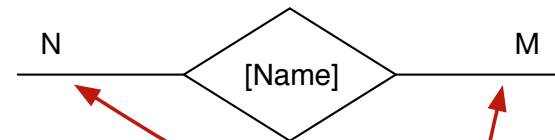
Beziehungstyp



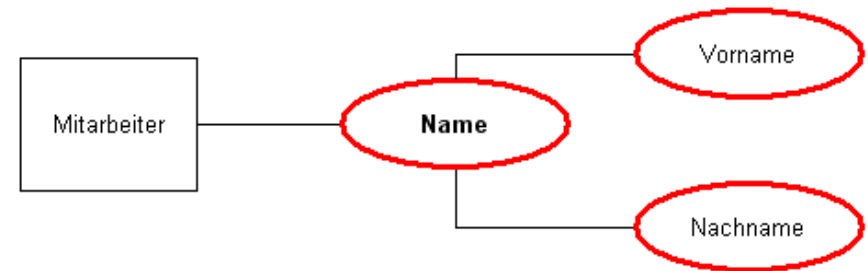
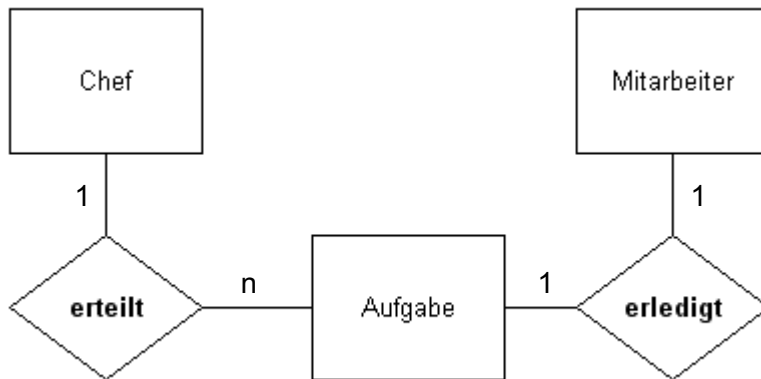
Attribut



Kardinalitäten

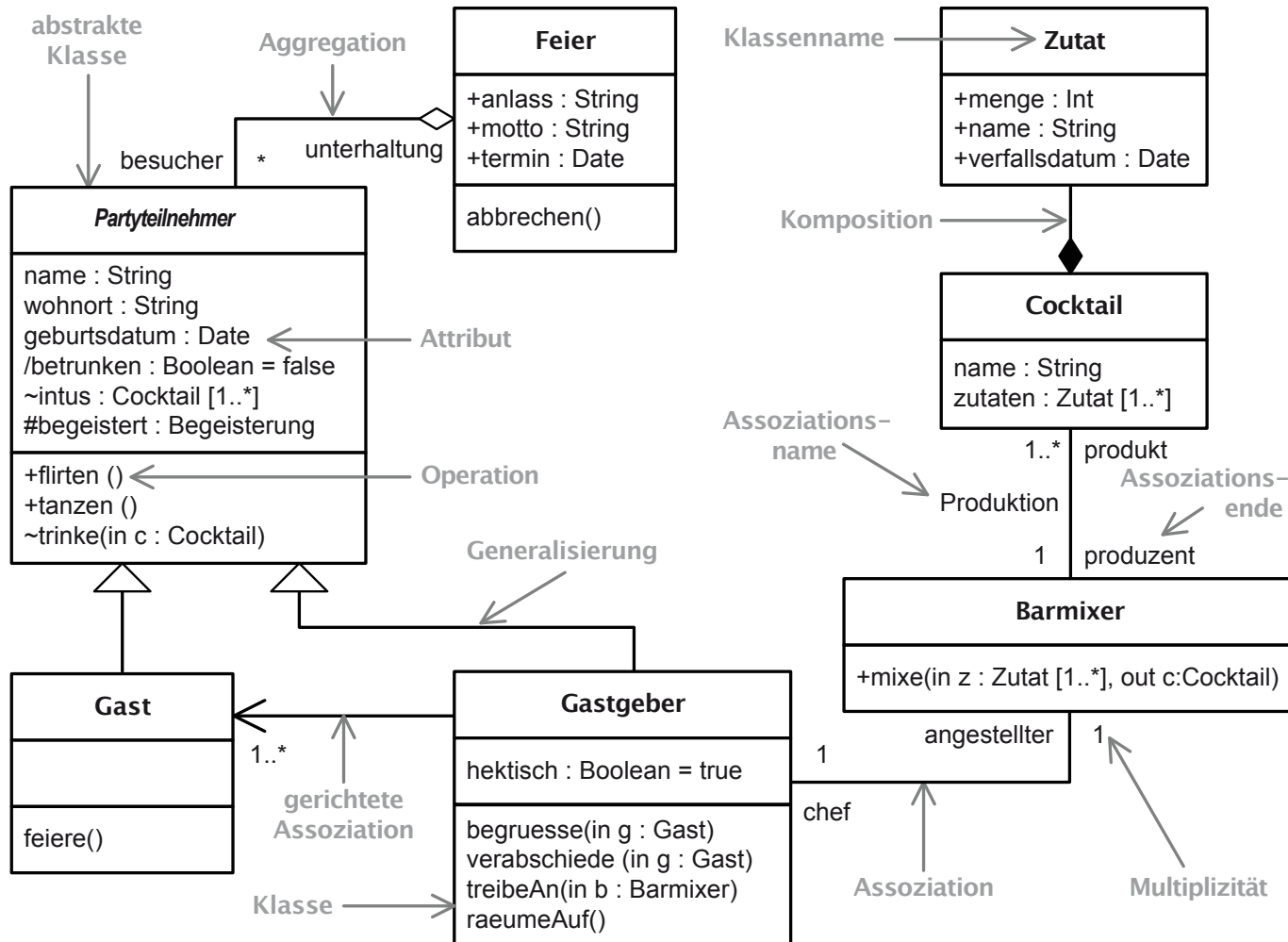


Strukturperspektive – Beispiel ER-Diagramm



6. Anforderungen modellbasiert dokumentieren

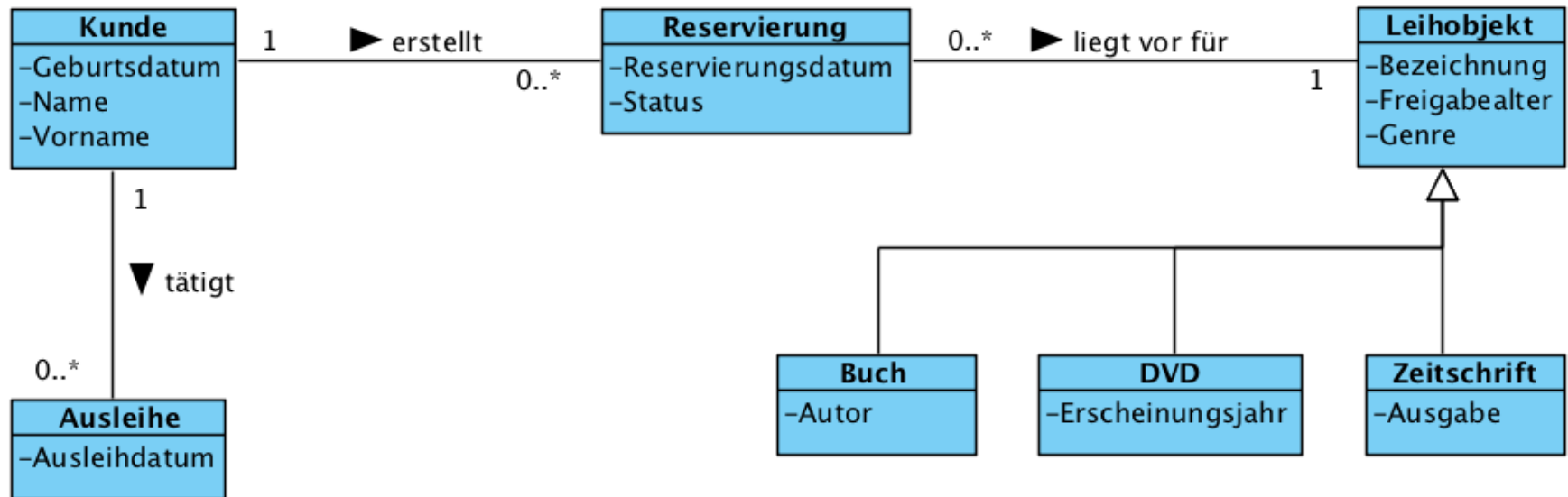
Elementes des Klassendiagramms



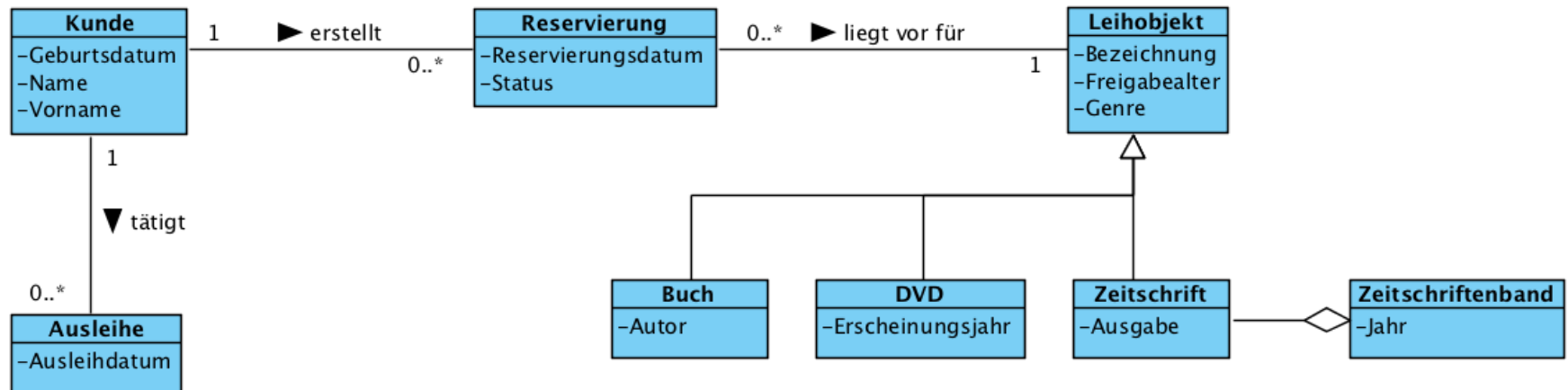
6. Anforderungen modellbasiert dokumentieren

Strukturperspektive

UML-Klassendiagramm – Beispiel Generalisierung



UML-Klassendiagramm – Beispiel Aggregation

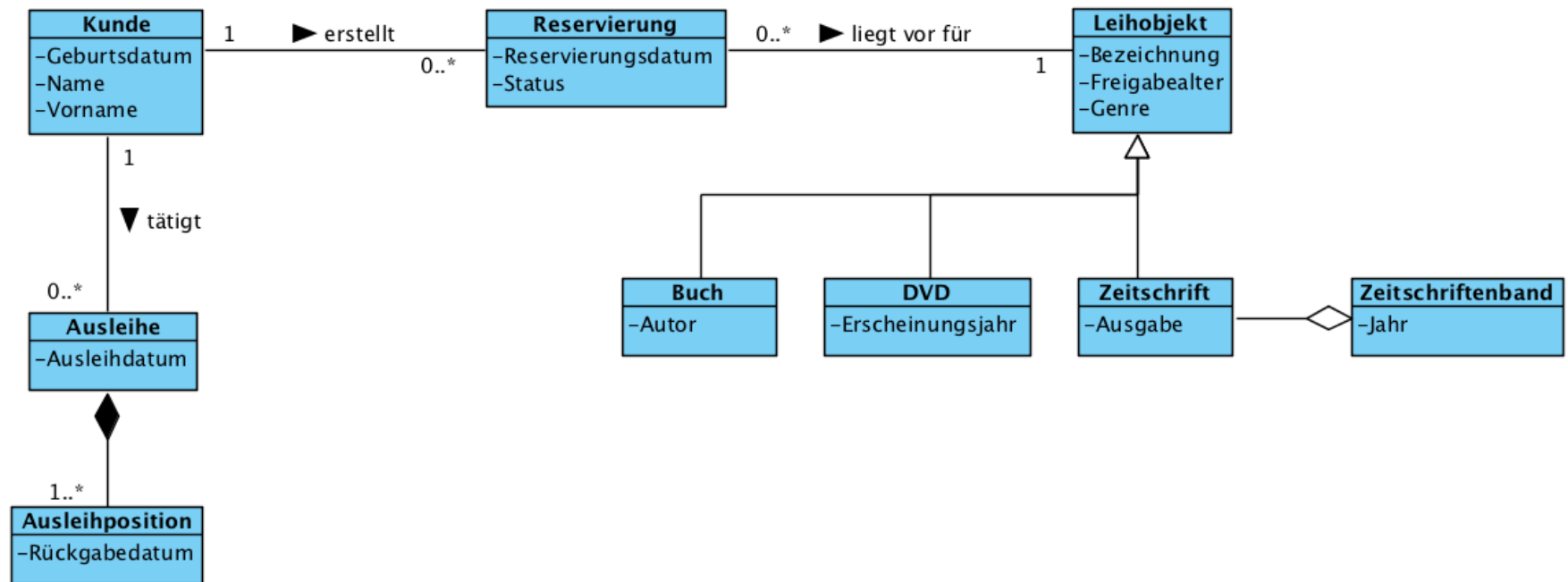


- Die Aggregation ist eine „Teil / Ganzes“ – Beziehung
- Entfällt das „Ganze“, so sind die „Teile“ dennoch weiterhin relevant und bestehen weiter

6. Anforderungen modellbasiert dokumentieren

Strukturperspektive

UML-Klassendiagramm – Beispiel Komposition



- Die Komposition ist eine „Teil / Ganzes“ – Beziehung
- Entfällt das „Ganze“, so sind auch die „Teile“ hinfällig

6. Anforderungen modellbasiert dokumentieren

UML-Klassendiagramm – Übungsaufgabe

Ein Onlineshop beinhaltet Artikel, die von Kunden bestellt werden können. Ein Kunde besitzt eine Kundennummer, einen Vornamen, einen Nachnamen und ein oder zwei Anschriften (jeweils bestehend aus Straße, PLZ und Ort). Aus Sicherheitsgründen kann ein Kunde nur maximal 10 Artikel bestellen.

Die Artikel besitzen eine Artikelnummer und einen Preis (bestehend aus einem Betrag und einer Währung). Sie werden von Redakteuren angelegt. Ein Redakteur kann beliebig viele Artikel anlegen. Einzelne Redakteure besitzen zudem ein Löschrecht und können beliebig viele Artikel auch wieder löschen. Daher existieren die Rollen „Redakteur“ und „Senior Redakteur“.

Alle Redakteure besitzen eine Personalnummer.

6. Anforderungen modellbasiert dokumentieren

UML-Klassendiagramm – Übungsaufgabe 2

Gegeben sei ein Onlineshop-System, dessen Nutzer Kunden und Administratoren sind. Kunden können Bestellungen mit einer beliebigen Anzahl von Bestellpositionen aufgeben. Zu jeder Bestellung wird im zeitlichen Verlauf eine Versandinformation generiert, die den Kunden über den Status dieser Bestellung informiert.

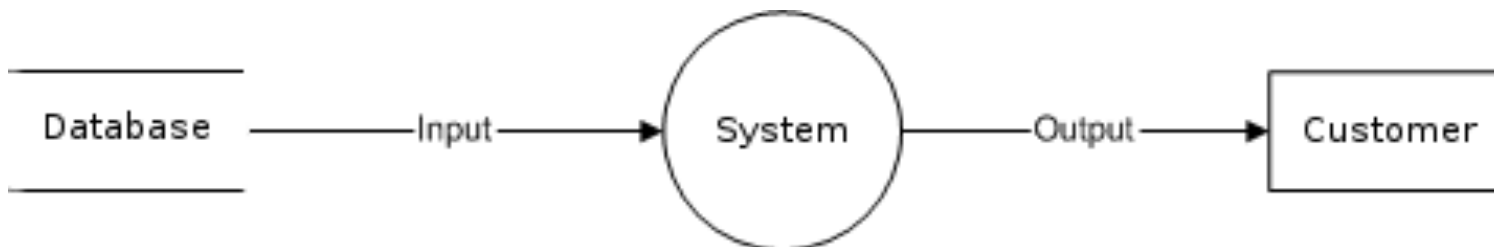
Entwerfen Sie ein Datenmodell in Form eines Klassendiagramms für das beschriebene Shopsystem. Geben Sie für die Assoziationen die Multiplizitäten an.

Funktionsperspektive

- In der Funktionsperspektive wird dokumentiert, welche Informationen aus dem Systemkontext durch das zu entwickelnde System bzw. dessen Funktionen manipuliert werden und welche Daten vom System in den Systemkontext fließen.
- Mögliche Modellierungssprachen sind:
 - **Datenflussdiagramme** (nach DeMarco): Datenflussdiagramme beschreiben den Transport von Daten zwischen Prozessen, Datenspeichern und Personen, Personengruppen oder Systemen.
 - **UML-Aktivitätsdiagramme**: Mit Aktivitätsdiagrammen werden (zeitliche) Abläufe modelliert.

Funktionsperspektive

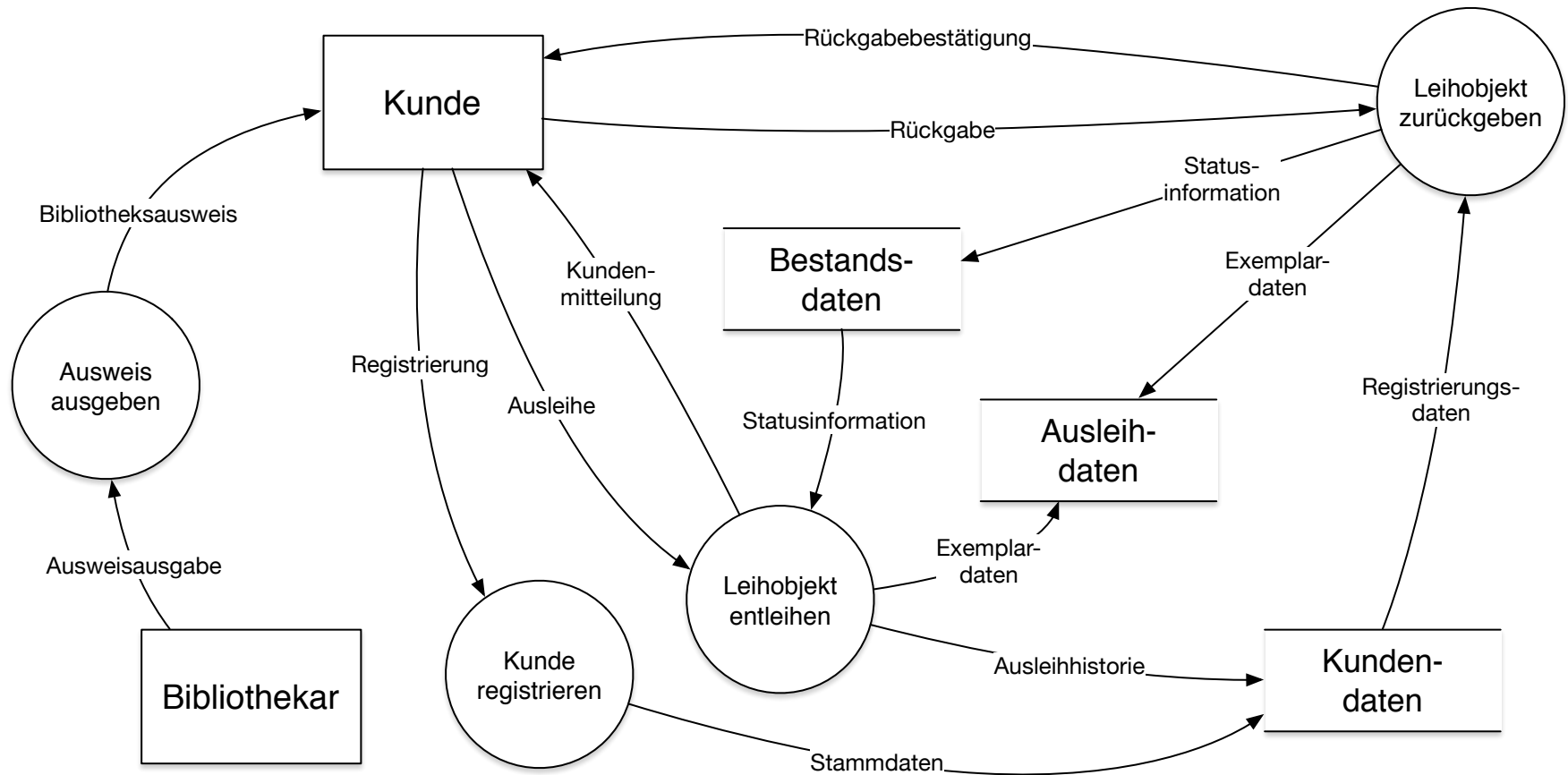
- Das Datenflussdiagramm besitzt folgende Komponenten:
 - **Datenspeicher:** dargestellt durch zwei parallele Linien, zwischen denen der Speichername steht
 - **Datenfluss:** dargestellt durch einen Pfeil mit Namen (greift eine Funktion lesend und schreibend auf einen Datenspeicher zu kann man dies entweder mit zwei getrennten Pfeilen oder mit einem Doppelpfeil darstellen)
 - **Funktion (oder Prozess):** dargestellt durch einen Kreis mit Namen
 - **Schnittstelle zur Umwelt:** dargestellt durch ein Rechteck, das den Schnittstellennamen enthält (externer Partner). Schnittstellen, an denen Daten in das System einfließen, werden Datenquellen genannt. Schnittstellen, an denen Daten aus dem System verschwinden heißen Datensenken.



6. Anforderungen modellbasiert dokumentieren

Funktionsperspektive

Datenflussdiagramm – Beispiel:



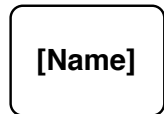
Funktionsperspektive

- Das UML-Aktivitätsdiagramm besitzt folgende Komponenten:
 - Aktion
 - Startknoten
 - Endknoten
 - Bedingung
 - Kontrollfluss
 - Objektfluss
 - Entscheidung (Entscheidungsknoten / Zusammenführung)
 - Nebenläufigkeit (fork / join)
 - Hierarchisierung

6. Anforderungen modellbasiert dokumentieren

Funktionsperspektive

UML-Aktivitätsdiagramm – Notationselemente



Aktion



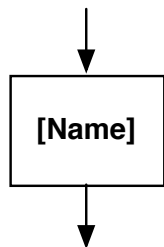
Startknoten



Endknoten

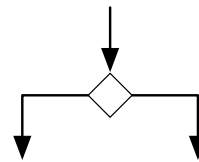


Kontrollfluss



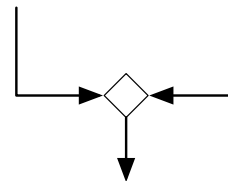
Objektfluss

Entscheidung



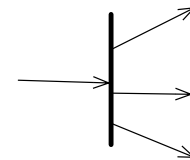
Entscheidungsknoten

[condition] Bedingung

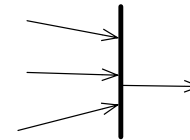


Zusammenführung
alternativer
Kontrollflüsse

Nebenläufigkeit

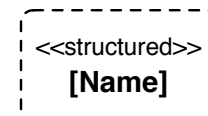


Fork



Join

Hierarchisierung

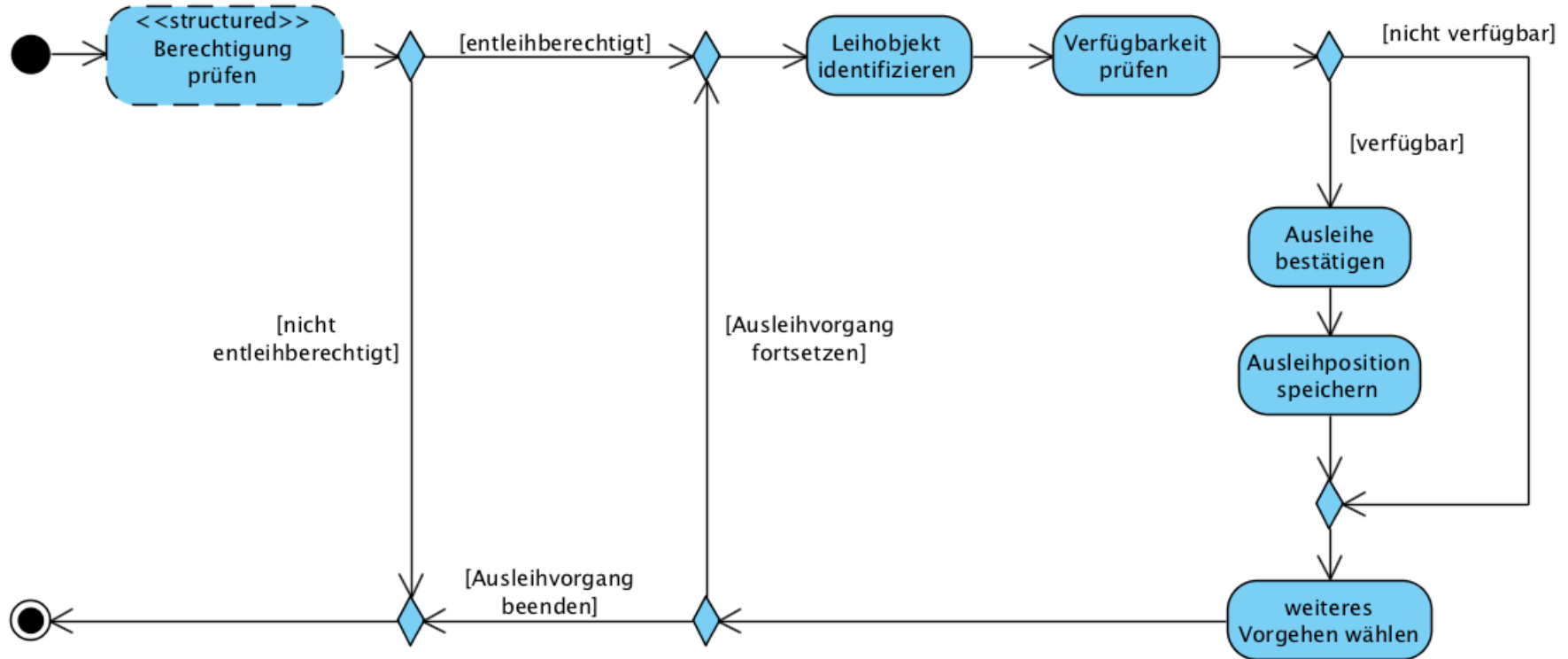


Komplexe
Aktion

6. Anforderungen modellbasiert dokumentieren

Funktionsperspektive

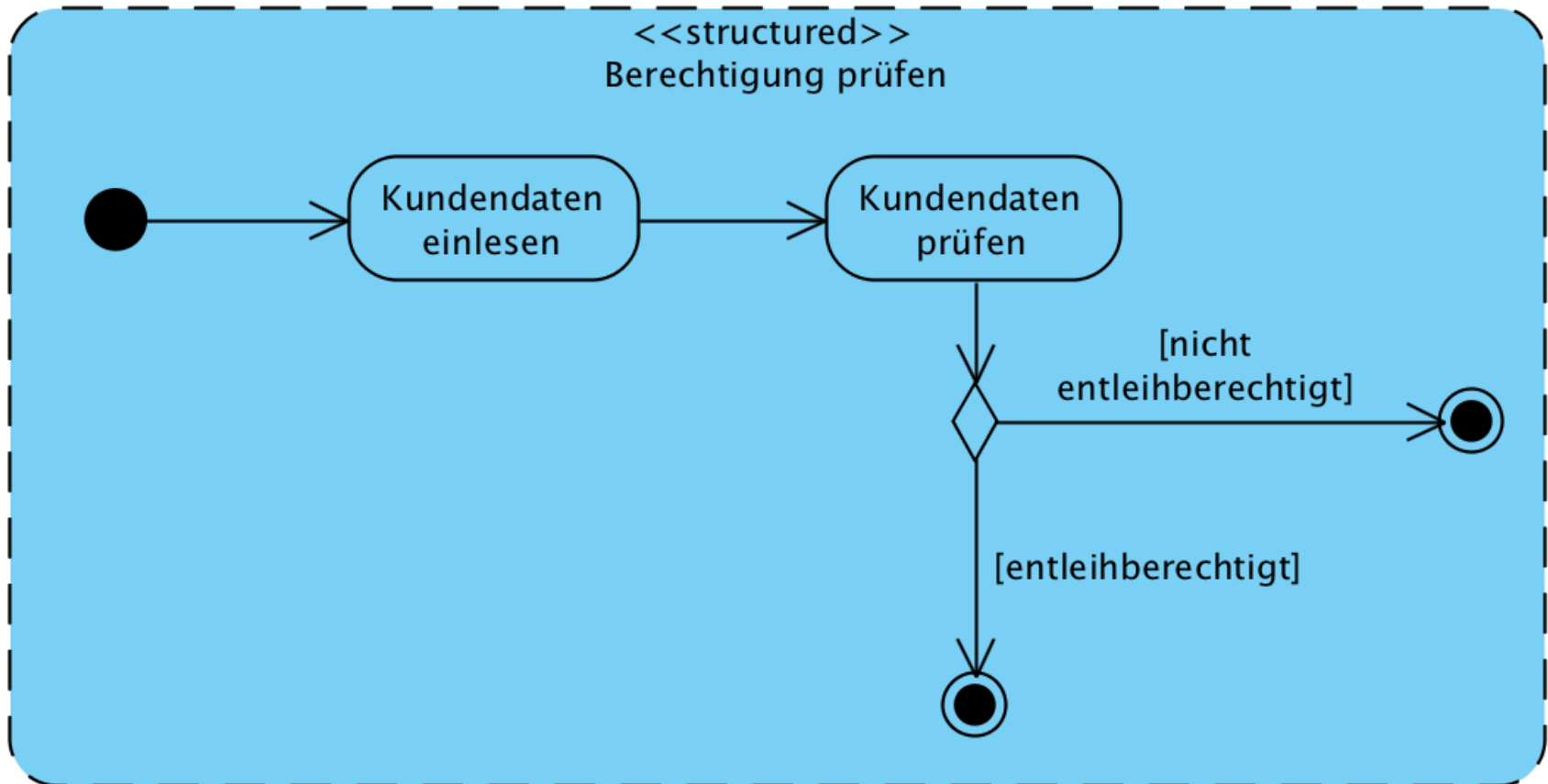
UML-Aktivitätsdiagramm – einfacher Ablauf:



6. Anforderungen modellbasiert dokumentieren

Funktionsperspektive

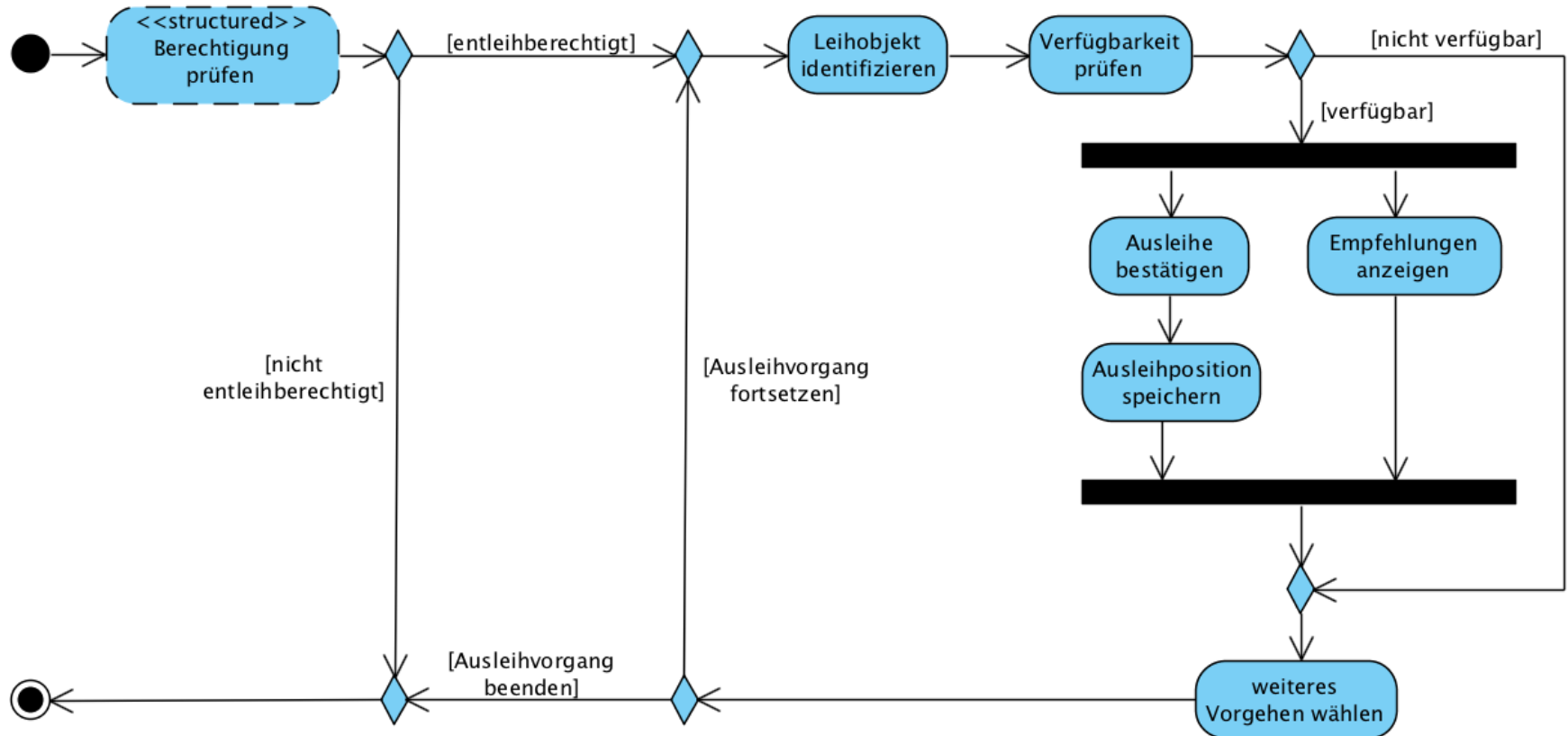
UML-Aktivitätsdiagramm –Aufruf einer Aktivität: (Verfeinerung Berechtigung prüfen)



6. Anforderungen modellbasiert dokumentieren

Funktionsperspektive

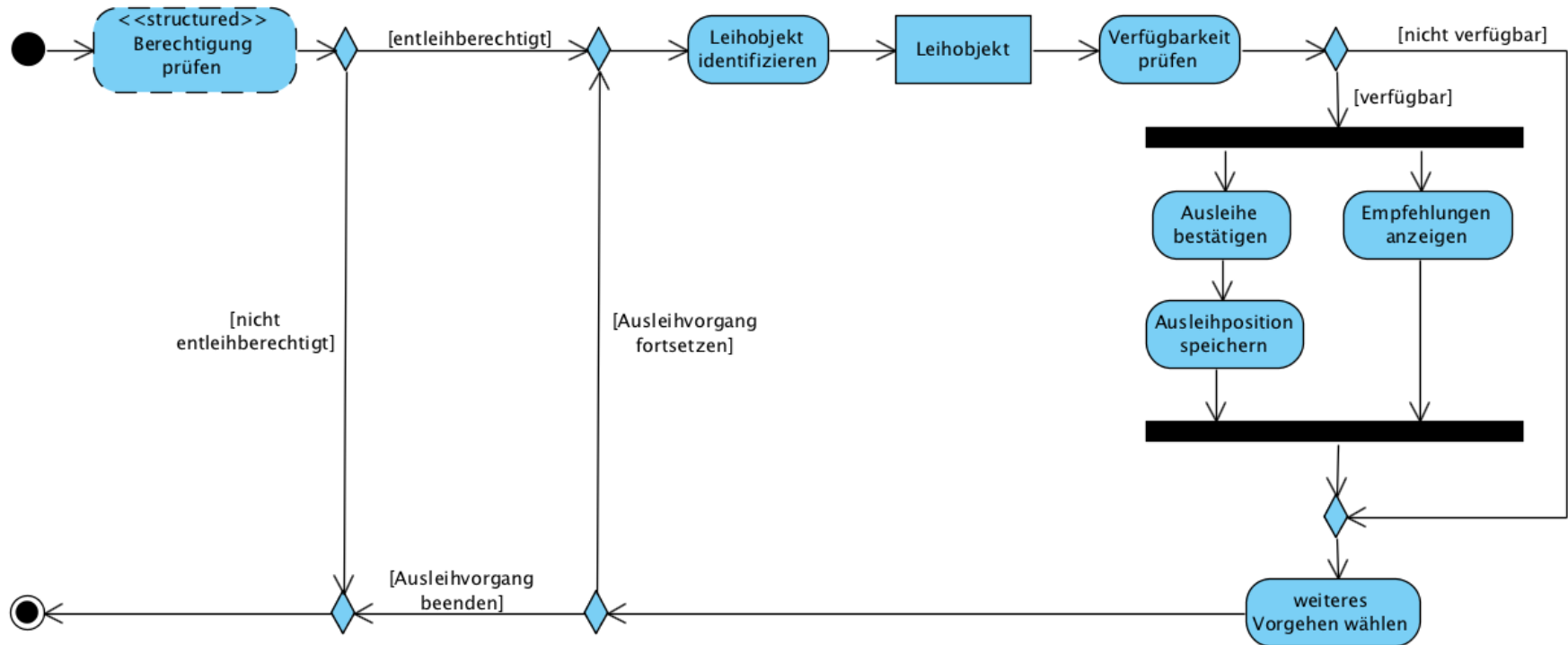
UML-Aktivitätsdiagramm – Fork und Join:



6. Anforderungen modellbasiert dokumentieren

Funktionsperspektive

UML-Aktivitätsdiagramm – Objektfluss:



6. Anforderungen modellbasiert dokumentieren

Funktionsperspektive

UML-Aktivitätsdiagramm – Übungsaufgabe

Der Kunde betritt zunächst den Onlineshop, um hiernach Artikel in einen Warenkorb zu legen. Wenn er dann auf „Kaufen“ klickt, fragt das System ihn, ob er ein registrierter Kunde sei. Ist dies der Fall, kann er sich direkt einloggen, ansonsten muss er sich zunächst noch registrieren.

Nachdem der Kunde eingeloggt ist, kann er die Artikel bezahlen. Danach werden zeitgleich drei Mitteilungen versandt: Eine Bestellbestätigung an den Kunden, ein Hinweis an das Lager und ein Hinweis an die Inkasso-Abteilung. Nach dem Versand der Mitteilungen ist der Prozess beendet.

6. Anforderungen modellbasiert dokumentieren

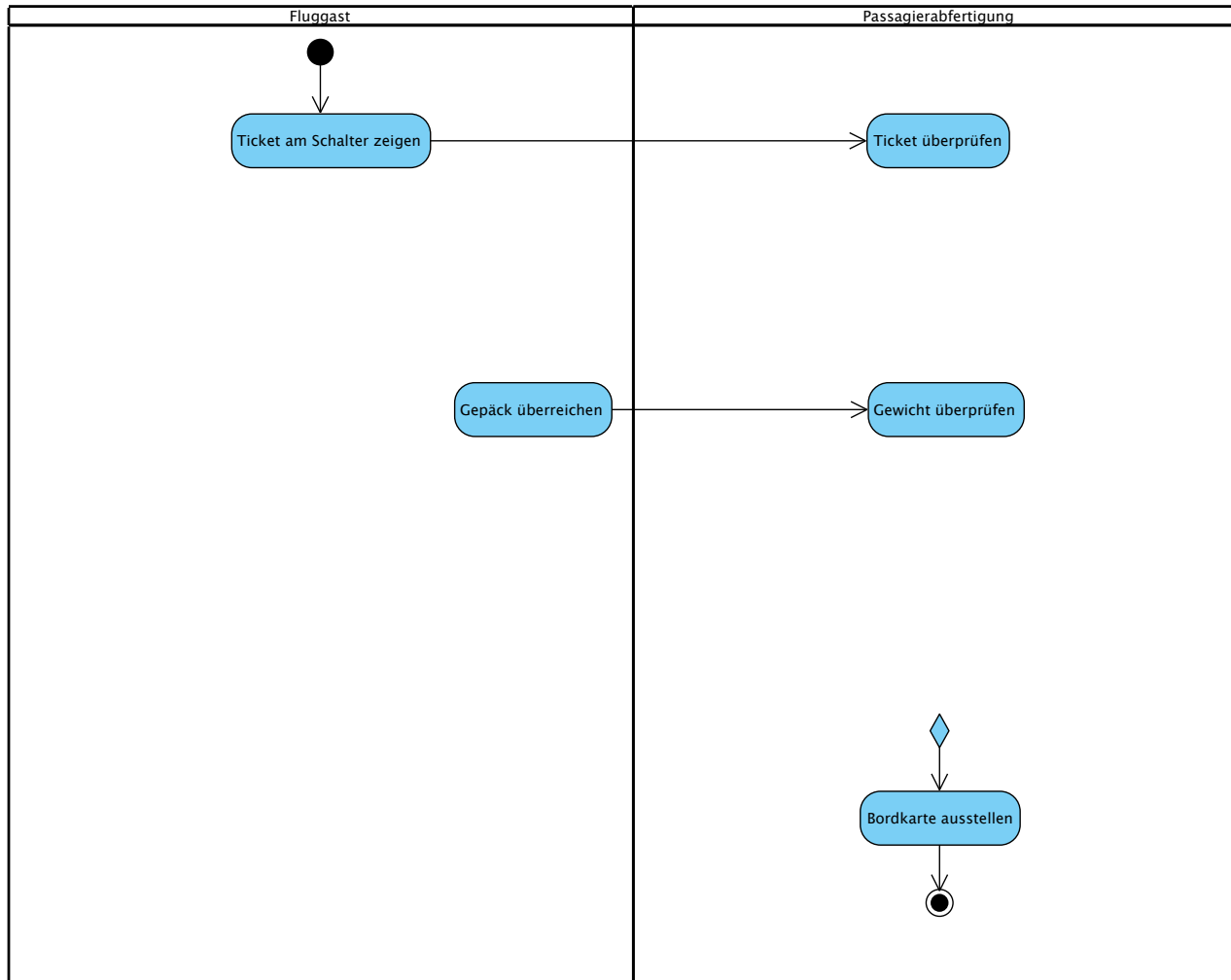
Funktionsperspektive

UML-Aktivitätsdiagramm – Übungsaufgabe 2

- Ein Fluggast ist am Flughafen angekommen. Zur Überprüfung seines Tickets begibt er sich zum Schalter seiner Fluggesellschaft.
- Falls das Ticket in Ordnung ist, übergibt er am Schalter sein Gepäck.
- Falls mit dem Ticket etwas nicht stimmt, muss der Fluggast den Kundendienst konsultieren und er kann nicht mitfliegen.
- Das Gepäck wird zudem auf Übergewicht überprüft. Falls dem so ist, muss der Fluggast zusätzliche Kosten übernehmen. Falls aber das Gewicht in Ordnung ist, wird die Bordkarte ausgestellt.

Modellieren Sie diesen Sachverhalt im folgenden unvollendeten UML-Aktivitätsdiagramm.

UML-Aktivitätsdiagramm – Übungsaufgabe 2 (Zusatzaufgabe für Schnelle)



Verhaltensperspektive

- In der Verhaltensperspektive wird das System und dessen Einbettung in den Systemkontext zustandsorientiert dokumentiert, indem bspw. die Reaktion des Systems auf Ereignisse im Systemkontext, Bedingungen eines Zustandswechsels sowie Effekte dokumentiert werden, die das System in der Umgebung erbringen soll.
- Mögliche Modellierungssprachen sind:
 - Statecharts (nach Harel)
 - **UML-Zustandsdiagramme**

Verhaltensperspektive

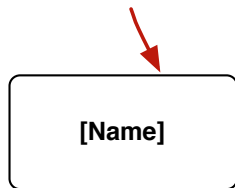
- Das UML-Zustandsdiagramm besitzt u.a. folgende Komponenten:
 - Startzustand / finaler Zustand
 - Zustand
 - Hierarchisierung
 - Nebenläufige Statecharts
 - Zustandsübergang

6. Anforderungen modellbasiert dokumentieren

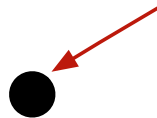
Verhaltensperspektive

UML-Zustandsdiagramm – Notationselemente:

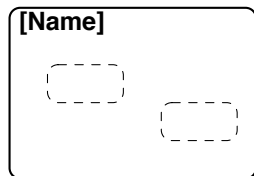
Zustand



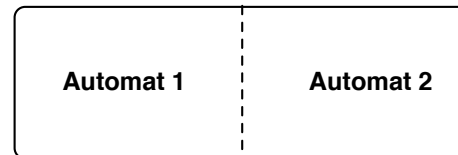
Startzustand



Finaler Zustand

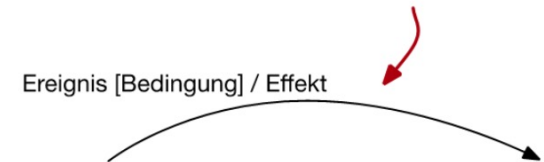


Hierarchisierung



Nebenläufige
Zustandsmaschinen

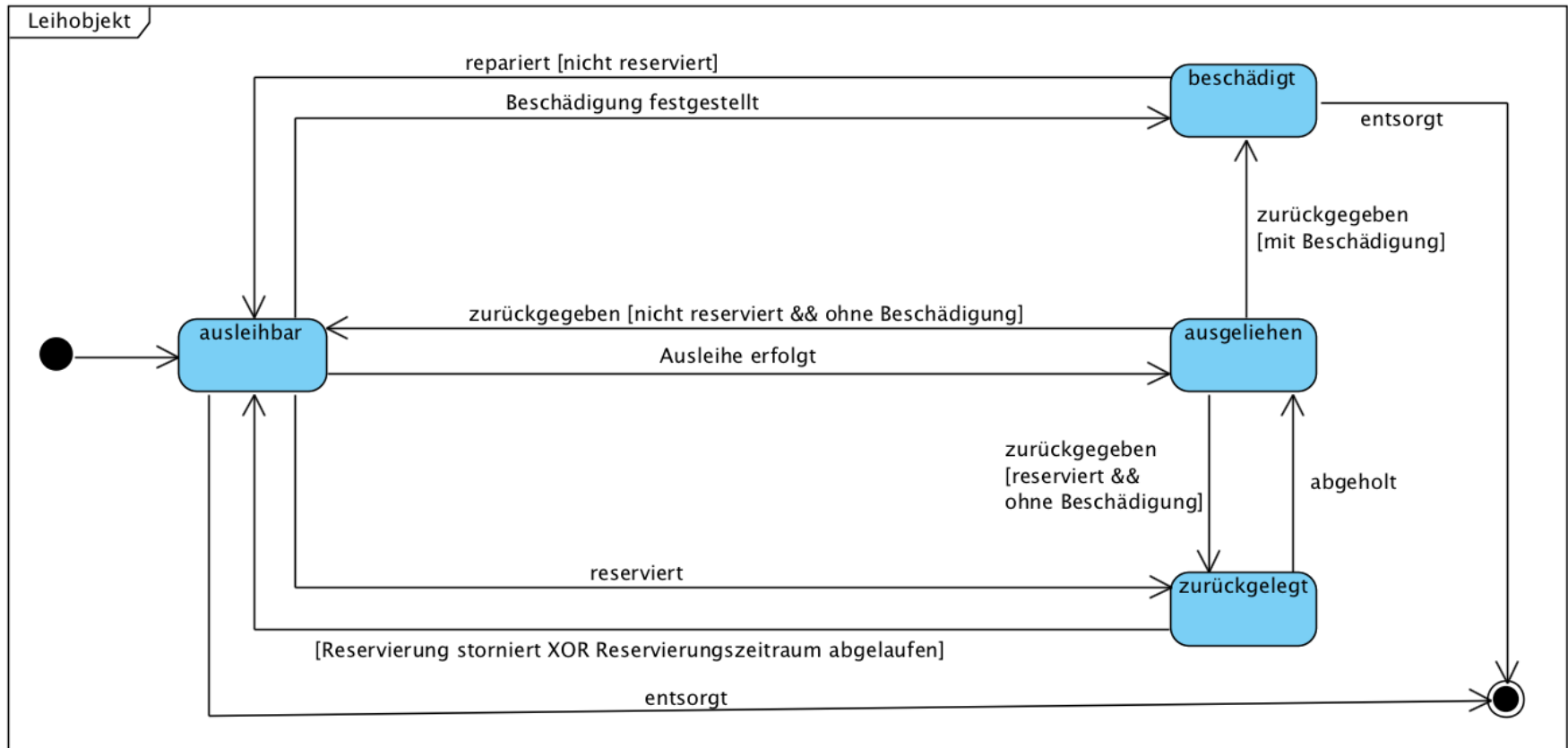
Zustandsübergang



6. Anforderungen modellbasiert dokumentieren

Verhaltensperspektive

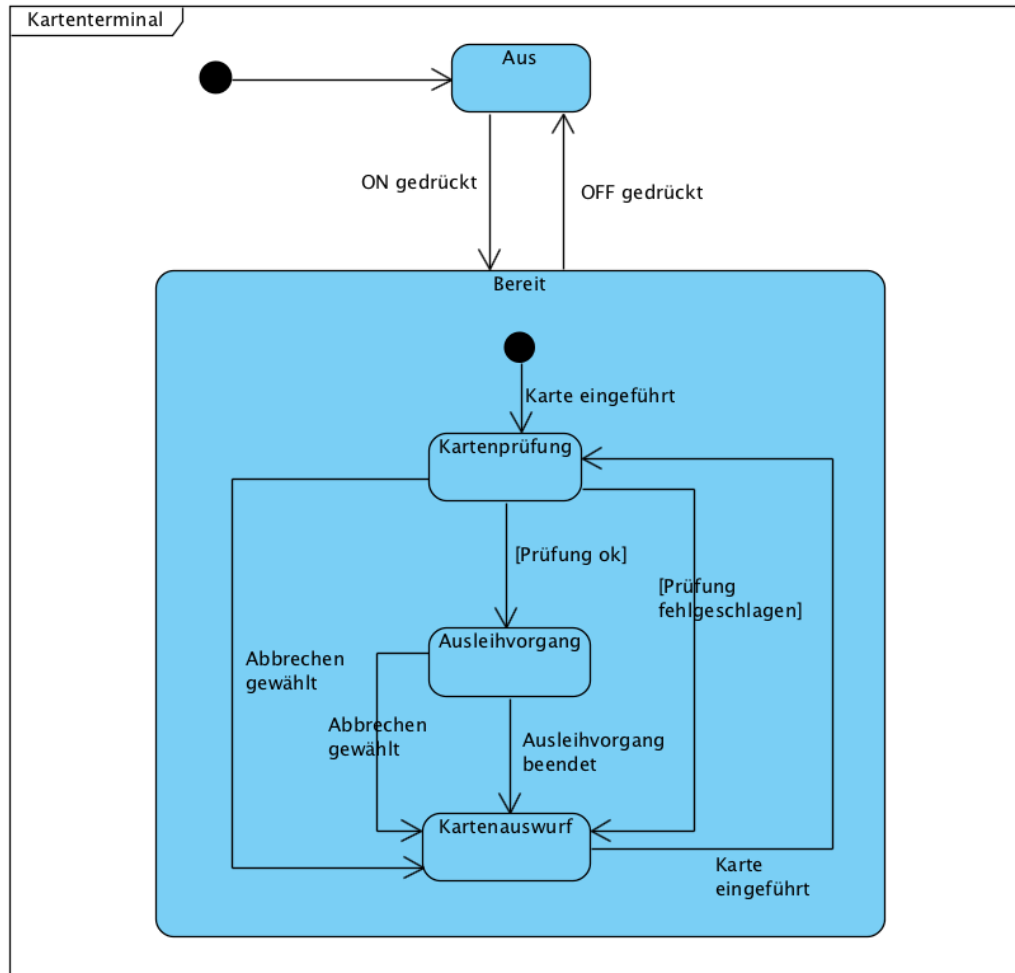
UML-Zustandsdiagramm – Beispiel 1:



6. Anforderungen modellbasiert dokumentieren

Verhaltensperspektive

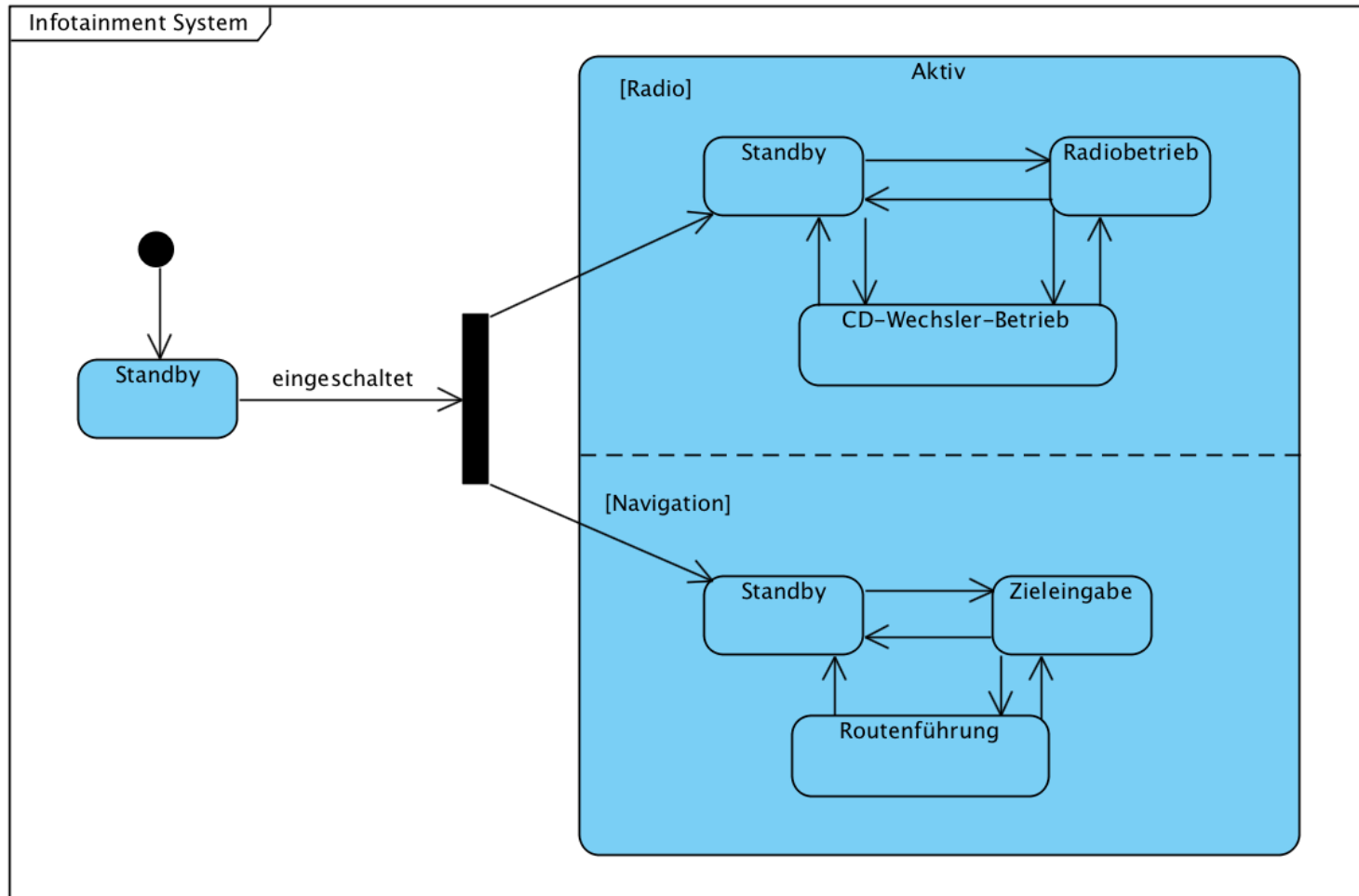
UML-Zustandsdiagramm – Hierarchisierung:



6. Anforderungen modellbasiert dokumentieren

Verhaltensperspektive

UML-Zustandsdiagramm – Nebenläufigkeit:



6. Anforderungen modellbasiert dokumentieren

Verhaltensperspektive

UML-Zustandsdiagramm – Übungsaufgabe:

Wenn ein Artikel in einen Warenkorb gelegt worden ist, dann wird er im System als reserviert vermerkt. Eine Reservierung bedeutet in diesem Zusammenhang, dass sich die Lagerbestandsgröße um diesen einen Artikel reduziert, damit nicht mehr Artikel verkauft werden können, als verfügbar sind.

Wenn der Artikel im Warenkorb liegt, dann kann er vom Kunden entweder gekauft oder wieder aus diesem entfernt werden.

6. Anforderungen modellbasiert dokumentieren

Verhaltensperspektive

UML-Zustandsdiagramm – Übungsaufgabe 2:

Modellieren Sie die Zustände einer Ampel aus dem Straßenverkehr.

Zusammenfassung

- Definition Modell
- Zielmodelle erstellen
- Use Cases erstellen
- Anforderungsmodellierung durchführen
 - Strukturperspektive
 - Funktionsperspektive
 - Verhaltensperspektive

Agenda

1. Einleitung und Grundlagen
2. System und Systemkontext abgrenzen
3. Anforderungen ermitteln
4. Anforderungen dokumentieren
5. Anforderungen natürlichsprachig dokumentieren
6. Anforderungen modellbasiert dokumentieren
7. **Anforderungen prüfen und abstimmen**
8. Anforderungen verwalten
9. Werkzeugunterstützung

7. Anforderungen prüfen und abstimmen

Prüfung von Anforderungen

- Im Rahmen der Überprüfung von Anforderungen wird die Entscheidung getroffen, ob eine Anforderung die nötige Qualität aufweist, um für die Entwicklung freigegeben zu werden.
- Ziel der Überprüfung ist es daher, Fehler wie Mehrdeutigkeit, Widersprüche und Unvollständigkeit aufzudecken.
- Es gilt, eine Fehlerfortpflanzung zu vermeiden und Auftragsvereinbarungen erfüllen zu können.

7. Anforderungen prüfen und abstimmen

Abstimmung von Anforderungen

- Widersprüchliche Anforderungen erzeugen Konflikte.
- Ungelöste Konflikte können dazu führen, dass einzelne Stakeholder die Entwicklung des Systems nicht weiter unterstützen.
- Bei der Lösungsfindung von Konflikten können jedoch auch neue Ideen entstehen.
- Ziel der Abstimmung von Anforderungen ist es, unter den relevanten Stakeholdern ein gemeinsames und übereinstimmendes Verständnis bezüglich der Anforderungen an das zu entwickelnde System herbeizuführen.
- Die Abstimmung (wie auch die Prüfung) sollte fortwährend geschehen. Die hierbei entstehenden Kosten sind vertretbar im Gegensatz zur Fehlerbehebung oder Konfliktlösung während der Entwicklungsphase.

7. Anforderungen prüfen und abstimmen

Qualitätsaspekte für Anforderungen

- Die Überprüfung von Anforderungen sollte anhand der folgenden drei Qualitätsaspekte vollzogen werden:
 - „Inhalt“: Wurden alle relevanten Anforderungen ermittelt und im erforderlichen Detaillierungsgrad erfasst?
 - „Dokumentation“: Wurden die Anforderungen gemäß der festgelegten Dokumentations- und Spezifikationsvorschriften dokumentiert?
 - „Abgestimmtheit“: Stimmen alle Stakeholder mit den dokumentierten Anforderungen überein und sind alle bekannten Konflikte aufgelöst?

7. Anforderungen prüfen und abstimmen

Qualitätsaspekt „Inhalt“

- Der Qualitätsaspekt „Inhalt“ besteht aus folgenden acht Prüfkriterien:
 - Vollständigkeit (Menge aller Anforderungen)
 - Vollständigkeit (einzeller Anforderung)
 - Verfolgbarkeit
 - Korrektheit / Adäquatheit
 - Konsistenz
 - Keine vorzeitigen Entwurfsentscheidungen
 - Überprüfbarkeit
 - Notwendigkeit

7. Anforderungen prüfen und abstimmen

Qualitätsaspekt „Dokumentation“

- Die Missachtung von Dokumentationsvorschriften führt u.a. zu folgenden Risiken:
 - Behinderung von Entwicklungsaktivitäten
 - Unverständlichkeit von Anforderungen
 - Unvollständigkeit
 - Übersehen von Anforderungen
- Die Prüfung kann anhand folgender Kriterien erfolgen:
 - Konformität zum Dokumentationsformat
 - Konformität zur Dokumentenstruktur
 - Verständlichkeit
 - Eindeutigkeit
 - Konformität mit Dokumentationsregeln

7. Anforderungen prüfen und abstimmen

Qualitätsaspekt „Abgestimmtheit“

- Der Qualitätsaspekt „Abgestimmtheit“ bezieht sich auf die Überprüfung von Anforderungen auf Mängel in der Abstimmung der Anforderungen unter den relevanten Stakeholdern.
- Im Rahmen der Prüfung von Anforderungen haben Stakeholder (letztmalig) die Möglichkeit, Änderungswünsche zu äußern, ohne die nachfolgenden Entwicklungsaktivitäten zu beeinträchtigen.
- Die Prüfung kann anhand folgender Kriterien erfolgen:
 - Abstimmung
 - Abstimmung nach Änderungen
 - Konflikte aufgelöst?

7. Anforderungen prüfen und abstimmen

Prüfungsprinzipien

- Die Beachtung der nachstehenden sechs Prinzipien verbessert die Qualität der Prüfungsergebnisse:
 - Prinzip 1: Beteiligung der richtigen Stakeholder
 - Prinzip 2: Trennung von Fehlersuche und Fehlerkorrektur
 - Prinzip 3: Prüfung aus unterschiedlichen Sichten
 - Prinzip 4: Geeigneter Wechsel der Dokumentationsform
 - Prinzip 5: Konstruktion von Entwicklungsartefakten
 - Prinzip 6: Wiederholte Prüfung

7. Anforderungen prüfen und abstimmen

Prüfungsprinzipien

- Prinzip 1: Beteiligung der richtigen Stakeholder
 - Die Auswahl von Stakeholdern für die Prüfung von Anforderungen richtet sich nach dem Ziel der Prüfung sowie nach den zu prüfenden Anforderungen.
 - Es sollte vermieden werden, dass eine Anforderung durch den Ersteller geprüft wird.
 - Die Prüfung kann durch interne oder externe Prüfer vorgenommen werden.
- Prinzip 2: Trennung von Fehlersuche und Fehlerkorrektur
 - Die Trennung zwischen der Suche nach Fehlern und der eigentlichen Korrektur des Fehlers ist ein bewährtes Prinzip bei der Qualitätssicherung und kann auch auf die Prüfung von Anforderungen übertragen werden.
 - Der Vorteil liegt in der höheren „Konzentration“ und der frühzeitigeren Identifikation von Fehlern.

7. Anforderungen prüfen und abstimmen

Prüfungsprinzipien

- Prinzip 3: Prüfung aus unterschiedlichen Sichten
 - Perspektivenbasierte Prüfung
- Prüfung 4: Geeigneter Wechsel der Dokumentationsform
 - Die Überführung einer bereits dokumentierten Anforderung in eine andere Dokumentationsform erleichtert das Auffinden von Fehlern.
- Prüfung 5: Konstruktion von Entwicklungsartefakten
 - Die Konstruktion von Entwicklungsartefakten zielt darauf ab, die Qualität von Anforderungen während der Umsetzung zu prüfen.

7. Anforderungen prüfen und abstimmen

Prüfungsprinzipien

- Prinzip 6: Wiederholte Prüfung
 - Da sich Wissensstände ändern können, sollten Prüfung wiederholt durchgeführt werden.
 - Eine Wiederholung ist insbesondere in folgenden Fällen wichtig:
 - Hoher Innovationsanteil im System
 - Hoher Wissenszugewinn während des Requirements Engineering.
 - Längerfristige Projekte
 - Sehr frühe Prüfung der Anforderungen
 - Unbekannte Domäne
 - Wiederverwendung von Anforderungen

7. Anforderungen prüfen und abstimmen

Techniken

- Die Techniken zur Prüfung von Anforderungen lassen sich in sechs verschiedene Techniken unterteilen. Die drei erstgenannten Techniken lassen sich unter dem Sammelbegriff *Review* subsummieren:
 - Stellungnahme
 - Inspektion
 - Walkthrough
 - Perspektivenbasiertes Lesen
 - Prüfung durch Prototypen
 - Einsatz von Checklisten

7. Anforderungen prüfen und abstimmen

Techniken

■ Stellungnahme

- Bei der Stellungnahme werden die Anforderungen an eine dritte Person gegeben, die diese hinsichtlich ihrer Qualität prüft.
- Identifizierte Mängel werden entsprechend gekennzeichnet.

■ Inspektion

- Inspektionen haben das Ziel, Anforderungen anhand eines strikten Schemas systematisch nach Fehlern zu durchsuchen.
- Es werden verschiedene Phasen durchgeführt: Planung, Übersicht, Fehlersuche, Fehlersammlung (sowie Fehlerkorrektur, Nachkontrolle, Reflexion).
- In der Planungsphase werden bspw. das Ziel der Inspektion, die zu inspizierenden Arbeitsergebnisse, die Art der Durchführung der Inspektion und die Rollen bzw. Teilnehmer der Inspektion festgelegt.

7. Anforderungen prüfen und abstimmen

Techniken

- Fortsetzung Inspektion
 - In der Übersichtsphase erhalten die Teilnehmer einen Überblick über die zu prüfenden Anforderungen.
 - In der Fehlersuchphase durchsuchen die Inspektoren die Anforderungen nach Fehlern (individuell und / oder im Team).
 - In der Phase Fehlersammlung werden die identifizierten Fehler zusammen-getragen, konsolidiert und dokumentiert. Hierbei können insbesondere doppelte und unechte Fehler beseitigt werden.
 - Mögliche Rollen:
 - Organisator
 - Moderator
 - Autor
 - Vorleser
 - Inspektoren
 - Protokollant

7. Anforderungen prüfen und abstimmen

Techniken

- Walkthrough
 - Ein Walkthrough ähnelt einer Inspektion, wird aber weniger formell durchgeführt. Es existieren mindestens die Rollen Reviewer, Autor, Protokollant und ggf. Moderator.
 - Ein Walkthrough von Anforderungen zielt darauf ab, in einem gemeinsamen Prozess Qualitätsmängel in den Anforderungen zu identifizieren (und ggf. unter den Teilnehmern zu konsolidieren).
 - In der Walkthrough-Sitzung besprechen die Teilnehmer (ggf. angeleitet durch den Moderator) schrittweise die zu überprüfenden Anforderungen.
 - Der Autor oder die Autoren haben die Möglichkeit, die Anforderungen zu erläutern (kritisch!).
 - Fehler werden dokumentiert.

7. Anforderungen prüfen und abstimmen

Techniken

- Perspektivenbasiertes Lesen
 - Das perspektivenbasierte Lesen ist eine Technik zur Prüfung von Anforderungen, bei der die zu prüfenden Anforderungen unter verschiedenen Perspektiven geprüft werden.
 - Mögliche Perspektiven könnten bspw. sein:
 - Adressatensicht
 - Kunde / Nutzer
 - Softwarearchitekt
 - Tester
 - Qualitätssicht
 - Inhalt
 - Dokumentation
 - Abgestimmtheit

7. Anforderungen prüfen und abstimmen

Techniken

- Fortsetzung Perspektivenbasiertes Lesen
 - Bei der perspektivenbasierten Überprüfung wird jedem Prüfer (zu einem Zeitpunkt) eine Perspektive zugeordnet, aus deren Blickwinkel der Prüfer die Anforderung liest und überprüft.
 - Im Rahmen der Nachbereitung des perspektivenbasierten Lesens werden die Ergebnisse der gewählten Perspektiven ausgewertet und konsolidiert.
 - Das perspektivenbasierte Lesen kann sowohl als eigenständige Technik als auch als Hilfsmittel für andere Prüfungstechniken verwendet werden.

7. Anforderungen prüfen und abstimmen

Techniken

- Prototypen
 - Die Prüfung von Anforderungen durch Prototypen verfolgt den Ansatz, die Anforderungen für den Prüfer erlebbar und damit ausprobierbar zu machen.
 - Es kann zwischen Wegwerfprototypen und evolutionären Prototypen unterschieden werden.
 - Vor der Realisierung des Prototyps müssen die Anforderungen ausgewählt werden, die mit dem Prototyp überprüft werden sollen.
 - Als Vorbereitung für die Prüfung mittels eines Prototyps dienen:
 - Handbuch / Schulung
 - Prüfszenarien
 - Checkliste mit Prüfkriterien

7. Anforderungen prüfen und abstimmen

Techniken

- Fortsetzung Prototypen
 - Im Rahmen der Prüfung sollten die Prüfer den Prototyp möglichst ohne äußere Beeinflussung benutzen.
 - Im Rahmen der Prüfung soll der Nutzer auch die Möglichkeit haben, explorativ zu agieren.
 - Die Ergebnisse der Prüfung werden sowohl vom Prüfer wie auch vom Beobachter dokumentiert, hiernach kann eine Auswertung erfolgen.

7. Anforderungen prüfen und abstimmen

Techniken

- Checklisten
 - Checklisten können immer dann eingesetzt werden, wenn bei einem komplexen Sachverhalt viele Aspekte berücksichtigt werden müssen und hierbei möglichst kein Aspekt vergessen werden soll.
 - Mögliche Quellen für Checklistenpunkte:
 - Die drei Qualitätsaspekte
 - Prinzipien der Prüfung
 - Qualitätskriterien für Anforderungsdokumente
 - Qualitätskriterien für die jeweiligen Anforderungen
 - Erfahrungen der Prüfer aus bisherigen Projekten
 - Fehlerstatistiken

7. Anforderungen prüfen und abstimmen

Techniken

- Fortsetzung Checklisten
 - Während der Verwendung einer Checkliste sollte stets nach Möglichkeiten gesucht werden, diese zu verbessern.
 - Die Checkliste kann auch gut als Leitfaden oder Strukturierungsmittel für andere Prüfungstechniken verwendet werden.
 - Die erfolgreiche Verwendung von Checklisten hängt von der Handbarkeit bzw. Komplexität ab.
 - Zu viele Fragen (mehr als eine Seite) sowie zu generische Fragen sollten vermieden werden.

7. Anforderungen prüfen und abstimmen

Abstimmung von Anforderungen

- Zur Abstimmung der Anforderungen an ein zu entwickelndes System ist es notwendig, etwaige Konflikte zu identifizieren und zu lösen.
- Dies geschieht im Rahmen eines Konfliktmanagements mit folgenden vier Aufgaben:
 - Konfliktidentifikation
 - Konfliktanalyse
 - Konfliktauflösung
 - Dokumentation der Konfliktauflösung

7. Anforderungen prüfen und abstimmen

Konfliktidentifikation

- Konflikte können in allen Requirements Engineering Aktivitäten auftreten (bspw. im Rahmen der Anforderungsermittlung sich gegenseitig ausschließende Anforderungen).
- Konflikte sind mitunter nicht offensichtlich erkennbar.
- Der Requirements Engineer sollte Anforderungen daher immer auch auf ihr Konfliktpotential hin betrachten.

7. Anforderungen prüfen und abstimmen

Konfliktanalyse

- Um einen Konflikt lösen zu können, muss zunächst die Ursache betrachtet werden.

- Es existieren unterschiedliche Arten von Konflikten:
 - Sachkonflikt
 - Interessenkonflikt
 - Wertekonflikt
 - Beziehungskonflikt
 - Strukturkonflikt
 - Vermischte Konfliktursachen

7. Anforderungen prüfen und abstimmen

Konfliktauflösung

- Der Konfliktauflösung wird im Rahmen der Einigung eine bedeutende Rolle zuteil, da die Art und Weise der Konfliktauflösung großen Einfluss auf die Kooperationsbereitschaft der beteiligten Akteure hat. Wichtig ist in jedem Fall die Einbeziehung aller relevanten Stakeholder.

- Folgende Konfliktauflösungstechniken können verwendet werden:
 - Einigung
 - Kompromiss
 - Abstimmung
 - Variantenbildung
 - Ober-sticht-Unter
 - Consider-all-Facts
 - Plus-Minus-Interesting
 - Entscheidungsmatrix

7. Anforderungen prüfen und abstimmen

Dokumentation der Konfliktlösung

- Die Auflösung eines Konflikts sollte in jedem Fall dokumentiert werden.
- Werden Konfliktauflösungen nicht dokumentiert, kann es zur wiederholten Behandlung des gleichen Konflikts oder ähnlichen Konflikten kommen. Darüber hinaus kann der (langfristige) Erfolg der Konfliktauflösung nur schwer überprüft werden.

7. Anforderungen prüfen und abstimmen

Zusammenfassung

- Grundlagen der Prüfung von Anforderungen
- Grundlagen der Abstimmung von Anforderungen
- Qualitätsaspekte für Anforderungen
- Prinzipien der Prüfung von Anforderungen
- Techniken zur Prüfung von Anforderungen
- Abstimmung von Anforderungen

Agenda

1. Einleitung und Grundlagen
2. System und Systemkontext abgrenzen
3. Anforderungen ermitteln
4. Anforderungen dokumentieren
5. Anforderungen natürlichsprachig dokumentieren
6. Anforderungen modellbasiert dokumentieren
7. Anforderungen prüfen und abstimmen
8. **Anforderungen verwalten**
9. Werkzeugunterstützung

8. Anforderungen verwalten

Attributierung von Anforderungen

- Über den gesamten Lebenszyklus eines Systems müssen Informationen über Anforderungen vorgehalten werden.
- Typische Informationen sind etwa ein Identifikator, der Name der Anforderung, Autor, Quelle etc.
- Zur Dokumentation der Informationen über Anforderungen hat es sich bewährt, die verschiedenen Informationen strukturiert (bspw. als Tabelle oder als Schablone) zu erfassen.

8. Anforderungen verwalten

Attributierung von Anforderungen

- Wichtige Attributtypen sind in der nachfolgenden Tabelle aufgeführt:

Attributtyp	Bedeutung
Identifikator	Kurze, eindeutige Information eines Anforderungsartefakts in der Menge der betrachteten Anforderungen.
Name	Eindeutiger, charakterisierende Name.
Beschreibung	Beschreibt in komprimierter Form den Inhalt der Anforderung.
Version	Aktueller Versionsstand der Anforderung.
Autor	Benennt den / die Autor(in) der Anforderung.
Quelle	Benennt die Quelle bzw. Quellen der Anforderung
Begründung	Beschreibt, weshalb diese Anforderung für das geplante System von Bedeutung ist.
Stabilität	Benennt die voraussichtliche Stabilität der Anforderung. Stabilität ist dabei der Umfang, in dem künftig noch Veränderungen bzgl. dieser Anforderungen erwartet werden. Mögliche Unterscheidung: „fest“, „gefestigt“, „volatil“.
Kritikalität	Im Sinne einer Abschätzung der Schadenshöhe und Eintrittswahrscheinlichkeit.
Priorität	Benennt die Priorität der Anforderung hinsichtlich der gewählten Merkmale der Priorisierung, z.B. „Bedeutung für die Akzeptanz am Markt“, „Reihenfolge der Umsetzung“, „Schaden bzw. Opportunitätskosten durch Nichtrealisierung“.

8. Anforderungen verwalten

Attributierung von Anforderungen

- Weitere mögliche Attributtypen:

Attributtyp	Bedeutung
Verantwortliche(r)	Benennt die Person, Stakeholdergruppe bzw. Organisation(seinheit), die für die Anforderung inhaltlich verantwortlich ist.
Anforderungstyp	Benennt abhängig vom eingesetzten Differenzierungsschema den Typ der Anforderung (z.B. „funktionale Anforderung“, „Qualitätsanforderung“ oder „Randbedingung“).
Status bzgl. des Inhalts	Benennt den aktuellen Status des Inhalts der Anforderung, z.B. „Idee“, „Konzept“, „detaillierter Inhalt“.
Status bzgl. der Überprüfung	Benennt den aktuellen Status der Validierung, z.B. „ungeprüft“, „in Prüfung“, „überprüft“, „fehlerhaft“, „in Korrektur“
Status bzgl. der Einigung	Benennt den aktuellen Status der Abstimmung, z.B. „nicht abgestimmt“, „abgestimmt“, „konfliktär“.

8. Anforderungen verwalten

Attributierung von Anforderungen

- Weitere mögliche Attributtypen:

Attributtyp	Bedeutung
Aufwand	Prognostizierter oder tatsächlicher Umsetzungsaufwand der Anforderung
Release	Nummer des Releases, in dem die Anforderung umgesetzt werden soll.
Juristische Verbindlichkeit	Gibt den Grad der juristischen Verbindlichkeit der Anforderung an.
Querbezüge	Benennt die Beziehungen zu anderen Anforderungen. Zum Beispiel wenn bekannt ist, dass die Realisierung dieser Anforderung die vorherige Realisierung einer anderen Anforderung voraussetzt.
Allgemeine Informationen	In diesem Attribut können beliebige, für relevant erachtete Informationen zu dieser Anforderung dokumentiert werden. Zum Beispiel wenn die Abstimmung dieser Anforderung beim nächsten Treffen mit dem Auftraggeber vorgesehen ist.

8. Anforderungen verwalten

Attributierung von Anforderungen

- Die vorgeschlagenen Attributtypen bilden die Basis zur Definition der Attributierungsschemata. Hierauf aufbauend können gegebenenfalls folgende Aspekte berücksichtigt werden:
 - Spezifische Merkmale des Projekts, z.B. Projektgröße, lokale bzw. verteilte Entwicklung oder Projektrisiko.
 - Vorgaben seitens des Unternehmens, z.B. Unternehmensvorschriften und Unternehmensstandards.
 - Eigenschaften und Vorschriften des Anwendungsgebiets, z.B. Referenzmodelle, Modellierungsvorschriften, Standards.
 - Randbedingungen und Restriktionen des Entwicklungsprozesses, z.B. Haftungsrecht und Prozessstandards.

8. Anforderungen verwalten

Attributierung von Anforderungen

- Als Alternative zur schematischen / tabellarischen Darstellung der Attribute, kann auch ein modellbasierter Ansatz gewählt werden.
- Hierbei können auch Beziehungen visualisiert werden.
- Zudem können selektive Sichten unter Berücksichtigung der Konsistenz erzeugt werden.

8. Anforderungen verwalten

Sichten auf Anforderungen

- In der Praxis zeigt sich, dass die Anzahl der Anforderungen (sowie ihre Abhängigkeiten) in Projekten immer mehr zunehmen. Um die Komplexität beherrschbar zu halten, müssen die Anforderungen daher (je nach Rolle) selektiv oder verdichtet dargestellt werden.
- Eine Sicht kann hierbei
 - bestimmte Anforderungen selektieren, d.h., nicht alle Anforderungen der Anforderungsbasis sind in der Sicht enthalten;
 - bestimmte Attribute von Anforderungen ausblenden, d.h., nicht alle Attribute einer Anforderung sind in der Sicht enthalten;
 - die beiden genannten Selektionsprinzipien miteinander kombinieren
- Bei einer Verdichtung werden einzelne Informationen generisch zusammengefasst (bspw. prozentuale Verteilung einzelner Attributwerte).

8. Anforderungen verwalten

Priorisierung von Anforderungen

- Zur Priorisierung von Anforderungen wird im ersten Schritt das Ziel / der Gegenstand der Priorisierung definiert. Zusätzlich können Randbedingungen formuliert werden.
- Ausgehend vom Ziel der Priorisierung wird festgelegt welche Priorisierungskriterien zu berücksichtigen sind. Typische Kriterien sind etwa:
 - Kosten für die Umsetzung
 - Risiko
 - Schaden bei nicht erfolgreicher Umsetzung
 - Volatilität
 - Wichtigkeit
 - Zeitdauer für die Umsetzung

8. Anforderungen verwalten

Priorisierung von Anforderungen

- Abhängig vom Ziel der Priorisierung ist es in der Regel notwendig, unterschiedliche Stakeholder in den Priorisierungsprozess miteinzubeziehen.
- Zudem müssen die zu priorisierenden Anforderungen ausgewählt werden. Hierbei ist darauf zu achten, dass es sich um Anforderungen auf dem gleichen Level handelt.
- Auf Basis der festgelegten Eigenschaften der Priorisierung (z.B. Randbedingungen, Priorisierungskriterien etc.) wird eine geeignete Priorisierungstechnik bzw. eine geeignete Kombination von Priorisierungstechniken ausgewählt.

8. Anforderungen verwalten

Priorisierung von Anforderungen

- Zur Priorisierung von Anforderungen existieren mehrere Techniken, die sich im Wesentlichen durch den zur Priorisierung benötigten Aufwand sowie in ihrer Eignung bezüglich der verschiedenen Priorisierungskriterien und Projektmerkmale unterscheiden.
- Es kann zwischen Ad-hoc-Techniken und analytischen Techniken (z.B. Analytical Hierarchy Processing, Kosten-Wert-Analyse und Quality Function Deployment) unterteilt werden.
- Folgende Techniken sind empfehlenswert:
 - Ranking
 - Top-Ten-Technik
 - Ein-Kriteriumsklassifikation
 - Kano-Klassifikation
 - Wiegers'sche Priorisierungsmatrix

8. Anforderungen verwalten

Priorisierung von Anforderungen

- Ranking
 - Beim Ranking wird von den ausgewählten Stakeholdern im Hinblick auf ein bestimmtes Kriterium eine Rangfolge der zu priorisierenden Anforderungen festgelegt.
- Top-Ten-Technik
 - Bei der Priorisierung mittels der Top-Ten-Technik werden für eine vorher festgelegte Anzahl n im Hinblick auf das betrachtete Kriterium die n wichtigsten Anforderungen ausgewählt.
 - Für diese Anforderungen wird anschließend eine Rangfolge festgelegt, die die Bedeutung der jeweiligen Anforderungen hinsichtlich der gewählten Kriterien widerspiegelt.

8. Anforderungen verwalten

Priorisierung von Anforderungen

■ Ein-Kriteriums-Klassifikation

- Eine in der Praxis häufig verwendete Priorisierungstechnik basiert auf einer Klassifizierung von Anforderungen im Hinblick auf die Wichtigkeit der Realisierung dieser Anforderungen für den Erfolg des Systems:
 - Mandatory
 - Optional
 - Nice-to-have

■ Kano-Klassifikation

- Unter Verwendung des Kano-Ansatzes können Anforderungen im Hinblick auf deren Marktwirkung klassifiziert und priorisiert werden:
 - Basisfaktor -> Basismerkmal
 - Leistungsfaktor -> Leistungsmerkmale
 - Begeisterungsfaktor -> Begeisterungsmerkmal

8. Anforderungen verwalten

Priorisierung von Anforderungen

Wiegerts'sche Priorisierungsmatrix

- Die Wiegerts'sche Priorisierungsmatrix ist eine analytische Priorisierungstechnik die speziell für die Priorisierung von Anforderungen entwickelt wurde.
- Die Technik basiert auf dem Vergleich des Nutzens, des Nachteils, der Kosten und des Risikos einer Anforderung im Vergleich zu dem Nutzen, Nachteil, Kosten und Risiko aller anderen Anforderungen.
- Eine detaillierte Beschreibung der Anwendung dieser Priorisierungstechnik können Sie nachlesen in: *Wiegerts, K.E.: Software Requirements. 3rd edition. Microsoft Press Books, Unterschleißheim, 2013*

8. Anforderungen verwalten

Priorisierung von Anforderungen

■ Wiegers'sche Priorisierungsmatrix

Relatives Gewicht	-> 2 (Gewicht Nutzen)	-> 1 (Gewicht Nachteil)			-> 1 (Gewicht Kosten)		-> 0,5 (Gewicht Risiko)			
Anforderung	Relativer Nutzen	Relativer Nachteil	Gesamt	Wert %	Relative Kosten	Kosten %	Relatives Risiko	Risiko %	Priorität	Rang
R1	5	3	13	16,8	2	13,3	1	9,1	0,941	1
R2	9	7	25	32,5	5	33,3	3	27,2	0,692	3
R3	5	7	17	22,1	3	20,0	2	18,2	0,759	2
R4	2	1	5	6,5	1	6,7	1	9,1	0,577	4
R5	4	9	17	22,1	4	26,7	4	36,4	0,489	5
Gesamt	25	27	77	100	15	100	11	100		

$$Priorität(R_i) = \frac{Wert\%(R_i)}{Kosten\%(R_i) \times GewichtKosten + Risiko\%(R_i) \times GewichtRisiko}$$

8. Anforderungen verwalten

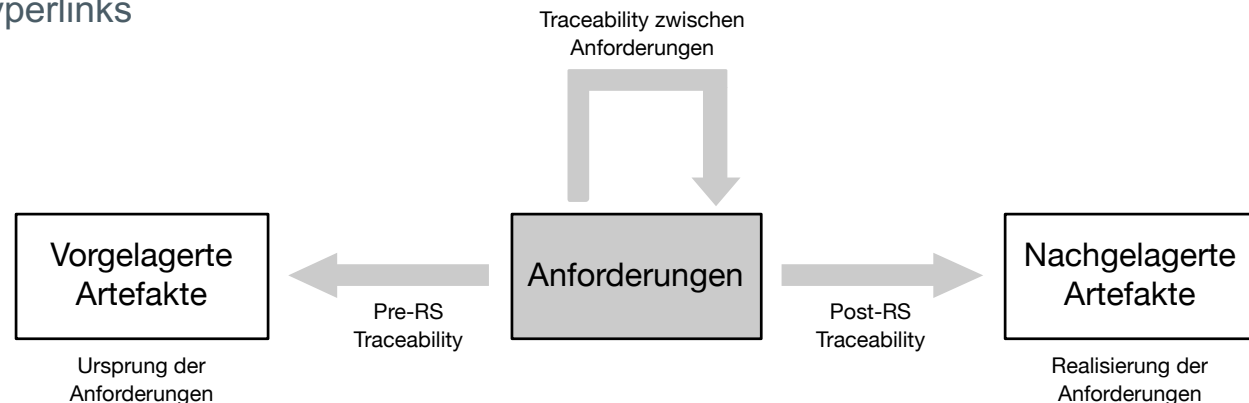
Verfolgbarkeit von Anforderungen

- Die Verfolgbarkeit einer Anforderung ist die Fähigkeit, eine Anforderung über den gesamten Lebenszyklus des Systems hinweg nachvollziehen zu können.
- Die Verfolgbarkeit von Anforderungen unterstützt den Nachweis, dass eine Anforderung im System umgesetzt, d.h. durch ein Systemmerkmal realisiert wurde
- Die Verfolgbarkeit von Anforderungen unterstützt die Identifikation von sogenannten Goldrandlösungen im entwickelten System.
- Die Verfolgbarkeit von Anforderungen unterstützt die Auswirkungsanalyse im Änderungsmanagement, indem beispielsweise bei der Änderung einer Anforderung durch die Verfolgbarkeit die Entwicklungsartefakte identifiziert werden können, die diese Anforderung realisieren und somit möglicherweise von der Anforderung betroffen sind.

8. Anforderungen verwalten

Verfolgbarkeit von Anforderungen

- Die Verfolgbarkeit von Anforderungen äußert sich in deren Beziehungen zu Entwicklungsartefakten oder anderen Anforderungen.
- Es können drei verschiedene Verfolgbarkeitsbeziehungen unterschieden werden:
 - Pre-RS-Traceability
 - Post-RS-Traceability
 - Traceability zwischen Anforderungen
- Informationen zur Verfolgbarkeit von Anforderungen können auf verschiedene Art und Weise repräsentiert werden:
 - Textuelle Referenzen und Hyperlinks
 - Verfolgbarkeitsmatrizen
 - Verfolgbarkeitsgraphen



8. Anforderungen verwalten

Verfolgbarkeit von Anforderungen

- Textuelle Referenzen und Hyperlinks
 - Eine einfache Form der Repräsentation von Verfolgbarkeitsinformationen einer Anforderung besteht darin, das Zielartefakt als textuelle Referenz zu annotieren oder einen Hyperlink zu erstellen.
- Verfolgbarkeitsmatrizen
 - Die Zeilen einer Verfolgbarkeitsmatrix enthalten die Ausgangsartefakte (Anforderungen) und die Spalten die Zielartefakte (z.B. Quellen von Anforderungen, Entwicklungsartefakte, Anforderungen).
 - Dieses Vorgehen eignet sich jedoch nur für Projekte mit einer geringen Anzahl an Anforderungen.

8. Anforderungen verwalten

Verfolgbarkeit von Anforderungen

■ Beispiel Verfolgbarkeitsmatrizen

abgeleitet	Anforderung 1	Anforderung 2	Anforderung 3	Anforderung 4
Stakeholder 1	X	X		
Stakeholder 2				X
Stakeholder 3			X	

abgeleitet	Anforderung 1	Anforderung 2	Anforderung 3	Anforderung 4
Anforderung 1		X	X	
Anforderung 2	X	X		X
Anforderung 3	X			

8. Anforderungen verwalten

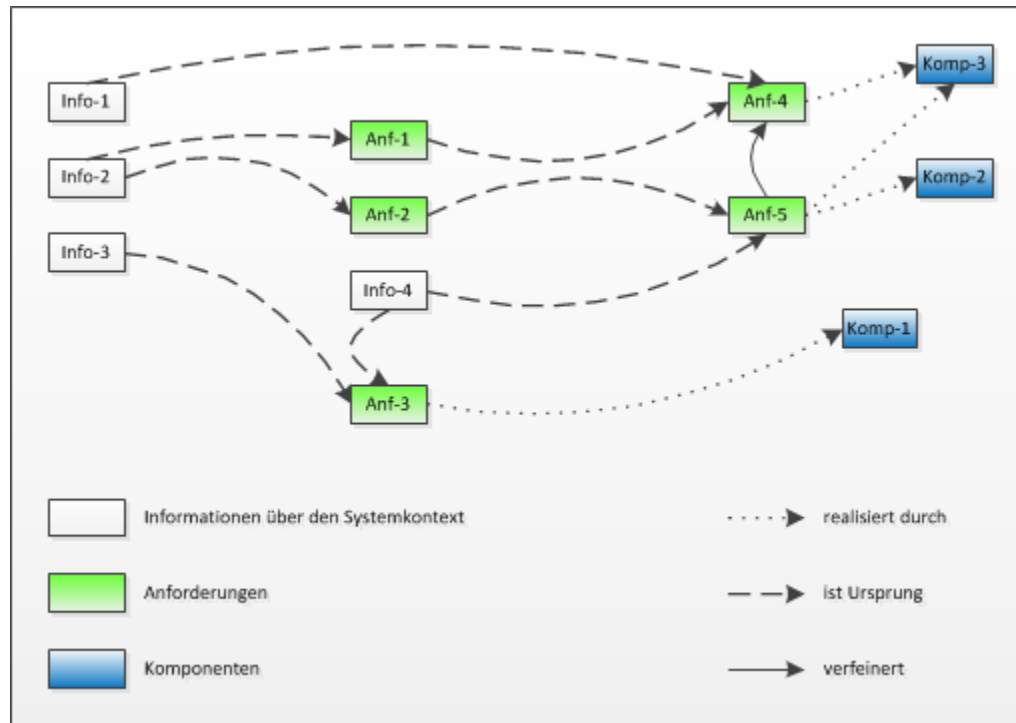
Verfolgbarkeit von Anforderungen

- Verfolgbarkeitsgraphen
 - Ein Verfolgbarkeitsgraph ist ein Graph, in dem die Knoten Artefakte und die Kanten Beziehungen zwischen den Artefakten darstellen.
 - Die Unterscheidung verschiedener Artefakte und Typen von Verfolgbarkeits-beziehungen kann durch entsprechende Attributierung der Knoten und Kanten des Graphen realisiert werden.
 - Werden auch Beziehungen zu vorgelagerten Artefakten (z.B. Stakeholder und Interviewprotokolle) und nachgelagerten Artefakten (z.B. Testfälle und Komponenten) verwaltet, so können Verfolgbarkeitsketten für die jeweilige Anforderung in verschiedenen Tiefen erstellt werden.

8. Anforderungen verwalten

Verfolgbarkeit von Anforderungen

■ Beispiel Verfolgbarkeitsgraphen



8. Anforderungen verwalten

Verfolgbarkeit von Anforderungen

- Die Verfolgbarkeit von Anforderungen unterstützt die Wiederverwendung von Entwicklungsartefakten in anderen Projekten.
- Die Verfolgbarkeit von Anforderungen unterstützt beispielsweise die nachträgliche Zuordnung von Entwicklungsaufwänden zu einer Anforderung.
- Die Verfolgbarkeit von Anforderungen unterstützt die Wartung eines Systems.
- Eine vollständige Erfassung aller denkbaren Informationen ist jedoch aufgrund von Ressourcenbeschränkungen kaum möglich, daher sollte eine projektspezifische Abgrenzung erfolgen.

8. Anforderungen verwalten

Versionierung von Anforderungen

- Über den Lebenszyklus eines Systems hinweg können Anforderungen hinzukommen und bestehende Anforderungen entfernt oder verändert werden.
- Die Versionierung von Anforderungen zielt darauf ab, über den gesamten Lebenszyklus eines Systems Zugriff auf die spezifischen Änderungsstände einzelner Anforderungen zu besitzen.
- Die Version einer Anforderung ist durch einen spezifischen inhaltlichen Änderungsstand dieser Anforderung definiert und wird durch eine eindeutige Versionsnummer gekennzeichnet.

8. Anforderungen verwalten

Versionierung von Anforderungen

- Bei der Versionierung von Anforderungen wird zwischen der Version und dem Inkrement einer Versionsnummer unterschieden (Versionsnummer x.y mit „x“ = Version und „y“ = Inkrement).
- Bei kleineren inhaltlichen Änderungen wird das Inkrement um 1 erhöht, bei größeren Änderungen wird die Version um 1 erhöht und das Inkrement auf 0 gesetzt.
- Optional kann der Versionsnummer ein „v“ vorangestellt werden (z.B. „v2.7“).
- Optional kann eine weitere Unterteilung mit einem Sub-Inkrement („z“) vorgenommen werden (Versionsnummer x.y.z).

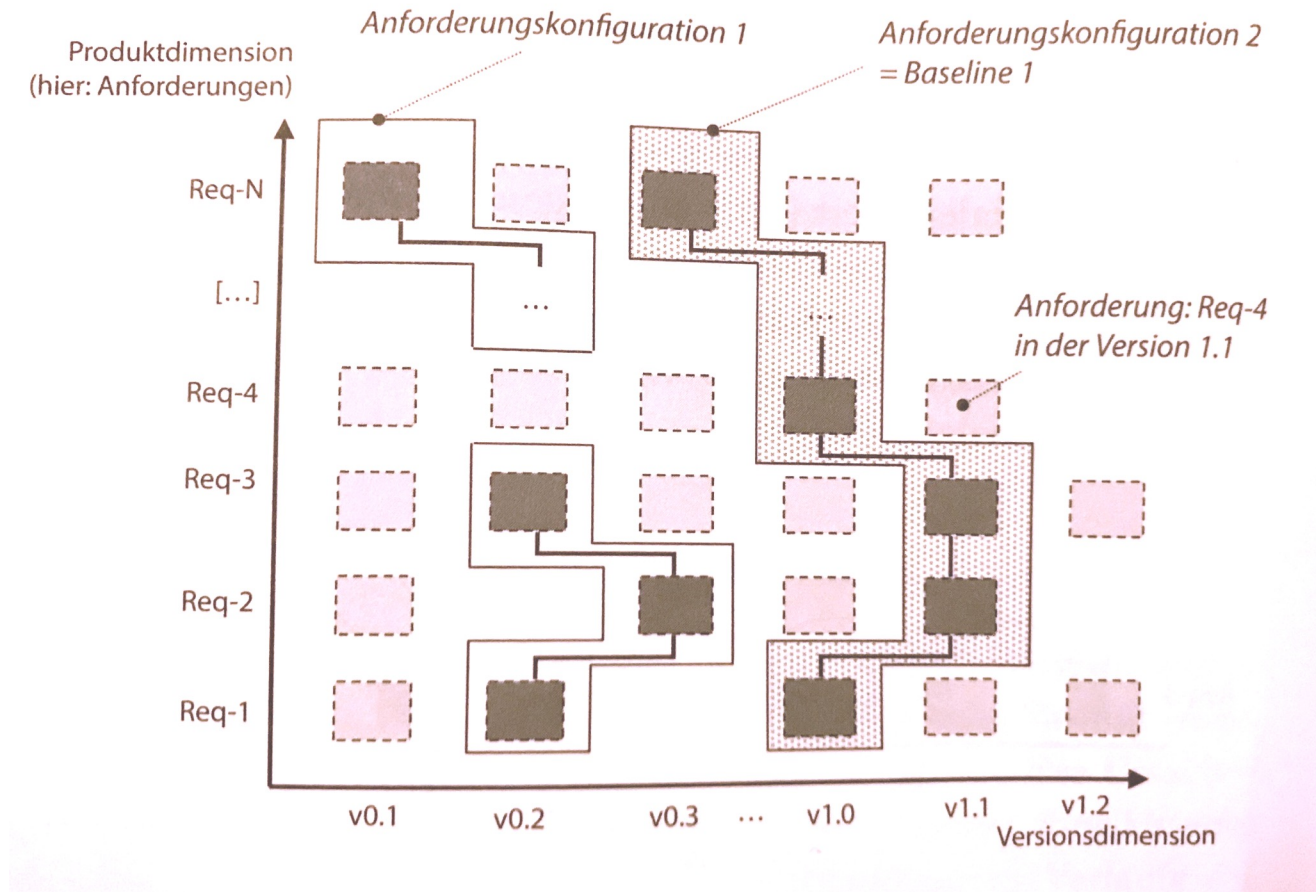
8. Anforderungen verwalten

Versionierung von Anforderungen

- Eine Anforderungskonfiguration besteht aus einer Menge von Anforderungen mit der zusätzlichen Bedingung, dass jede der ausgewählten Anforderungen in genau einer Version (identifiziert durch eine Versionsnummer) enthalten ist.
- Die Verwaltung von Anforderungskonfigurationen kann in zweierlei Hinsicht beschrieben werden:
 - In der Produktdimension betrachtet das Konfigurationsmanagement die einzelnen Anforderungen der Anforderungsbasis.
 - In der Versionsdimension berücksichtigt die Konfigurationsverwaltung die unterschiedlichen Änderungsstände der Anforderungen in der Produktdimension.

8. Anforderungen verwalten

Versionierung von Anforderungen



8. Anforderungen verwalten

Versionierung von Anforderungen

- Eine Anforderungskonfiguration muss nicht von jeder Anforderung eine Version enthalten.
- Eine Anforderungskonfiguration besitzt folgende Eigenschaften:
 - Sachlogischer Zusammenhang
 - Konsistenz
 - Eindeutige Identifikation
 - Nicht veränderbar
 - Grundlage für „Rücksetzen“

8. Anforderungen verwalten

Versionierung von Anforderungen

- Anforderungsbasislinien sind spezielle Konfigurationen von Anforderungen, die in der Regel stabile Versionen von Anforderungen umfassen und häufig auch Auslieferungsstufen des Systems definieren.
- Anforderungsbasislinien unterstützen eine Reihe wichtiger Tätigkeiten:
 - Als Grundlage zur Planung von Auslieferungsstufen
 - Zur Abschätzung des Realisierungsaufwands
 - Zum Vergleichen mit Konkurrenzprodukten
- Die Auslieferungsstufe eines Systems wird häufig auch als Systemrelease bezeichnet. Die Begriffe Anforderungsrelease und Anforderungsbasislinie werden in der Praxis dabei häufig synonym verwendet.

8. Anforderungen verwalten

Verwaltung von Anforderungsänderungen

- Es gibt unterschiedliche Ursachen für Änderungen in den Anforderungen (bspw. geänderte Nutzungswünsche seitens der Stakeholder).
- Änderungen von Anforderungen sind nicht unbedingt negativ, sondern ein Zeichen dafür, dass sich die Stakeholder kritisch mit dem System auseinandersetzen.
- Nehmen die Anforderungsänderungen jedoch überhand, so wird die Entwicklung eines mit den relevanten Stakeholdern abgestimmten Systems nahezu unmöglich.
- Daher ist es notwendig, Änderungsanträge zu kanalisieren und in einem systematischen Prozess eine begründete Entscheidung bezüglich der Änderung herbeizuführen.

8. Anforderungen verwalten

Verwaltung von Anforderungsänderungen

- Die Entscheidungsfindung sollte von einem Change-Control Board (CCB) vorgenommen werden. Das CCB hat typischerweise folgende Verantwortlichkeiten:
 - Bestimmung des Aufwands zur Umsetzung eines Änderungsantrags
 - Beurteilung der Änderungsanträge (z.B. im Hinblick auf der Verhältnis von Aufwand zum Nutzen)
 - Definition von Anforderungsänderungen bzw. Definition neuer Anforderungen auf Grundlage der Änderungsanträge
 - Entscheidung über Annahme / Ablehnung eines Änderungsantrags
 - Klassifikation eingehender Änderungsanträge
 - Priorisierung der angenommenen Anforderungsänderungen
 - Zuordnung der angenommenen Änderungsanträge zu Änderungsprojekten

8. Anforderungen verwalten

Verwaltung von Anforderungsänderungen

- Folgende Stakeholder sollten im CCB vertreten sein:
 - Änderungsmanager
 - Auftraggeber
 - Architekt
 - Entwickler
 - Konfigurationsmanager
 - Kundenvertreter / Nutzervertreter
 - Produktmanager
 - Projektmanager
 - Qualitätsbeauftragter
 - Requirements Engineer

- Der Änderungsmanager ist für die Kommunikation und Dokumentation der Entscheidungen verantwortlich.

8. Anforderungen verwalten

Verwaltung von Anforderungsänderungen

- Ein Änderungsantrag (Change Request) dokumentiert die gewünschte Änderung und enthält zusätzliche Informationen zur Verwaltung des Änderungsantrages.
- Ein Änderungsantrag sollte die folgenden Informationen enthalten:
 - Identifikator
 - Titel
 - Beschreibung
 - Begründung
 - Datum
 - Antragsteller
 - Priorität (aus Sicht der Antragstellers)

8. Anforderungen verwalten

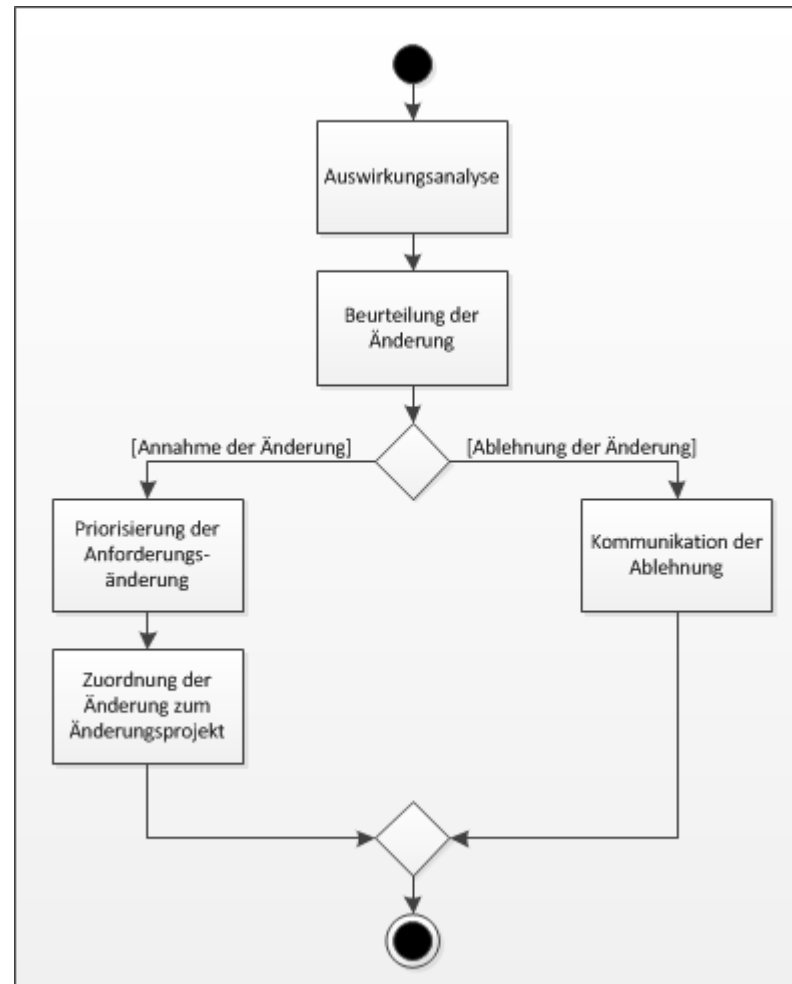
Verwaltung von Anforderungsänderungen

- Darüber hinaus können folgende Informationen hilfreich sein:
 - Prüfer der Änderung
 - Status Auswirkungsanalyse
 - Status Entscheidung CCB
 - Priorität CCB
 - Verantwortlicher
 - Systemrelease

- Ein Änderungsantrag kann wie folgt klassifiziert werden:
 - Korrektive Anforderungsänderung
 - Adaptive Anforderungsänderung
 - Ausnahmeänderung („Hotfix“) – i.e., korrektiv oder adaptiv

8. Anforderungen verwalten

Verwaltung von Anforderungsänderungen



8. Anforderungen verwalten

Zusammenfassung

- Attributierung von Anforderungen
- Sichten auf Anforderungen
- Priorisierung von Anforderungen
- Verfolgbarkeit von Anforderungen
- Versionierung von Anforderungen
- Verwaltung von Anforderungsänderungen

Agenda

1. Einleitung und Grundlagen
2. System und Systemkontext abgrenzen
3. Anforderungen ermitteln
4. Anforderungen dokumentieren
5. Anforderungen natürlichsprachig dokumentieren
6. Anforderungen modellbasiert dokumentieren
7. Anforderungen prüfen und abstimmen
8. Anforderungen verwalten
9. **Werkzeugunterstützung**

9. Werkzeugunterstützung

Allgemeine Werkzeugunterstützung

- Die verschiedenen Aktivitäten im Requirements Engineering sollten durch geeignete Werkzeuge unterstützt werden.
- Eine Vielzahl von Werkzeugen, die zur Systementwicklung und -wartung eingesetzt wird, kann auch im Requirements Engineering eingesetzt werden:
 - Testverwaltungswerkzeuge
 - Fehlerverfolgungswerkzeuge
 - Konfigurationsmanagementwerkzeuge
- Ein Vorteil bei diesen Werkzeugen kann eine direkte Verknüpfung sein (bspw. für Testfälle).

9. Werkzeugunterstützung

Allgemeine Werkzeugunterstützung

- Auch Wiki-Technologien werden mittlerweile eingesetzt, bspw. um Glossare aufzubauen oder um gemeinsam Anforderungen zu erarbeiten.
- Auch Werkzeuge wie etwa Visualisierungsprogramme können eingesetzt werden.
- Standardmäßig finden natürlich auch die gängigen Office-Anwendungen und Modellierungswerkzeuge Einsatz.
- Letztlich existieren auch Requirements Management Werkzeuge (z.B. Rational DOORS).

Modellierungswerkzeuge

- Modellierungswerkzeuge unterstützen die modellbasierte Dokumentation von Anforderungen und bieten meistens auch eine syntaktische Analyse dieser Modelle.
- Kriterien für die Auswahl eines Modellierungswerkzeugs:
 - Eindeutige ID-Vergabe
 - Verfolgbarkeit zwischen den verschiedenen Modellelementen
 - Mehrbenutzerzugriff
 - Versionsverwaltung
 - Einfache Integration in andere Werkzeuge (insbesondere im Hinblick auf Änderungen)

Requirements Management Werkzeuge

- Requirements Management Werkzeuge dienen dem Erfassen und Verwalten von Anforderungen.
- Funktionen, die ein Requirements Management Werkzeug besitzen sollte:
 - Verwalten verschiedener Informationen
 - Verwalten von logischen Beziehungen zwischen verschiedenen Informationen
 - Eindeutige Identifizierbarkeit
 - Bearbeiten der verwalteten Informationen
 - Bilden von unterschiedlichen Sichten
 - Organisieren der verwalteten Informationen
 - Erstellen von Reports und Auswertungen
 - Generieren von Ergebnisdokumenten

9. Werkzeugunterstützung

Werkzeugeinführung

- Die Einführung eines Werkzeuges sollte wie nachstehend dargestellt realisiert werden:
 - Verantwortlichkeiten schaffen
 - Werkzeug auswählen („Das Werkzeug folgt der Methode“)
 - Benötigte Ressourcen planen
 - Pilotprojekt initiieren
 - Werkzeug evaluieren
 - (Nachhaltige) Kosten kalkulieren
 - Benutzer schulen

9. Werkzeugunterstützung

Beurteilung von Werkzeugen

- Die Beurteilung eines Werkzeuges sollte aus unterschiedlichen Perspektiven erfolgen:
 - Projektsicht
 - Benutzersicht
 - Produktsicht
 - Prozesssicht
 - Anbietersicht
 - Technische Sicht
 - Betriebswirtschaftliche Sicht

9. Werkzeugunterstützung

Zusammenfassung

- Das Requirements Engineering kann durch zahlreiche Werkzeuge unterstützt werden.
- Die Auswahl der Werkzeuge sollte anhand bestimmter Kriterien erfolgen.
- Die Einführung von Werkzeugen sollte einem abgestimmten Prozess folgen.
- Die Beurteilung von Werkzeugen sollte unterschiedliche Sichten berücksichtigen.